TechBridge
INSTITUTE OF TECHNOLOGY

OFFICIAL CURRICULUM • MASTER REFERENCE NOTES

HTML5 & CSS3

Complete Developer Handbook

A comprehensive engineering guide covering Web Architecture, Semantic HTML5 Elements, Media, Modern Forms, CSS Selectors, Specificity, Box Model, Flexbox, Grid, and Modern Responsive Layouts.

◆ HTML5 Core & Semantics

Elements, tags, attributes, tables, forms, audio/video players, links & DOM tree hierarchy.

◆ Advanced Selectors

Combinators, attribute selectors, pseudo-classes (:nth-child) & pseudo-elements (::before, ::after).

◆ Layout & Box Model

Content, padding, border, margin calculations, box-sizing, and typography systems.

◆ Responsive Architecture

Modern Flexbox containers, CSS Grid systems, viewport units, and multi-device media queries.

TechBridge Learning Systems • Full-Stack Engineering Program
www.techbridge.in • Edition 2026

TABLE OF CONTENTS

Topics	Page no
Introduction to HTML	1
History of HTML	2
Structure of HTML	4
Basic HTML Tags	6
Formatting Tags	14
Links	22
Table	28
List	37
Iframe	41
Form	42
Introduction to CSS	54
Basic CSS properties	55
CSS Selectors	61
Basic Selectors	61
Advanced Selectors	77
Pseudo Classes	101
Pseudo Elements	124
Specificity	131
Inheritance	137
Typography	140
Box-Model	149
Flex-Box	156
CSS Units	162
Grid	166
Responsiveness in CSS	168
Summary	171

INTRODUCTION TO HTML

➤ HTML is used to display the content in the browser.
➤ HTML is an interpreted language where each and every statement will be executed line by line.
➤ HTML stands for **Hypertext Markup Language.**

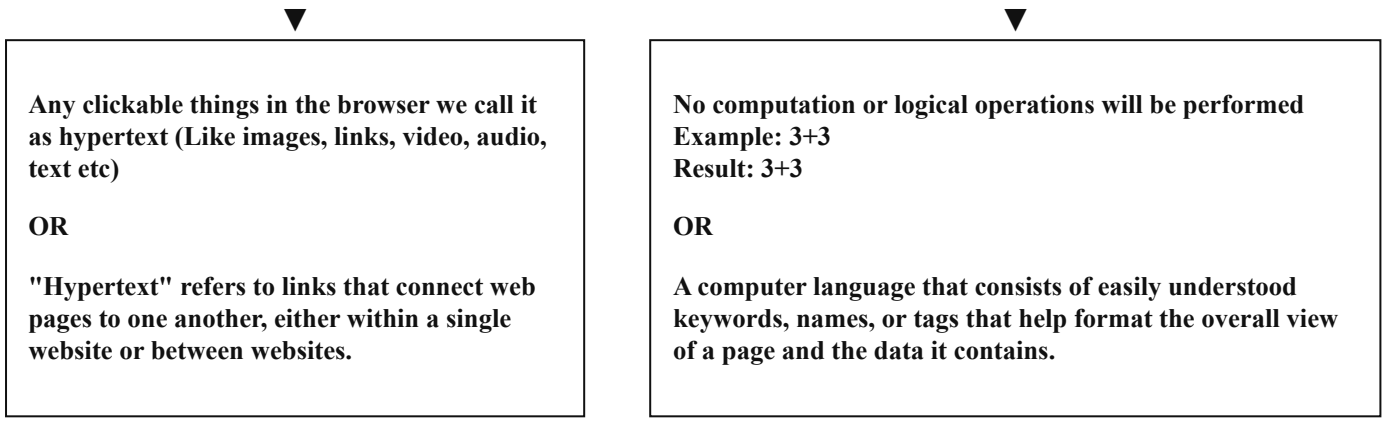


Figure 1: The two readings of "Hypertext Markup Language" (left arrow from "Hypertext", right arrow from "Markup Language")

Why HTML was introduced?

➤ HTML which stands for **H**ypertext **M**arkup **L**anguage was invented or created by **Tim Berners-Lee** who is also known as 'father of web'.
➤ At that time, he was a physicist working at European Laboratory for Particle Physics in Geneva, Switzerland.
➤ He wanted to create a way for researchers so that they would be able to use the internet to collaborate and share information.
➤ Instead of data being stored on single computer, he wanted a system by which people could link all their data together and enable people to be able to search by following links. So, he created HTML as a means for being able to convert any document on a simple word processor into a file viewable on internet and thus available to anybody with an internet connection.

HISTORY OF HTML

➤ Tim Berners-Lee, a physicist at the CERN research institute in Switzerland invented HTML in 1991.
➤ 1991- Tim Berners-Lee invents HTML 1.0
➤ 1993- HTML 1.0 is released.
➤ 1995- HTML 2.0 is published.
➤ 1997- HTML 3.0 was invented.
➤ 1999- The widely-used HTML 4.0
➤ 2014- HTML 5.0 is released and used worldwide.

Types of Applications

➤ Mobile Application (Calculator)
➤ Desktop Application (Eclipse, MySQL Workbench)
➤ Web Application (Gmail, Facebook etc)

Note: Anything that is display in the browser by using an internet we call as website or web application.

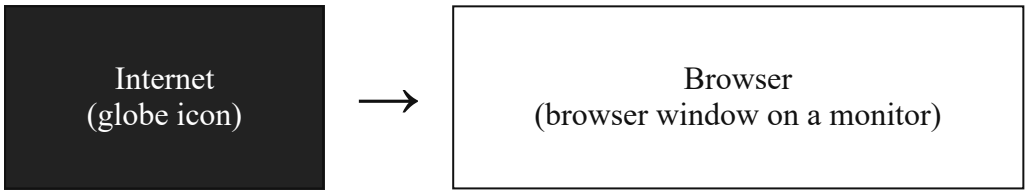


Figure 2: Internet → Browser

➤ **What is Web?**
A WWW or W3 is also known as a web, it is a system of interconnected public webpages accessible through the internet.
➤ **What is Web Page?**
A web page is a simple document displayable by a browser.
➤ **What is Website?**
A website is a collection of many web pages, and web pages are digital files that are written using HTML.

➤ **What is Web Application?**
A web application (or web app) is application software that is accessed using a web browser.
➤ **What is the difference between website and web application?**

Web Application	Website
In a web application we can interact with the users.	In a website we cannot interact with the users because we will have a static content.
In web application we can read and manipulate the contents.	In website we can only read the contents but we cannot manipulate the contents.
Example: Amazon etc	Example: TechBridge website

Note: If we want to communicate with the browser by using the HTML, we need to use the TAG.

What is tag?

A tag is a keyword that holds the contents that should be displayed in the browser in webpage in the proper format. It will be enclosed within the angle brackets (<>).

Example: <html>, <head>, <body> etc.

Types of tags

1. **Paired tag:** Paired tags are such tags where you need to start it and you need to end it as well by using the forward slash as:
   <html>--------</html> etc.
2. **Unpaired tag:** Unpaired tags are such tags where you need to start it but you no need to end such as:
   **
 etc.**

STRUCTURE OF HTML

First_page.html

```
<!DOCTYPE html>                          1
<html>                                   2
  <head>                                 4
    <title>First_page</title>            6
  </head>                                5
  <body>                                 7
    <p>Hi, welcome to HTML </p>          9
  </body>                                8
</html>                                  3
```

Output:

First_page | 127.0.0.1:5500/First_page.html

Hi, welcome to HTML

Note: In the about statements the numbering for each statement according to the execution flow.

<!DOCTYPE html>: DOCTYPE HTML is a declaration that tells the browser what version of HTML the document is written in.

<html>: The <html> tag is a paired so we need to start it and we need to end it as well and before we start writing statements in html, we need to tell the browser that we are starting with html by using this <html> tag.

<head>: The <head> tag is used to define the head section where all the titles of the web page will be display and it is also paired tag once we start it, we need to end it as well.

<title>: The <title> tag is used to give the title for the web page and it is also a paired tag once you start it you need to end it as well.

<body>: The <body> tag is used to give and display the contents inside the body of the webpage and it is also a paired tag once you start it you need to end it as well.

<p>: The <p> tag is a paragraph tag which is used to display the character, word, text, sentence or paragraph in the web page, its stand for paragraph and it is also a paired tag where once you start it you need to end it as well.

What is an Element?



Figure 3: Anatomy of an element (bracket "Tag" spans ... ; bracket "Content" spans the text between them)

The tag+content we call it as element.

BASIC HTML TAGS

- <p>
- <pre>
-
- <hr>
- Heading tags
  - <h1>
  - <h2>
  - <h3>
  - <h4>
  - <h5>
  - <h6>
- <img>
- <video>
- <audio>
- <a>

➤ **<p>:** The <p> is a paragraph tag which is used to display the character, word, text, sentence or paragraph in the web page and it is a paired tag. It will occupy the entire width of your screen to display the paragraph and then it will break the paragraph to next line.

Example:

```
<!DOCTYPE html>

<html>
<head>
  <title>Paragraph_tag</title>
</head>
<body>
    <p>One of the Unique features of TechBridge is the use of Augmented
    Reality in our training. Through this concepts are visualized using
    cutting-edge animation to ensure an in-depth understanding of the
    concepts & experience the pure joy of learning as well as upskilling your
    coding skills
    </p>
</body>
</html>
```

Output:

Paragraph_tag | 127.0.0.1:5500/paragraph.html

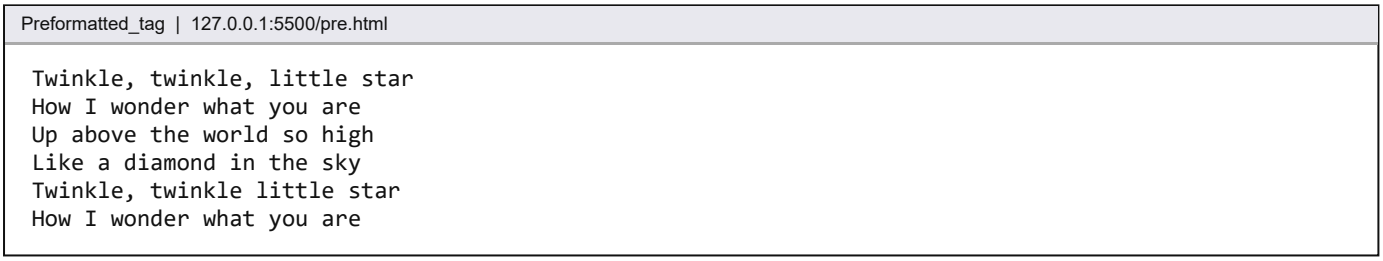
One of the Unique features of TechBridge is the use of Augmented Reality in our training. Through this concepts are visualized using cutting-edge animation to ensure an in-depth understanding of the concepts & experience the pure joy of learning as well as upskilling your coding skills

➤ **<pre>:** The <pre> is a preformatted tag which is used to display the character, word, text, sentence or paragraph in the webpage same as given in statement(code). This type of tag is used most to display the poems, short statements etc.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Preformatted_tag</title>
</head>
<body>
  <pre>
  Twinkle, twinkle, little star
  How I wonder what you are
  Up above the world so high
  Like a diamond in the sky
  Twinkle, twinkle little star
  How I wonder what you are
  </pre>
</body>
</html>
```

Output:



➤ **
:** The
 tag stands for break and it is used to break the link in between the paragraph and it is an unpaired tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Break_tag</title>
</head>
<body>
  <p>
One of the Unique features of TechBridge is the use of   <br>Augmented
Reality in our training.Through this concepts are visualized using cutting-
edge animation<br> to ensure an in-depth understanding of the concepts &
experience the pure joy<br> of learning as well as upskilling your coding skills
  </p>
</body>
</html>
```

Note: In the above example we are using a paragraph tag so it will occupy the entire width of the screen to display the paragraph, but we are using the
 tag to explicitly break the line as per our requirement.

Output:

Break_tag | 127.0.0.1:5500/br.html

One of the Unique features of TechBridge is the use of
Augmented Reality in our training.Through this concepts are visualized using cutting-edge animation
to ensure an in-depth understanding of the concepts & experience the pure joy
of learning as well as upskilling your coding skills

➤ **<hr>:** The <hr> tag stands for horizontal rule and it is used to give one horizontal in between the contents in the webpage to differentiate between the contents and it is an unpaired tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>horizontal_rule_tag</title>
</head>
<body>
  <p>
     One of the Unique features of TechBridge is the use of Augmented
     Reality in our training.  Through this concepts are visualized
     using cutting-edge animation to ensure an in-depth understanding of
     the concepts & experience the pure joy of learning as well as upskilling
     your coding skills
  </p>
  <hr>
  <pre>
    Twinkle, twinkle, little star
    How I wonder what you are
    Up above the world so high
    Like a diamond in the sky
    Twinkle, twinkle little star
    How I wonder what you are
  </pre>
</body>
</html>
```

Output:

horizontal_rule_tag | 127.0.0.1:5500/hr.html

One of the Unique features of TechBridge is the use of Augmented Reality in our training. Through this concepts are visualized using cutting-edge animation to ensure an in-depth understanding of the concepts & experience the pure joy of learning as well as upskilling your coding skills

---

```
Twinkle, twinkle, little star
How I wonder what you are
Up above the world so high
Like a diamond in the sky
Twinkle, twinkle little star
How I wonder what you are
```

➤ **heading:** The heading is used to display the headings in the webpage. There are 6 types of heading tags such as: <h1>, <h2>, <h3>, <h4>, <h5>, <h6>. All these tags are differentiated by the size. The most

used heading tag is <h1> tag and least used heading tag is <h6> tag. All these tags are paired tags.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Heading_tags</title>
</head>
<body>
  <h1>This is H1 tag</h1>
  <h2>This is H2 tag</h2>
  <h3>This is H3 tag</h3>
  <h4>This is H4 tag</h4>
  <h5>This is H5 tag</h5>
  <h6>This is H6 tag</h6>
</body>
</html>
```

Output:

Heading_tags | 127.0.0.1:5500/heading.html

This is H1 tag

This is H2 tag

This is H3 tag

This is H4 tag

This is H5 tag

This is H6 tag

➤ **<img>:** The <img> tag stands for image and it is used to display the image in the webpage. It has some main attributes such as:

- ➤ **src:** It stands for source where the name of the image should be specified.
- ➤ **alt:** It stand for alternate which is used to display any message if the image is not displayed in the webpage.
- ➤ **height:** It is used to set the proper height of the image to display in the webpage.

- **width:** It is used to set the proper width of the image to display in the webpage.

Alternate where the message is given to display if image is not working.

```
<img src="techbridge_logo.svg" alt="Image not found">
```

Source where the name of the image is specified.

Figure 4: Attributes of the <img> tag (src = source where the name of the image is specified; alt = alternate message displayed if the image is not working)

Note: HTML attributes are used to describe the characteristic of an HTML element in detail (provide more information about the tag).

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Image_tag</title>
</head>
<body>
  <img src="techbridge_logo.svg" alt="Image not found">
</body>
</html>
```

Note: image format can be anything like png or jpg etc, but make sure your browser supports that format and height and width are optional if required use it and set them accordingly. And make sure that image should be present in the current folder in which you're working.

Output:



Figure 5: Browser output showing the TechBridge logo image

➤ **<audio>:** The <audio> tag is used to display the audio in the webpage. It has some attributes such as:
  ➤ **src:** It stands for source where the name of the audio should be specified.
  ➤ **controls:** It is used to display all the controls of the audio such as play, pause etc.

Controls to display the controls of audio like play, pause.

```
<audio src="aud.mp3" controls></audio>
```

Source where the name of the audio is specified.

Figure 6: Attributes of the <audio> tag

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Audio_tag</title>
</head>
<body>
  <audio src="aud.mp3" controls></audio>
</body>
</html>
```

Note: audio format can be anything like png etc but make sure your browser supports. And make sure that audio should be present in the current folder in which you're working. You can used height and width attributes if its required.

Output:

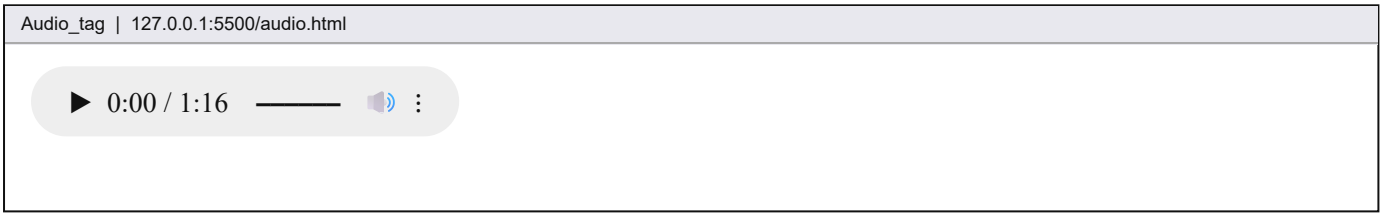


Figure 7: Browser output of the <audio> tag (audio player showing play button, 0:00 / 1:16, volume and menu)

➤ **<video>:** The <video> tag is used to display the video in the webpage. It has some attributes such as:
- ➤ **src:** It stands for source where the name of the video should be specified.
- ➤ **controls:** It is used to display all the controls of the video such as play, pause etc.

Source where the name of the video is specified.

width is used to set the width of the video.

```
<video src="LAXMI KHOT 1.mp4" controls width="50%" height="160px" </video>
```

Controls to display the controls of audio like play, pause.

height is used to set the height of the video.

Figure 8: Attributes of the <video> tag (src, controls, width, height)

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Video_tag</title>
</head>
<body>
  <video src="LAXMI KHOT 1.mp4" controls width="50%"
height="160px"></video>
</body>
</html>
```

Note: video format can be anything like mp4 etc but make sure your browser supports. And make sure that video should be present in the current folder in which you're working. You can used height and width attributes if its required.

Output:



Figure 9: Browser output of the <video> tag (video player showing 0:00 / 1:24)

FORMATTING TAGS

HTML provides us ability to format text without using CSS. By using the below formatting tags.

- <u>
- <b>
- <i>
- <mark>
- <sub>
- <sup>
- <small>
- <big>
- <s>

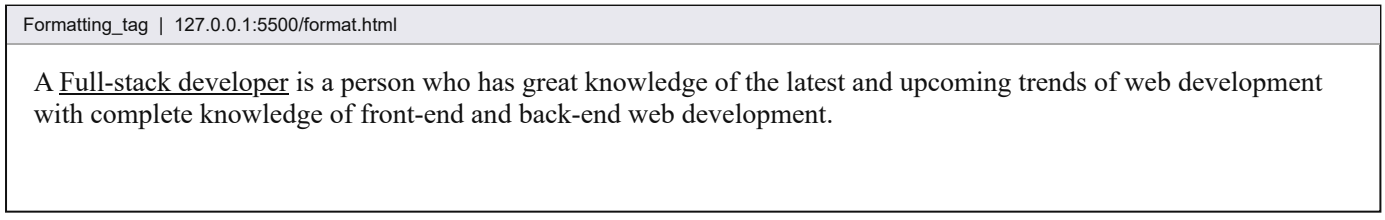
➤ **<u>:** The <u> tag stands for underline, it is used to underline the character, word or sentence.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A <u>Full-stack developer</u> is a person who has great
  knowledge of the latest and upcoming trends of web development
  With complete knowledge of front-end and back-end web development.
  </p>
</body>
</html>
```

In the above example we are taking one paragraph and in that we are underlining the Full-stack developer.

Output:



➤ **<b>:** The <b> tag stands for bold, it is used to bold the character, word or sentence.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has <b>great knowledge</b> of the
  latest and upcoming trends of web development with complete knowledge of
  front-end and back-end web development.
  </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has **great knowledge** of the latest and upcoming trends of web development with complete knowledge of front-end and back-end web development.

➤ **<i>:** The <i> tag stands for italic, it is used to change the font style to italic of the character, word or sentence.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has great knowledge
      of the latest and upcoming trends of <i>web development</i> with
  complete knowledge of front-end and back-end web development.
</p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of *web development* with complete knowledge of front-end and back-end web development.

➤ **<mark>:** The <mark> is used to highlight the character, word or sentence by default the background color that will be displayed in yellow color.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has great knowledge
    of the latest and upcoming trends of web development with
    complete <mark>knowledge</mark> of front-end and back-end web
    development.
  </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and back-end web development.

➤ **<sub>:** The <sub> tag stands for subscript it is used to display the subscript character, word or sentences as shown below:

front-end and $_{\text{back-end}}$ web development.

↓

Subscript text or word

Figure 10: Subscript text (arrow points to "back-end")

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has great knowledge
      of the latest and upcoming trends of web development with
      complete knowledge of front-end and <sub>back-end</sub> web
      development.
  </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and $_{\text{back-end}}$ web development.

➤ **<sup>:** The <sup> tag stands for superscript it is used to display the superscript character, word or sentences as shown below:

Subscript text
or word
↑

front-end and $^{\text{back-end}}$ web development.

Figure 11: Superscript text (arrow points to "back-end"; the label is printed as "Subscript text or word" in the source)

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has great knowledge
     of the latest and upcoming trends of web development with
     complete knowledge of front-end and <sup>back-end</sup>web
     development.
  </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and $^{\text{back-end}}$ web development.

➤ **<small>:** The <small> tag is used to display the character, word or sentences in small size compared to others as shown below:

knowledge of the latest and upcoming trends

↓

This is small text compared
to other words in sentence.

Figure 12: Small text (arrow points to "upcoming trends")

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
  <p>A Full-stack developer is a person who has great knowledge
     of the latest and <small>upcoming</small> trends of web development
     with complete knowledge of front-end and back-end web development.
  </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and back-end web development.

➤ **<big>:** The <big> tag is used to display the character, word or sentences in big size compared to other words as shown below:

This is big text compared to
other words in sentence.

↑

a person who has great knowledge of the latest and upcoming trends

Figure 13: Big text (arrow points to "knowledge")

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
   <p>A Full-stack developer is a person who has great
   <big>knowledge</big> of the latest and upcoming trends of web
   development with complete knowledge of front-end and back-end web
   development.
   </p>
</body>
</html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a person who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and back-end web development.

➤ **<s>:** The <s> tag stands for strike and it is used to strike the character, word or as shown below:

The person word is strike

↑

developer is a ~~person~~ who has great knowledge

Figure 14: Strikethrough text (arrow points to "person")

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formatting_tag</title>
</head>
<body>
      <p>A Full-stack developer is a <s>person</s> who has great knowledge
      of the latest and upcoming trends of web development with complete
      knowledge of front-end and back-end web development.
      </p>
</body></html>
```

Output:

Formatting_tag | 127.0.0.1:5500/format.html

A Full-stack developer is a ~~person~~ who has great knowledge of the latest and upcoming trends of web development with complete knowledge of front-end and back-end web development.

LINKS

➤ Links are hyperlinks which can be clicked and redirect from one page to another page.
➤ Links can be anything like text, image or any other elements.
➤ If any text or image or any other element is link as soon as you keep your cursor on that it will change into little hand.
➤ To create a link, we need to use the <a> anchor tag.

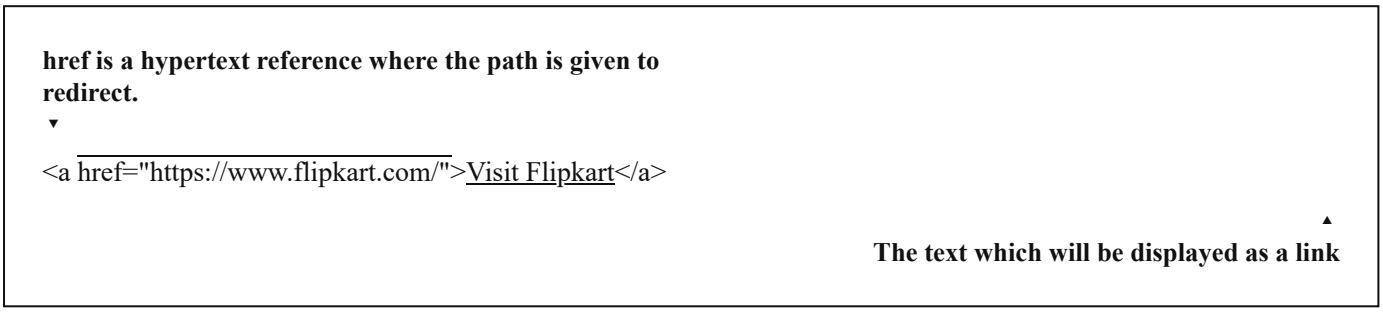


Figure 15: Anatomy of the anchor tag (href = hypertext reference where the path is given to redirect; "Visit Flipkart" = the text which will be displayed as a link)

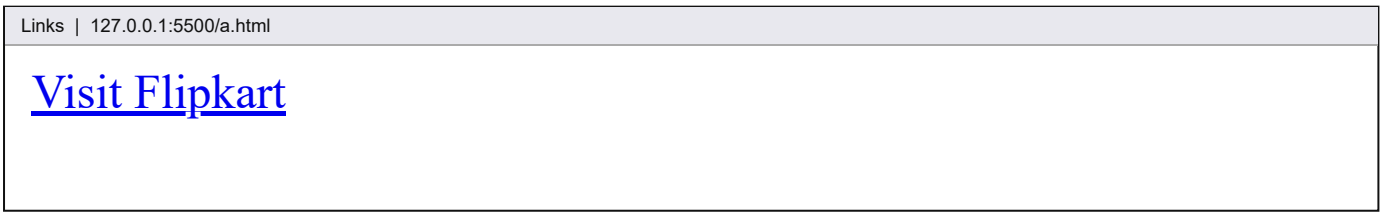
➤ The link has three default colors as follows:
  ➤ Unvisited----------Blue color
  ➤ Visited----------Purple color
  ➤ Active----------Red color

Example for Unvisited link:

```
<!DOCTYPE html>

<html>
<head>
  <title>Links</title>
</head>
<body>
  <a href="https://www.flipkart.com/">Visit Flipkart</a>
</body>
</html>
```

Output:



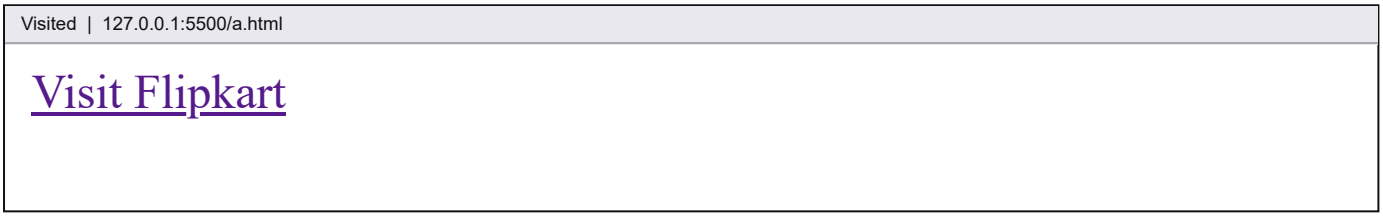
Note: The above link is unvisited hence it is displaying in blue color.

Example for Visited link:

```
<!DOCTYPE html>

<html>
<head>
  <title>Visited</title>
</head>
<body>
  <a href="https://www.flipkart.com/">Visit Flipkart</a>
</body>
</html>
```

Output:



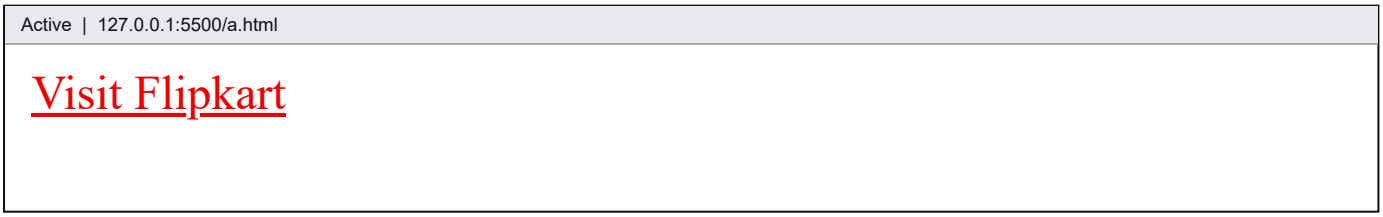
Note: The above link is visited hence it is displaying in purple color.

Example for Active link:

```
<!DOCTYPE html>

<html>
<head>
  <title>Active</title>
</head>
<body>
  <a href="https://www.flipkart.com/">Visit Flipkart</a>
</body>
</html>
```

Output:



Note:

- The above link is active hence it is displaying in red color.
- For the anchor tag we have target attribute which helps us to open the webpage in same page or in new tab:
  - Values: _blank will open the webpage in new tab.
    _self will open the webpage in the same tab only.
- By default, the link(webpage) will open in the same tab only.

Example for _self:

```
<!DOCTYPE html>
<html>
<head>
  <title>self</title>
</head>
<body>
  <a href="https://www.flipkart.com/" target="_self">Visit Flipkart</a>
</body>
</html>
```

Output:

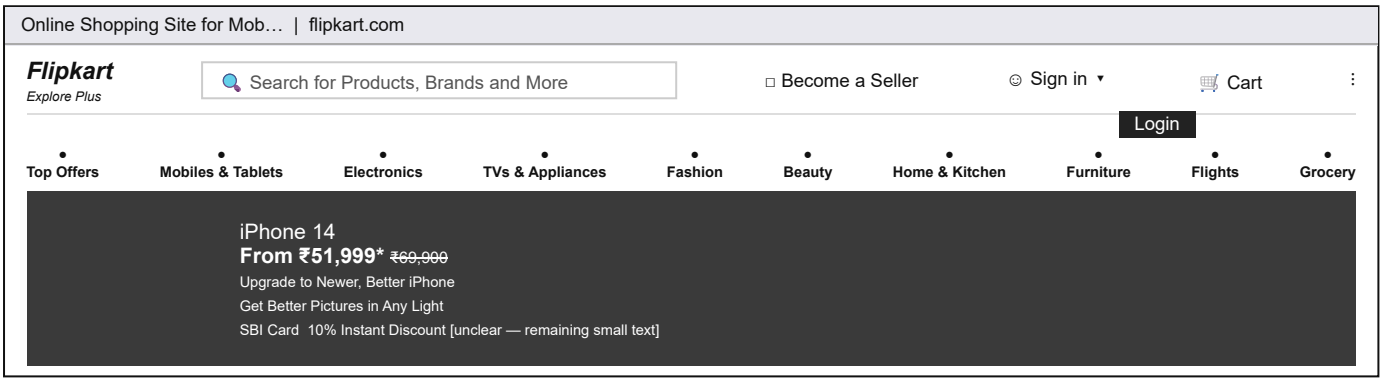


Figure 16: Browser output after clicking the link with target="_self" — the Flipkart home page opens in the same tab

Note: In the above example since we have used as _self value for target attribute the page is displaying in the same tab only.

Example for _blank:

```
<!DOCTYPE html>
<html>
<head>
  <title>self</title>
</head>
<body>
  <a href="https://www.flipkart.com/" target="_blank">Visit Flipkart</a>
</body>
</html>
```

Output

Figure 17: Browser output for target="_blank" — the Flipkart page opens in a new tab (the new tab is highlighted next to the original "self" tab)

Note: In the above example since we have used as _blank value for target attribute the page is displaying in the new tab.

Image as a Link Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Image_as_link</title>
</head>
<body>
<a href="https://www.techbridge.in/">
  <img src="techbridge_logo.svg" alt="TechBridge_logo">
</a>
</body>
</html>
```

Output:



Note: In the above example inside the anchor tag as a content we are giving image instead of text.

Mail as a Link Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mail</title>
</head>
<body>
  <a href="mailto:contact@techbridge.in"  target="_blank">
  Send Mail</a>
</body>
</html>
```

Output:

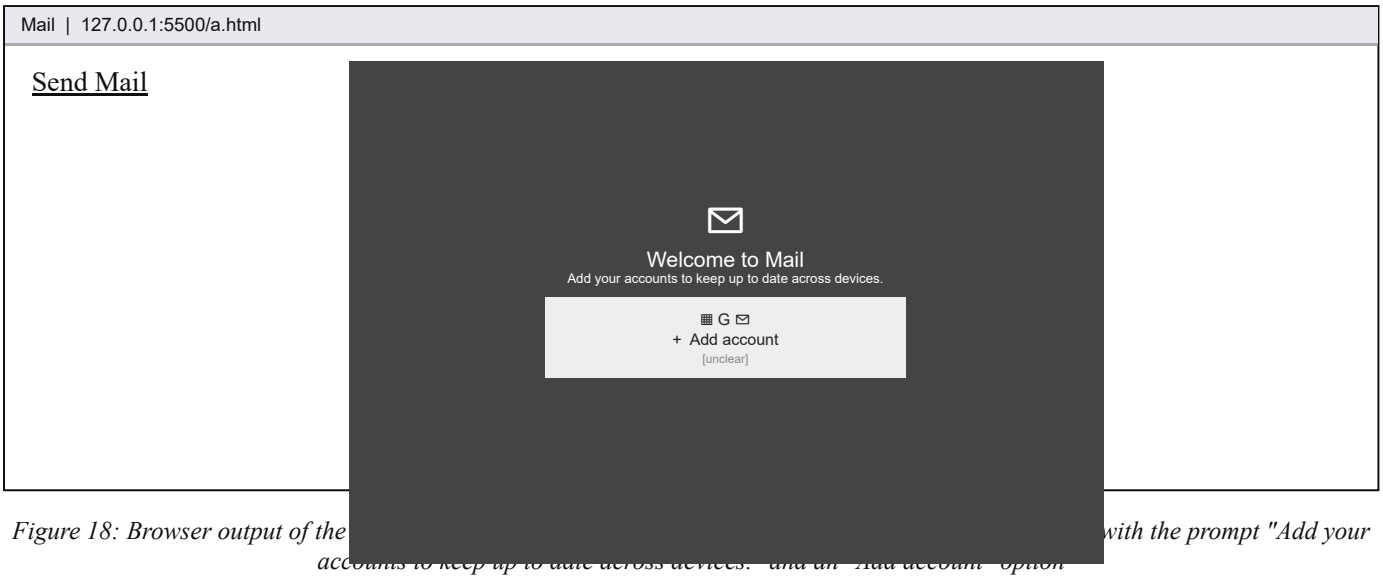
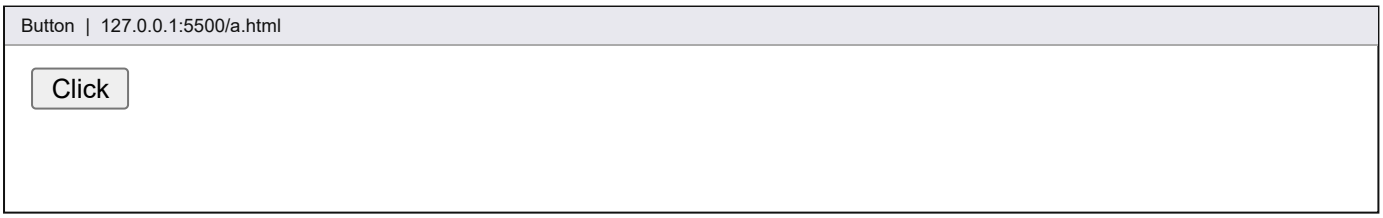


Figure 18: Browser output of the with the prompt "Add your accounts to keep up to date across devices." and an "Add account" option

Button as a Link Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Button</title>
</head>
<body>
  <a href="https://www.flipkart.com/" target="_blank">
    <button>Click</button>
  </a>
</body>
</html>
```

Output:



Note: In the href as a link we can give absolute url (Uniform Resource Locator) or relative url.

Absolute Url: An absolute URL contains all the information necessary to locate a resource.

Example: https://www.flipkart.com/

Relative Url: A relative URL typically contains only the path to a specific file.

Example: /images/photo.png

TABLE

- The table contains the rows and column in a webpage.
- We need to use the **<table>** tag to start creating the table.
- We need to use the **<tr>** tag which stands for table row, and it is used to specify the row in a table.
- We need to use the **<th>** tag which stands for table header, and it is used to specify the header(column) in a table.
- We need to use the **<td>** tag which stands for table data, and it is used to specify the table data in a table.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Table</title>
</head>
<body>
   <table>
      <tr>
        <th>Student_Id</th>
        <th>Student_Name</th>
        <th>Student_Age</th>
        <th>Student_Gender</th>
      </tr>
      <tr>
        <td>101</td>
        <td>Ravi</td>
        <td>23</td>
        <td>Male</td>
      </tr>
      <tr>
        <td>102</td>
        <td>Ramya</td>
        <td>23</td>
        <td>Female</td>
      </tr>
      <tr>
        <td>103</td>
        <td>Sam</td>
        <td>24</td>
        <td>Male</td>
      </tr>
      <tr>
        <td>104</td>
        <td>Roopa</td>
        <td>23</td>
        <td>Female</td>
      </tr>
      <tr>
        <td>105</td>
        <td>Ram</td>
        <td>23</td>
        <td>Male</td>
      </tr>
   </table>
</body>
</html>
```

Note: If we create a table just by specifying the row, header and table data as above example the result will be as shown below:

Table | 127.0.0.1:5500/table.html

Student_Id	Student_Name	Student_Age	Student_Gender
101	Ravi	23	Male
102	Ramya	23	Female
103	Sam	24	Male
104	Roopa	23	Female
105	Ram	23	Male

In order to get the proper table look we need to use **border attribute** in the table tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Table</title>
</head>
<body>
   <table border>
      <tr>
         <th>Student_Id</th>
         <th>Student_Name</th>
         <th>Student_Age</th>
         <th>Student_Gender</th>
      </tr>
      <tr>
        <td>101</td>
        <td>Ravi</td>
        <td>23</td>
        <td>Male</td>
      </tr>
      <tr>
         <td>102</td>
         <td>Ramya</td>
         <td>23</td>
         <td>Female</td>
      </tr>
      <tr>
         <td>103</td>
         <td>Sam</td>
         <td>24</td>
         <td>Male</td>
      </tr>
      <tr>
         <td>104</td>
         <td>Roopa</td>
         <td>23</td>
         <td>Female</td>
      </tr>
      <tr>
         <td>105</td>
         <td>Ram</td>
         <td>23</td>
         <td>Male</td>
      </tr>
   </table>
</body>
</html>
```

Note: If we create a table using the border attribute and specify the row, header and table data as above example the result will be as shown below:

Table | 127.0.0.1:5500/table.html

Student_Id	Student_Name	Student_Age	Student_Gender
101	Ravi	23	Male
102	Ramya	23	Female
103	Sam	24	Male
104	Roopa	23	Female
105	Ram	23	Male

Creation of vertical table Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Vertical_table</title>
</head>
<body>
  <table border>
    <tr>
      <th>Student_Id</th>
      <td>101</td>
      <td>102</td>
      <td>103</td>
      <td>104</td>
      <td>105</td>
    </tr>
    <tr>
       <th>Student_Name</th>
       <td>Ravi</td>
       <td>Ramya</td>
       <td>Sam</td>
       <td>Roopa</td>
       <td>Ram</td>
    </tr>
    <tr>
        <th>Student_Age</th>
        <td>23</td>
        <td>23</td>
        <td>24</td>
        <td>23</td>
        <td>23</td>
    </tr>
    <tr>
        <th>Student_Gender</th>
        <td>Male</td>
        <td>Female</td>
        <td>Male</td>
        <td>Female</td>
        <td>Male</td>
    </tr>
  </table>
</body>
</html>
```

Output:

Vertical_table | 127.0.0.1:5500/table1.html

Student_Id	101	102	103	104	105
Student_Name	Ravi	Ramya	Sam	Roopa	Ram
Student_Age	23	23	24	23	23
Student_Gender	Male	Female	Male	Female	Male

Note: We can provide the table caption as well by using the **<caption>** tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Vertical_table</title>
</head>
<body>
  <table border>
    <caption>Student_Details</caption>
    <tr>
      <th>Student_Id</th>
      <td>101</td>
      <td>102</td>
      <td>103</td>
      <td>104</td>
      <td>105</td>
    </tr>
    <tr>
      <th>Student_Name</th>
      <td>Ravi</td>
      <td>Ramya</td>
      <td>Sam</td>
      <td>Roopa</td>
      <td>Ram</td>
    </tr>
    <tr>
      <th>Student_Age</th>
      <td>23</td>
      <td>23</td>
      <td>24</td>
      <td>23</td>
      <td>23</td>
    </tr>
    <tr>
      <th>Student_Gender</th>
      <td>Male</td>
      <td>Female</td>
      <td>Male</td>
      <td>Female</td>
      <td>Male</td>
    </tr>
  </table>
</body>
</html>
```

Output:

Vertical_table | 127.0.0.1:5500/table1.html

Student_Details

Student_Id	101	102	103	104	105
Student_Name	Ravi	Ramya	Sam	Roopa	Ram
Student_Age	23	23	24	23	23
Student_Gender	Male	Female	Male	Female	Male

Example for merging multiple columns as one:

Note: We need to use the **colspan** attribute to merge the column and display as one.

```
<!DOCTYPE html>
<html>
<head>
  <title>colspan</title>
</head>
<body>
  <table border>
    <tr>
      <th colspan="2">Full Name</th>
      <th>Rating</th>
    </tr>
    <tr>
      <td>Ravi</td>
      <td>V</td>
      <td>7.8</td>
    </tr>
    <tr>
      <td>Ramya</td>
      <td>Suresh</td>
      <td>8</td>
    </tr>
    <tr>
      <td>Sachin</td>
      <td>Shekar</td>
      <td>6.7</td>
    </tr>
    <tr>
      <td>Smitha</td>
      <td>R</td>
      <td>9</td>
    </tr>
    <tr>
      <td>Divya</td>
      <td>D</td>
      <td>9</td>
    </tr>

  </table>
</body>
</html>
```

In the above example we have merged two columns as one by using the colspan attribute and by specifying the value as 2.

Output:

colspan | 127.0.0.1:5500/table3.html

Full Name		Rating
Ravi	V	7.8
Ramya	Suresh	8
Sachin	Shekar	6.7
Smitha	R	9
Divya	D	9

Example for merging multiple columns and rows as one:

Note: We need to use the **rowspan** attribute to merge the rows and display as one.

```
<!DOCTYPE html>
<html>
<head>
  <title>rowspan</title>
</head>
<body>
  <table border>
    <tr>
      <th rowspan="5">Full Name</th>
      <td>Ravi</td>
      <td>V</td>
    </tr>
    <tr>
      <td>Ramya</td>
      <td>Suresh</td>
    </tr>
    <tr>
      <td>Sachin</td>
      <td>Shekar</td>
    </tr>
    <tr>
      <td>Smitha</td>
      <td>R</td>
    </tr>
    <tr>
      <td>Divya</td>
      <td>D</td>
    </tr>
    <tr>
      <th rowspan="5">Rating</th>
      <td colspan="2">7.8</td>
    </tr>
    <tr>
      <td colspan="2">8</td>
    </tr>
    <tr>
      <td colspan="2">6.7</td>
    </tr>
    <tr>
      <td colspan="2">9</td>
    </tr>
    <tr>
      <td colspan="2">9</td>
    </tr>
  </table>
</body>
</html>
```

Output:

rowspan | 127.0.0.1:5500/table4.html

Full Name	Ravi	V
	Ramya	Suresh
	Sachin	Shekar
	Smitha	R
	Divya	D
Rating	7.8	
	8	
	6.7	
	9	
	9	

LIST

- In HTML a list will allow us to display the group of items as list.
- There are three types of list:
  - Ordered List
  - Unordered List
  - Description List

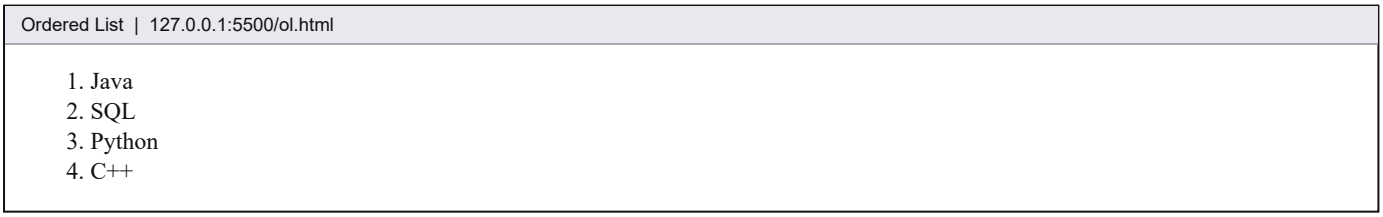
Ordered List

- We can create the ordered list by using the <ol> tag.
- We need to put the items inside the list by using the <li> tag.
- By default, the ordered list will be display as 1,2,3....

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Ordered List</title>
</head>
<body>
  <ol>
    <li>Java</li>
    <li>SQL</li>
    <li>Python</li>
    <li>C++</li>
  </ol>
</body>
</html>
```

Output:



Note: we can change the default numbering for ordered list by using the type attribute.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Ordered List</title>
</head>
<body>
  <ol type="A">
    <li>Java</li>
    <li>SQL</li>
    <li>Python</li>
    <li>C++</li>
  </ol>
</body>
</html>
```

Output:

Ordered List | 127.0.0.1:5500/ol.html

A. Java
B. SQL
C. Python
D. C++

Unordered List

➤ We can create the unordered list by using the <ul> tag.
➤ We need to put the items inside the list by using the <li> tag.
➤ By default, the unordered list will be displayed in bullets.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Unordered List</title>
</head>
<body>
 <ul>
 <li>HTML</li>
 <li>CSS</li>
 <li>JavaScript</li>
 <li>Bootstrap</li>
 </ul>
</body>
</html>
```

Output:

Unordered List | 127.0.0.1:5500/ul.html

- HTML
- CSS
- JavaScript
- Bootstrap

Note: In HTML we cannot change the default bullets to any other, if we want to do it then we need to apply the styling for that.

Description List

➤ We can create the description list by using the <dl> tag.
➤ We can give the description term by using the <dt> tag.
➤ We can give the definition for the description term by using the <dd> tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Description List</title>
</head>
<body>
  <dl>
    <dt>Java</dt>
    <dd>Java is a object oriented programming language</dd>
    <dt>SQL</dt>
    <dd>SQL stands for structured query language</dd>
    <dt>HTML</dt>
    <dd>HTML stands for Hypertext markup language</dd>
  </dl>
</body>
</html>
```

Output:

Description List | 127.0.0.1:5500/dl.html

Java
: Java is a object oriented programming language
SQL
: SQL stands for structured query language
HTML
: HTML stands for Hypertext markup language

IFRAME

➤ Iframe is used to display a webpage inside a webpage as embedded or nested.
➤ We can achieve this by using the <iframe> tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Iframe</title>
</head>
<body>
  <iframe src="https://www.flipkart.com/" height="250px"
width="100%"></iframe>
</body>
</html>
```

Output:

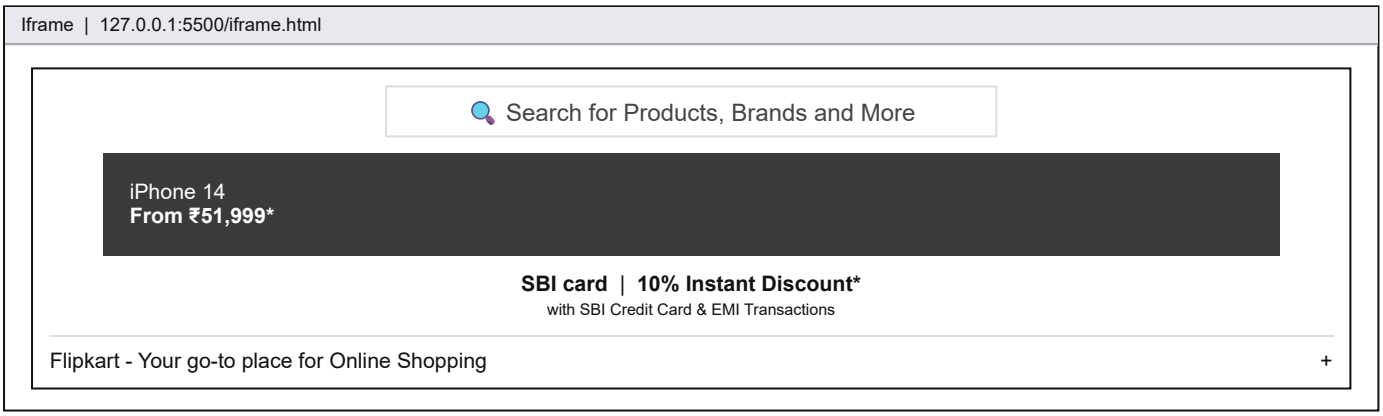


Figure 19: Browser output — the Flipkart page displayed inside the iframe (schematic reproduction)

In the above example we are embedding the flipkart website inside our webpage by using the <iframe> tag, hence the entire website we can go through it in our webpage only and we can set the proper height and width of the website.

FORM

➤ The form is used to collect the data or information from the user side.
➤ We can create a form by using the <form> tag.
➤ Form has the structure as shown below:

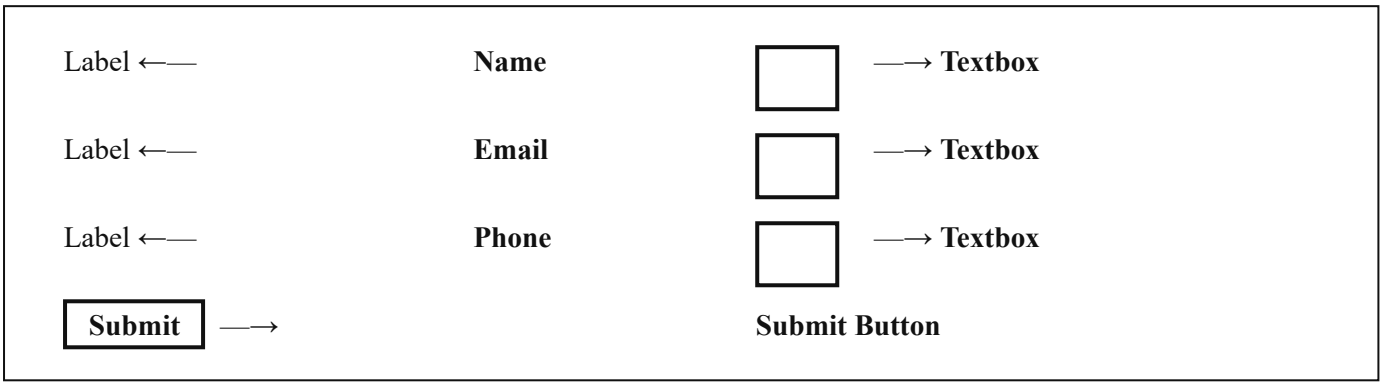
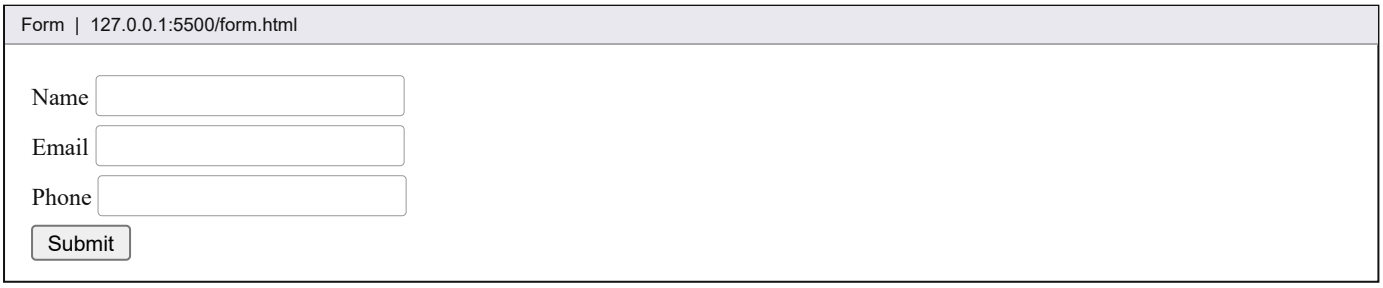


Figure 20: Structure of a form (Label → Name/Email/Phone, each with a Textbox; Submit → Submit Button)

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Form</title>
</head>
<body>
  <form>
    <label>Name</label>
    <input type="text"><br><br>
    <label>Email</label>
    <input type="text"><br><br>
    <label>Phone</label>
    <input type="text"><br><br>
    <input type="submit">
  </form>
</body></html>
```

Output:



Note:

- To make the text fields mandatory or important to fill we can use the **required attribute**.

- Once we click on the submit button if every data is correct, we can perform any action by using the **action attribute.**
- To identify the textbox uniquely we can use the **id attribute** and assign the value for it and by keeping that value we can identify them uniquely.
- To tell the textbox is to created to which label we can use the **for attribute** in that we can give the **id value** so that it will map it properly.
- According to the input we can change the type of the input box.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Form</title>
</head>
<body>
 <form action="https://www.www.techbridge.in">
   <label for="name">Name</label>
   <input type="text" required id="name"><br><br>
   <label for="email">Email</label>
   <input type="email" required id="email"><br><br>
   <label for="phone">Phone</label>
   <input type="text" required id="phone"><br><br>
   <input type="submit">
 </form>
</body>
</html>
```

Output:



Figure 21: Browser output when the required "Name" field is left empty — the message "Please fill out this field." appears

- You can see in the above example once we give as required if we don't fill that field, it will be displaying a message as "Please fill out this field".
- Once all the fields are filled properly then only you can submit the form and you can perform the given option.

Creating the Checkbox in Form

➤ In the forms usually we use a checkbox button to allow only **multiple option** to click like **Skills, Course obtained etc.**

➤ To create the checkbox the input type we need to give as **checkbox.**

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Checkbox</title>
</head>
<body>
  <form action="https://www.www.techbridge.in">
    <label>Skills</label><br><br>
    <input type="checkbox" required id="Java">
    <label for="Java">Java</label><br><br>
    <input type="checkbox" required id="SQL">
    <label for="SQL">SQL</label><br><br>
    <input type="checkbox" required id="HTML">
    <label for="HTML">HTML</label><br><br>
    <input type="checkbox" required id="CSS">
    <label for="CSS">CSS</label><br><br>
    <input type="checkbox" required id="JavaScript">
    <label for="JavaScript">JavaScript</label><br><br>
    <input type="submit">
</form></body></html>
```

Output:



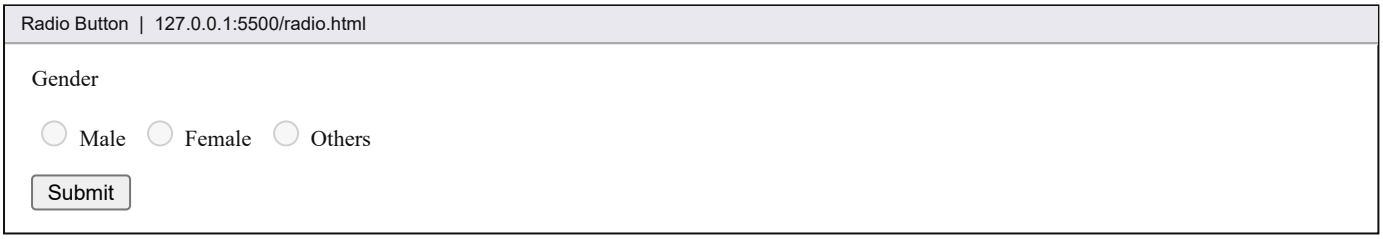
Creating the Radio Buttons in Form

- ➤ In the forms usually we use a radio button to allow only **one option** to click like **gender, department** etc.
- ➤ To create the radio button the input type we need to give as **radio.**
- ➤ When ever we are creating the radio buttons if it should allow only one option in the all the provided options, we need to **group it as one**, for this we need to use the name attribute for every text box and value of that **name attribute** should be same.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Radio Button</title>
</head>
<body>
  <form action="https://www.www.techbridge.in">
    <label>Gender</label><br><br>
    <input type="Radio" required id="Male" name="gender">
    <label for="Male">Male</label>
    <input type="Radio" required id="Female" name="gender">
    <label for="Female">Female</label>
    <input type="Radio" required id="Others" name="gender">
    <label for="Others">Others</label><br><br>
    <input type="submit">
  </form>
</body>
</html>
```

Output:



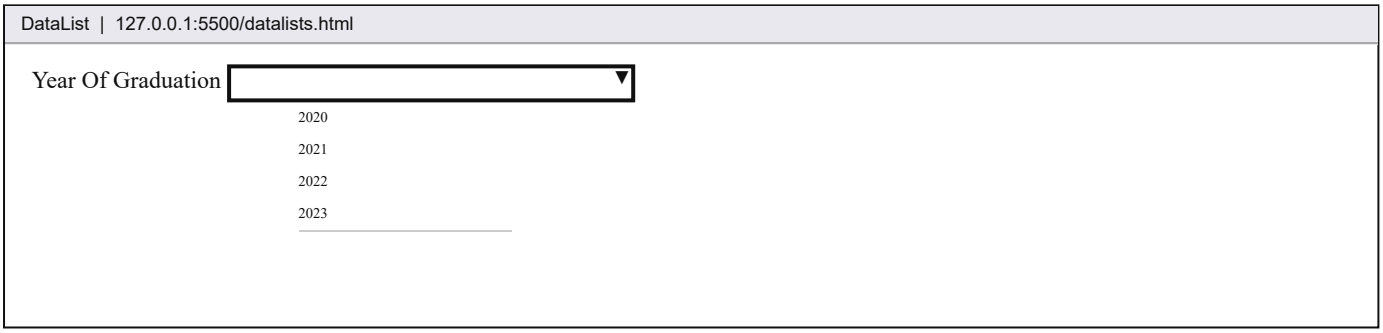
Creating the Data List in Form

- To create the data list in the forms we need to use the **<datalist> tag** and inside that we need to use the **<option> tag** to specify the options.
- To create this, we need to use the **list attribute** and the value of **list attribute** for **input** and value of **id attribute** in <datalist> tag should be same.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>DataList</title>
</head>
<body>
  <label>Year Of Graduation</label>
  <input list="YOG">
  <datalist id="YOG">
    <option>2020</option>
    <option>2021</option>
    <option>2022</option>
    <option>2023</option>
  </datalist>
</body>
</html>
```

Output:



UserName Validation

- ➤ To validate the username, in html we have to use pattern attribute and give the proper pattern to the username.
- ➤ Username should contain only the alphabets that is from lowercase a to z and uppercase A to Z.
- ➤ We should also tell the minimum and maximum characters it should accept.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>UserName_Validation</title>
</head>
<body>
  <form action="https://www.www.techbridge.in">
  <label for="uname">Username</label>
  <input type="text" required id="uname" pattern="[a-zA-Z]{3,8}">
  <input type="submit">
  </form>
</body>
</html>
```

Output:



Figure 22: Browser output when the username "1" does not match the pattern

Note: In the above example as we have given the [unclear] pattern, we have to match the data as per the given pattern only if we try to give any other format it will not allow.

Phone Number Validation

- To validate the phone number, in html we have to use pattern attribute and give the proper pattern to the phone number. [partly obscured in source: "phone [unclear]umber", "pat[unclear] [unclear], the phone number"]
- Phone Number should contain only the numbers, it should not contain any alphabets or special characters.
- Phone number should contain only 10 digits.
- Phone number first digit should start with either 6,7,8 or 9 and remaining 9 digits can be anything from 0 to 9.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Phone Number Validation</title>
</head>
<body>
  <form>
    <label for="phone">Phone Number</label>
    <input type="text" required id="phone" pattern="[6-9]{1}[0-9]{9}">
    <input type="submit">
  </form>
</body>
</html>
```

Output:

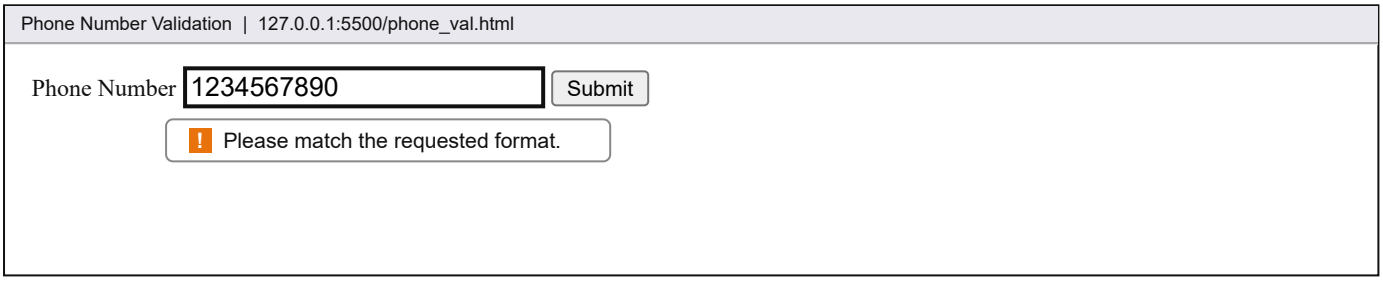


Figure 23: Browser output when "1234567890" is entered — it starts with 1, so it does not match the pattern

Note: In the above example as we have given the pattern, we have to match the data as per the given pattern only if we try to give any other format it will not allow so the phone number should contain only 10 digits and in that the first digit should be in range 6 to 9 and remaining 9 digits can be in range 0 to 9.

Email Validation

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Email_Validation</title>
</head>
<body>
  <form action="">
    <label for="">Email</label>
    <input type="email" id="email" required pattern="[a-z0-9_.]+@[a-
z]+.[a-z]{2,}">
    <input type="submit">
  </form>
</body>
</html>
```

Output:

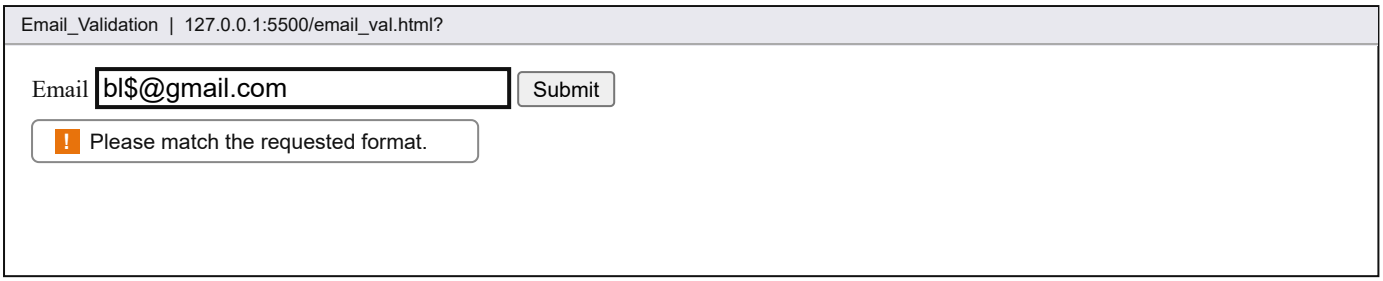


Figure 24: Browser output when "bl$@gmail.com" is entered — the "$" does not match the pattern

Example for student details form:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student_details</title>
</head>
<body>
<form action="https://www.www.techbridge.in">
  <fieldset>
    <legend>Person_details</legend>
      <label for="fname">First Name</label>
      <input type="text" required id="fname" pattern="[a-zA-
z]{3,8}"><br><br>
      <label for="lname">Last Name</label>
      <input type="text" required id="lname" pattern="[a-zA-
z]{3,8}"><br><br>
      <label for="email">Email</label>
      <input type="email" required id="email" pattern="[a-z0-9_.]+@[a-
z]+.[a-z]{2,}

"><br><br>
      <label for="phone">Phone Number</label>
      <input type="text" required id="phone" pattern="[6-9]{1}[0-
9]{9}"><br><br>
  </fieldset>
  <fieldset>
    <legend>Education_Details</legend>
    <label>Department</label><br><br>
    <input type="radio" id="cs" name="dept">
    <label for="cs">Computer Science</label>
    <input type="radio" id="ec" name="dept">
    <label for="ec">Electrical and Communication</label>
    <input type="radio" id="mech" name="dept">
    <label for="mech">Mechanical</label>
    <input type="radio" id="civil" name="dept">
    <label for="civil">Civil</label><br><br>
    <label>Year Of Graduation</label>
    <input list="yog"><br><br>
    <datalist id="yog">
      <option>2020</option>
      <option>2021</option>
      <option>2022</option>
      <option>2023</option>
      <option>2024</option>
    </datalist>
    <label>Skills</label><br><br>
    <input type="checkbox" id="Java">
    <label for="Java">Java</label><br>
```

```
    <input type="checkbox" id="SQL">
    <label for="SQL">SQL</label><br>
    <input type="checkbox" id="HTML">
    <label for="HTML">HTML</label><br>
    <input type="checkbox" id="CSS">
    <label for="CSS">CSS</label><br>
  </fieldset>
  <input type="submit">
</form>
</body>
</html>
```

Output:

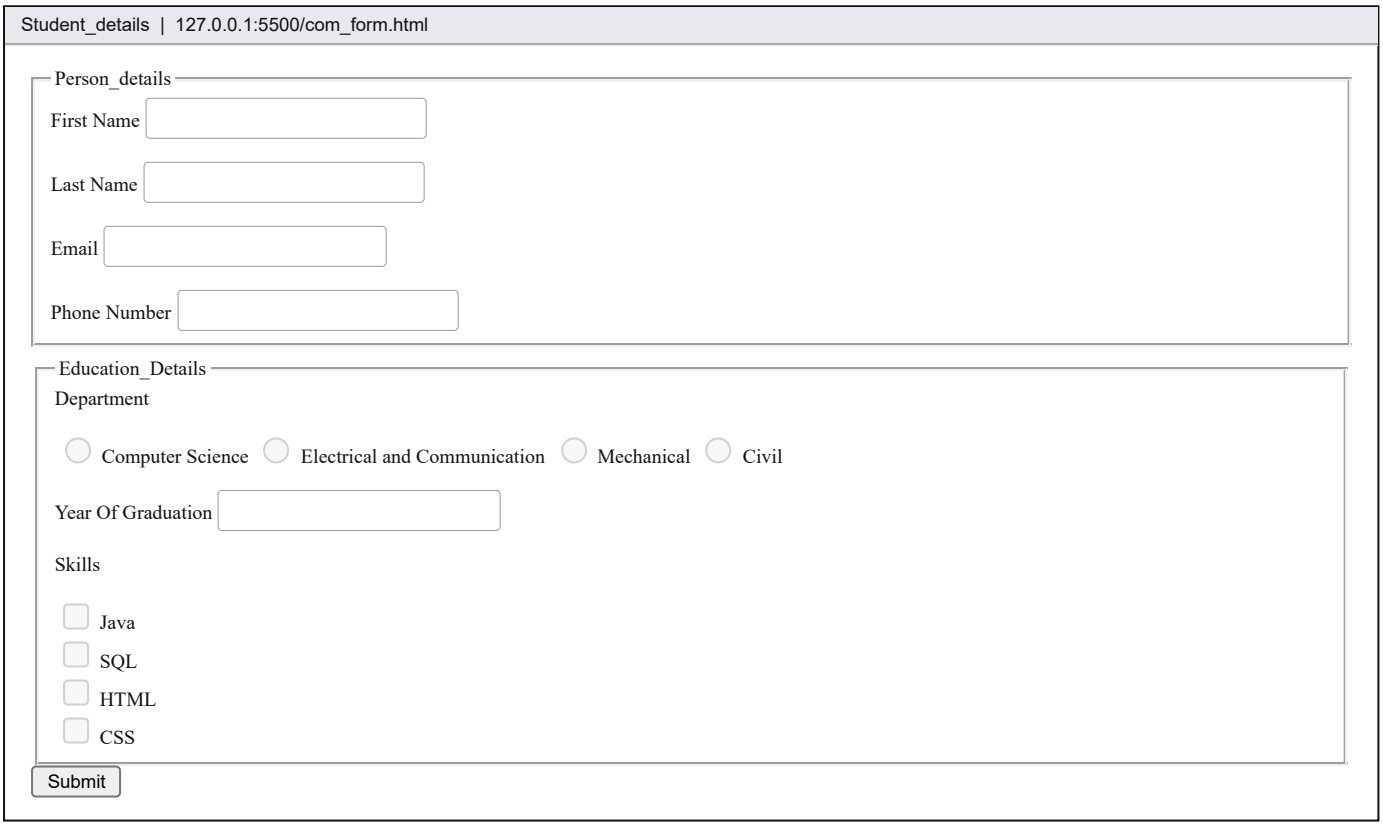


Figure 25: Browser output of the student details form (two groups, "Person_details" and "Education_Details")

Note: In the above example to differentiate and segregate the information of student we are using the **<fieldset> tag** and to give the name/label for that field we are using the **<legend> tag** which is a child tag of <fieldset> tag.

Summary

Basic HTML Tag

1. <html>: Used to tell to browser that we are starting with HTML.
2. <head>: Used to redirect to HEAD.
3. <title>: Used to give the title for the web page
4. <body>: Used to redirect to the body.
5. <p>: Used to give the paragraph.
6. <pre>: Used to give the Preformatted Text.
7.
: Used to break the line
8. <hr>: Used to give the horizontal line between the content
9. Heading: Used to the Headings of content.
    <h1>
    <h2>
    <h3>
    <h4>
    <h5>
    <h6>
10. <img>: Used to place the image in the webpage.
    Syntax:
        <img src="img_name.extension">
11. <video>: Used to place the video in the webpage.
    Syntax:
        <video src="video_name.extension" controls>
12. <audio>: Used to place the audio in the webpage.
    Syntax:
        <audio src="audio_name.extension" controls>
13. <a>: Used to place the links in the webpage, which redirect from one page to another.
    Syntax:
        <video href="path">content

Formatting Tags

1. <u>: Used to underline the content.
2. <b>: Used to bold the content.
3. <i>: Used to change the font italic in the webpage.
4. <mark>: Used to mark for highlight the content in the webpage.
5. <sub>: Used to give the subscripts in the webpage.
6. <sup>: Used to give the superscripts in the webpage.
7. <small>: Used to change the size of font to the small.
8. <big>: Used to change the size of font to big
9. <s>:Used to strike the content in the webpage.

Table

1. <table>: Used to create the table.
2. <tr>: Used to specify the table row.
3. <th>: Used to specify the table header.
4. <td>: Use to give the table data.

List

1. : Used to create the ordered list.
2. : Used to give the items for the list.
3. : Used to create the unordered list.
4. <dl>: Used to create the description list
5. <dt>: Used to give the data term for the description list
6. <dd>: used to give the data definition for data term in the description list.

Iframe: Used to embed the webpages in the other webpage

Syntax:

```
<iframe src="path" height="size" width="size"></iframe>
```

Form

1. <form>: Used to create the form.
2. <label>: Used to give the label for the input box.
3. <input>: Used to create the input box to accept the input from the user
   Syntax:
   <input type="text/email/number etc">
4. <submit>: Used to create a submit to submit the form once all the data is filled.
5. <fieldset>: Used to create the fieldset for the content to differentiate properly.
6. <legend>: Used to give the name for the fieldset.

INTRODUCTION TO CSS

➤ **CSS** stands for **Cascading Style Sheets** and is used for styling content on **web pages**. While HTML allows us to structure and place content, CSS is required for styling it.

➤ In CSS, styling is defined using **property-value** pairs.

Syntax of CSS

Consists of **selectors** and **declaration** blocks.

❋ **Selector**: HTML element to be styled.

❋ **Declaration Block**: One or more declarations separated by semicolons.
Declaration: Consists of a property and a value, separated by a colon.

Basic Structure

```
selector {
    property: value;
    property: value;
}
```

Example:

```
h1 {
    color: crimson;
    background-color: yellow;
}
```

BASIC CSS PROPERTIES

1. **color:** This property is used to apply/give the color for the character, text, word or sentence.
2. **text-align:** This property is used to place or align the text in center, right or left.
3. **font-family:** This property is used to change font style of text like italic, cursive, Georgia etc.
4. **background-color:** This property is used to give the background color for the text.
5. **font-size:** This property is used to change the size of the text.
6. **border:** This property is used to give the border for the text.
7. **height:** This property is used to set the height for the image content etc.
8. **width:** This property is used to set the width for the image content etc.

Note: We have many other CSS properties we will use them according to the requirement.

TYPES OF CSS

Inline CSS

➤ If we apply the styling to the content inside the element only by using the style attribute, we call it **"inline CSS".**

Example

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inline CSS</title>
</head>
<body>
  <h1 style="color: brown; font-family: cursive; text-align: center;">TechBridge</h1>
</body>
</html>
```

Output:

TechBridge

Figure 26: Browser output of the inline-styled heading (centered, brown, cursive; the screenshot is very small in the source)

Note: In the above example we are using the color property and value for that as brown, whenever we are using the property a proper value also, we need to assign for it. And based on our required we are style our contents by using many more CSS properties.

The <meta> tag will help us to set the content according to our browser width properly.

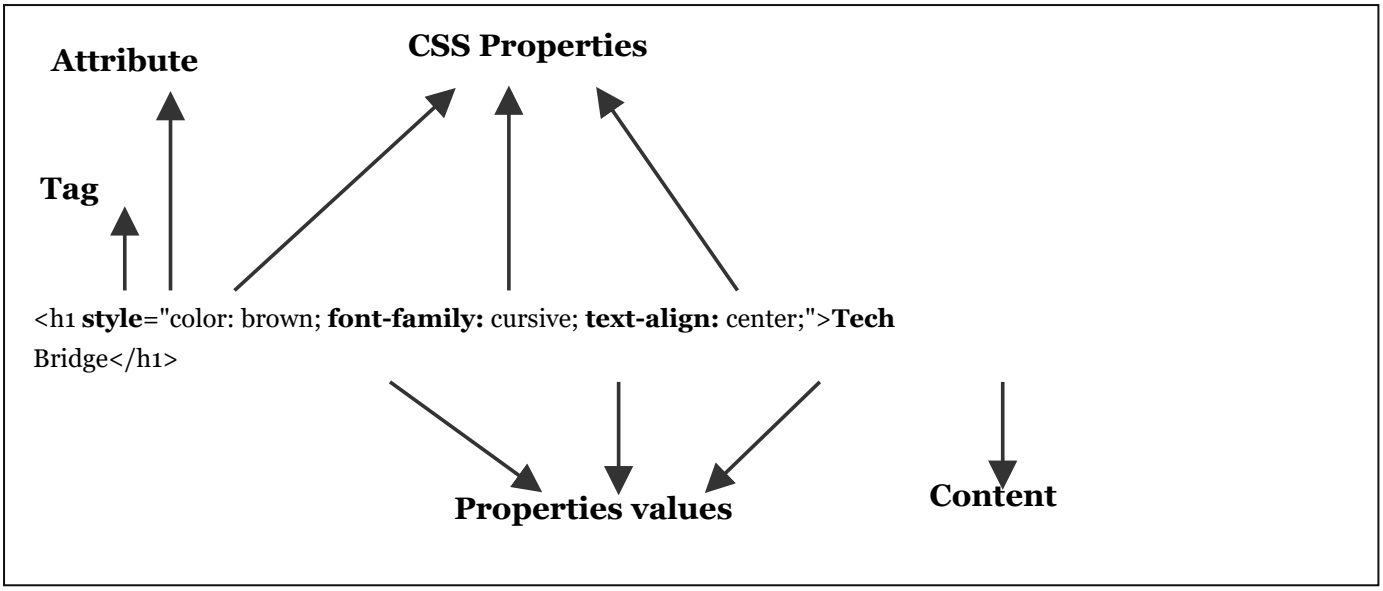


Figure 27: Anatomy of an inline style — Tag (<h1>), Attribute (style), CSS Properties (color, font-family, text-align), Properties values (brown, cursive, center) and Content (TechBridge)

Internal CSS

➤ If we apply the styling to the content by using the <style> tag, we call it as **"internal CSS".**

➤ <style> tag we can specify in **<head>** section or in the **<body>** section as well. But as a standard, we use it in the <head> section.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internal CSS</title>
  <style>
   p{
     color: blue;
     background-color: aquamarine;
   }
  </style>
</head>
<body>
  <p>This is a first paragraph</p>
</body></html>
```

Output:

Internal CSS

This is a first paragraph

Note: In the above example, we have taken <p> tag and given the content, for that content we are applying the styling in the internal approach, in the internal styling we need to select the element name and apply the

styling.

Syntax:

```
element_name{
   CSS_property: value,
   CSS_property: value
}
```

Wherever that element name matches for all those elements the styling will be applied.

External CSS

- ➤ External CSS means we should create a separate file for the CSS with the extension as **.css** and give all the styling properties in it.
- ➤ The content should be present in the HTML file only so here we will have two file one is for content and one is for CSS.
- ➤ That CSS file we have to link it to the content file then only the styling will be applied to the contents.
- ➤ To link the CSS file we need to use the **<link>** tag in html file and link the CSS file by specifying the CSS file in the <link> tag.
- ➤ We need to execute the HTML file only, not the CSS file.

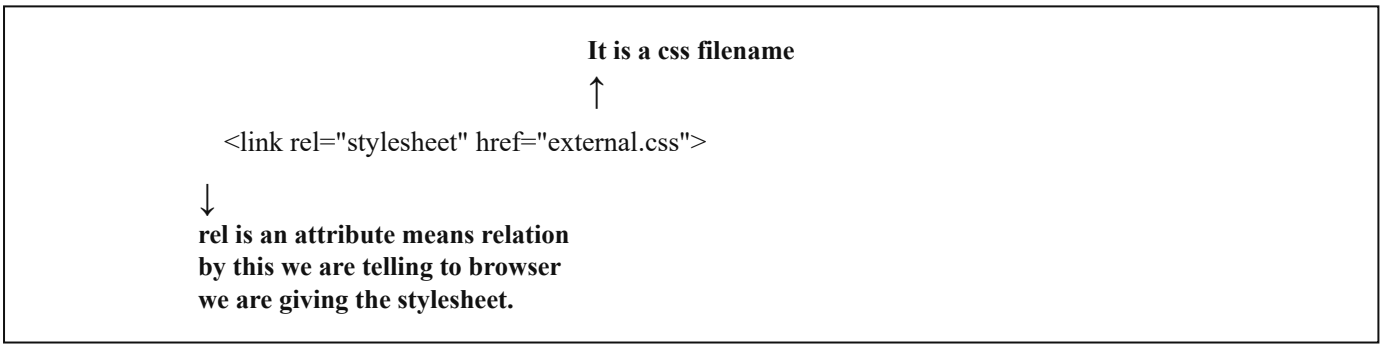


Figure 28: The <link> tag — href gives the css filename ("It is a css filename"); rel is an attribute meaning relation, telling the browser we are giving the stylesheet

Example:

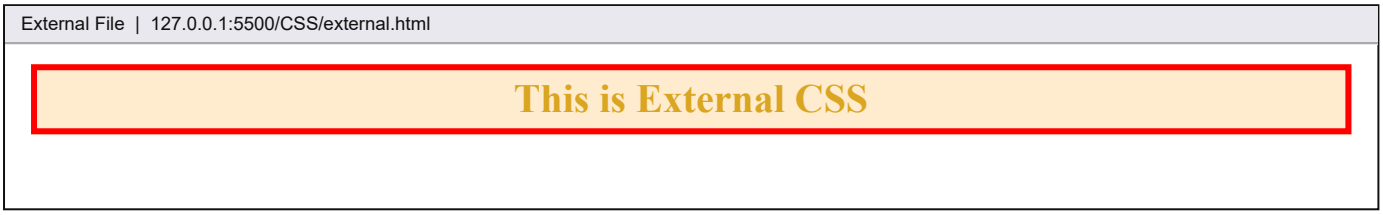
HTML File

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>External File</title>
  <link rel="stylesheet" href="external.css">
</head>
<body>
  <h1>This is External CSS</h1>
</body>
</html>
```

CSS File

```
h1{
  color: goldenrod;
  text-align: center;
  border: 3px solid red;
  font-family: 'Times New Roman', Times, serif;
  background-color: blanchedalmond;
}
```

Output:



Note:

- In the CSS file while giving the styling we no need to specify style attribute or <style> tag directly we can select the element and apply the styling.

Two Ways To Linking External CSS:

1. **Link Tag:**

   Used within the `<head>` section to link an external CSS file.

Example:

```
<link rel="stylesheet" href="style.css">
```

Attributes:

`**rel**`="stylesheet": Specifies the relationship between the current document and the linked file.

`**href**` : Specifies the path to the CSS file.

2. **Import Statement:**

   Another way to include an external CSS file.

Example:

```
@import url("style.css");
```

Typically used within a CSS file to import other CSS files.

Why CSS is Called as Cascading:

CSS is known as Cascading Style Sheets due to the **cascading order of priority**:

1. Inline CSS
2. Internal CSS
3. External CSS

When multiple styles are applied to the same element, the priority follows this order:

➤ Inline CSS has the highest priority.
➤ Internal CSS takes priority if no inline styles are present.
➤ External CSS is applied if neither inline nor internal styles are present.

Example Demonstrating CSS Priority:

1. External CSS:

CSS File

```
h1 {
    color: black;
    background-color: white;
   }
```

2. Internal CSS:

Within HTML file:

HTML File

```
<!DOCTYPE html>
<html>
  <head>
   <style>
    h1 {
      color: blue;
      background-color: lightgrey;
    }
    </style>
   </head>
<body>
  <h1>This is External CSS</h1>
</body>
</html>
```

3. Inline CSS:
 - Directly in HTML element:

```
<h1 style="color: red; background-color: yellow;">Hello World</h1>
```

Result:

Inline CSS (`color: red; background-color: yellow;`) will be applied due to its highest priority.

CSS SELECTORS

CSS selectors are used to select the content you want to style. CSS Selectors are used to select HTML elements based on their element name, id, attributes, etc. It can select one or more elements simultaneously. There are many different types of **CSS selector** they are listed below.

General syntax:
Selector
{
property : value;
property : value;
property : value;
.

.
}

Let's explore all different type of selectors one by one.

BASIC SELECTORS

UNIVERSAL SELECTOR

CSS universal selectors select any type of elements in an HTML page. It matches a single element. An asterisk i.e. "*" is used to denote a CSS universal selector. This will come into the picture when all the elements inside the <body> tag should be styled. Let's understand with an example index.html

index.html (html-1)

```
<!DOCTYPE html>
<html>
    <head>
        <title>Home Page</title>
    <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <h1>Full stack Web Development</h1>
        <p>A full-stack web developer is a person who can develop
        both client and server software.</p>
        <h2>Technologies related to full stack web development</h2>
        <h3>Front End Technologies</h3>
        <p>Front end development is the process of building the user
        interface of an application. It includes everything that the user sees and interacts
        with-text, images, video, and audio-basically the visual appearance.</p>
        <img src="f.png" height="300" width="700">
        <h3>Back End Technologies</h3>
        <p>Backend technology refers to the programming languages used in
        web development to create and manage a website or application.</p>
        <img src="b.jpeg" >
        <h3>Database Technologies</h3>
        <p>Database technologies take information and store, organize, and process
        it in a way that enables users to easily and intuitively go back
        and find details they are searching for.</p>
        <img src="d.jpg" height="300" width="700">
    </body>
</html>
```

style.css

```
*{
background-color: powderblue;
font-family: cursive;
}
```

OUTPUT



Figure 29: Output of index.html (html-1) with the universal selector — all elements show the powder-blue background and cursive font.

As we can see clearly in the output all the elements present inside the body background color and font as changed to powder-blue and cursive so universal selector means it will select all the elements present inside the body of the HTML page.

ELEMENT SELECTOR

The element selector selects all elements with the specified element name. Let's understand with an example.

style.css refer **(html-1)**

```
h1{
   background-color: red;
color: yellow;
font-family: cursive;
}
```

OUTPUT



Figure 30: Output of index.html (html-1) with the element selector h1 — the heading "Full stack Web Development" is styled (red background, yellow text, cursive), the rest of the page is unchanged.

As we can see in the output **element selector** selects all the elements that matches the **selector** in this program it is h1 and applies the styling. If you mention the selector as p tag it will select all the element that matches p tag and applies styling. If we select **body** element as a selector then styling is applied to body of the html page, if we change the css file of the above example.

style.css refer **(html-1)**

```
body{
  background-color: powderblue;
  color: black;
  font-family: cursive;
}
```

OUTPUT

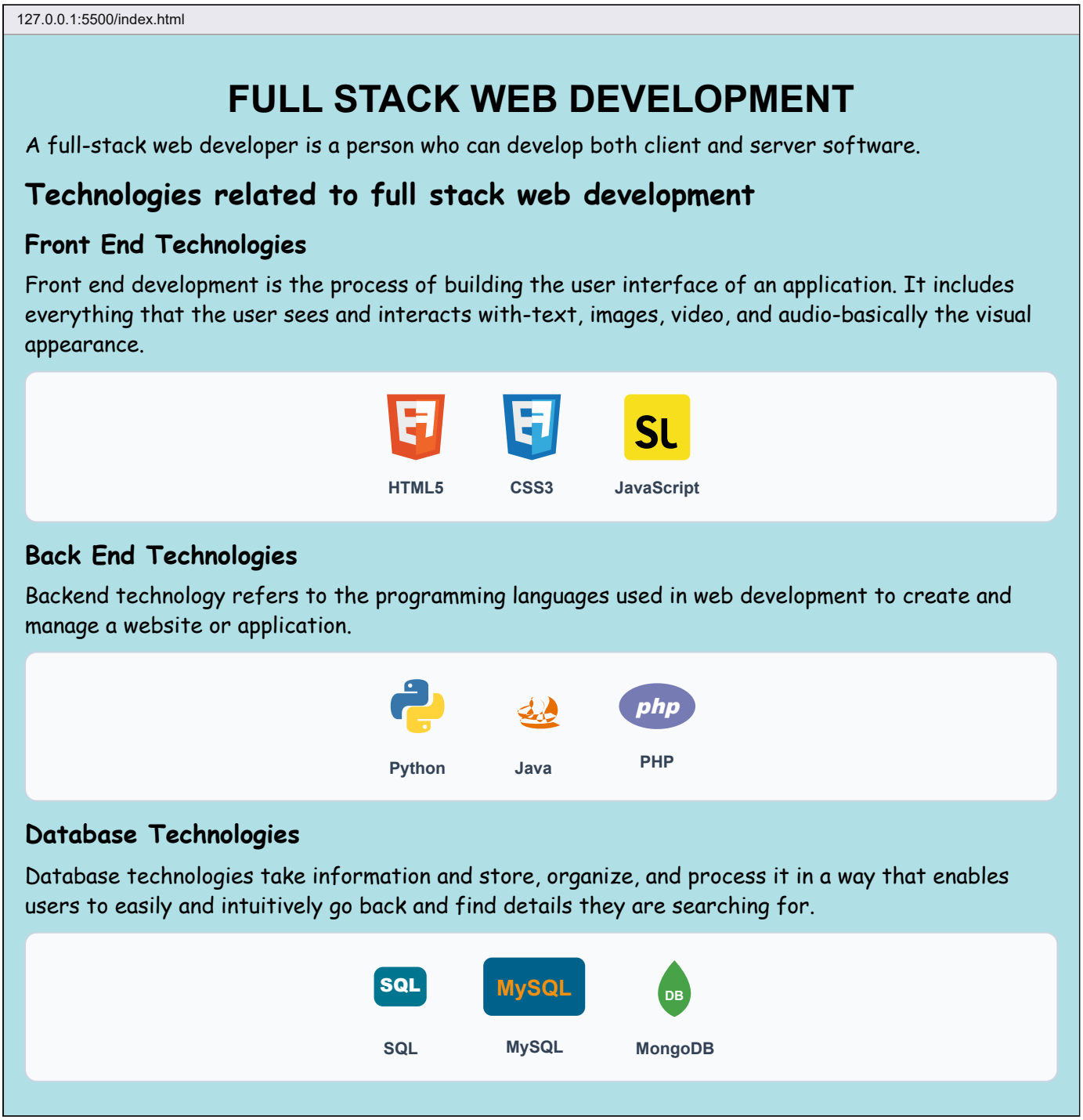


Figure 31: Output of index.html (html-1) with the body selector — all content shows the powder-blue background and cursive font.

As we can see in the output using body element, we can style the body of the elements, if you recollect, output is similar to that of universal selector but there is a difference between universal selector and element selector. Now let's understand the difference between universal selector and element selector with an example.

Using Element Selector

index.html (html-2)

```
<!DOCTYPE html>
<html>
    <head>
        <title>Home Page</title>
        <link rel="stylesheet" href="style.css">
    </head>

    <body>
      <h1>Hi</h1>
      <h2>Hello</h2>
      <h3>Hola</h3>
      <label for="">Name</label>
      <input type="text">
    </body>
</html>
```

style.css

```
body{
  background-color: cyan;
}
```

OUTPUT

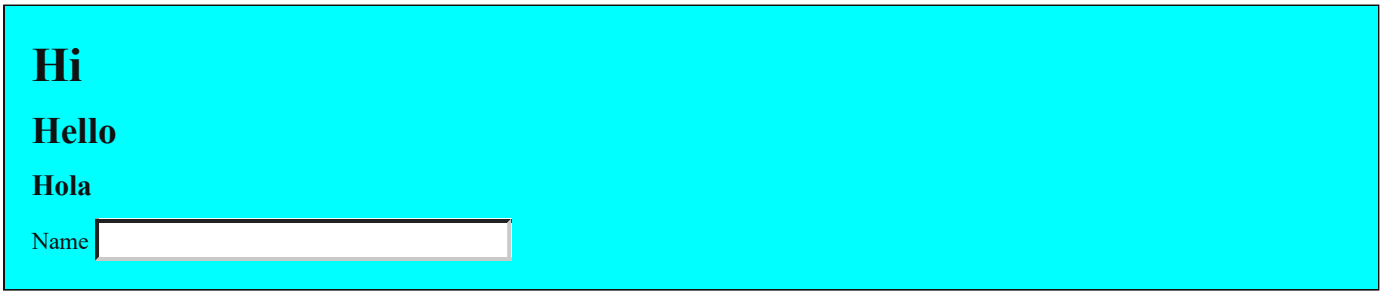


Figure 32: Output with the element selector (body) — the page background is cyan, but the input box keeps its own white background

Using Universal Selector

style.css refer (html-2)

```
*{
   background-color: cyan;
}
```

OUTPUT

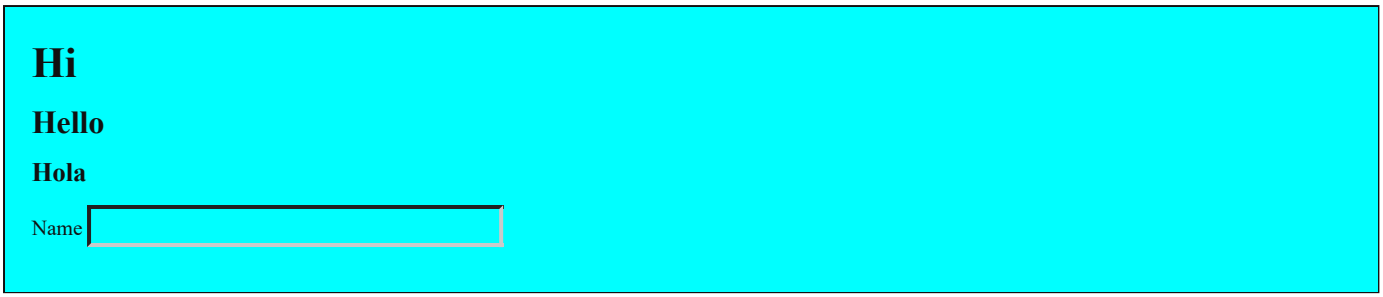


Figure 33: Output with the universal selector (*) — every element, including the input box, gets the cyan background

As we can notice the output of element selector and universal selector, we can change the background color of all the elements in the body using universal selector but using element selector it is not possible because few elements have their own css property for example input tag as its own css property that background color is white. This is the main difference between element selector and universal selector.

GROUP SELECTOR

In css group selector is used to group together multiple selectors into one. We have to group together by separating each element with comma. We can give the same properties to a number of selectors without having to repeat them, now let's understand this with an example.

style.css refer **(html-2)**

```
h1,h2,h3{
    background-color: cyan;
    font-family: cursive;
}
```

OUTPUT

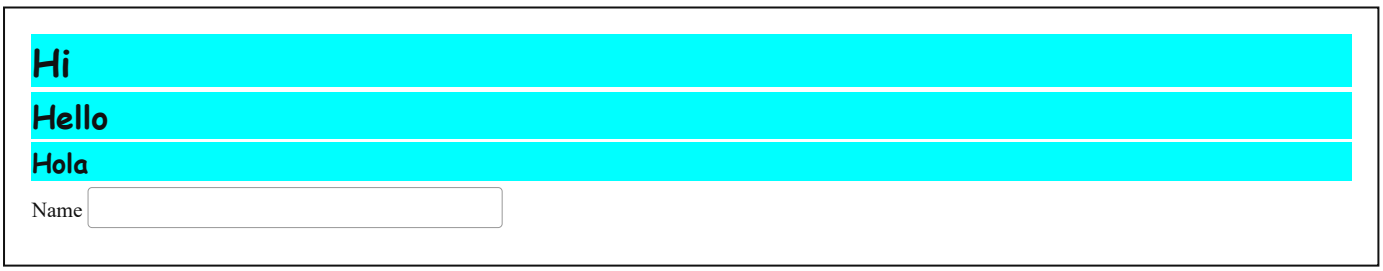


Figure 34: Output with the group selector h1,h2,h3 — only the three headings have the cyan background and cursive font; the label and input are unchanged

As we can see from the above example using group selector, we can style multiple elements simultaneously.

CLASS SELECTOR

Class selector is used to select all elements which belong to a particular class attribute. To select the elements with a particular class, use (.) character with specifying class name. Class name is mostly used to set the CSS property to given class. To give clear picture about class selector let's see the example below.

index.html (html-3)

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Home Page</title>
        <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <h1 class="heading">Full stack Web Development</h1>
        <p class="info" id="intro">A full-stack web developer is a person who can develop
        both client and server software.</p>
        <h2 class="heading">Technologies related to full stack web development</h2>
        <h3 class="heading">Front End Technologies</h3>
        <p class="info">Front end development is the process of building the user
        interface of an application. It includes everything that the user sees and interacts
        with-text, images, video, and audio-basically the visual appearance.</p>
        <img src="front.jpg" height="300" width="700">
        <h3 class="heading">Back End Technologies</h3>
        <p class="info">Backend technology refers to the programming languages used in
        web development to create and manage a website or application.</p>
        <img src="b.jpeg" >
        <h3 class="heading">Database Technologies</h3>
        <p class="info">Database technologies take information and store, organize, and process
        it in a way that enables users to easily and intuitively go back
        and find details they are searching for.</p>
        <img src="d.jpg" height="300" width="700">
    </body>
</html>
```

style.css

```css
.heading{
  background-color: red;
  color: yellow;
  font-family: monospace;
}

.info{
  background-color: black;
  color: white;
  font-family: cursive;
}
```

OUTPUT

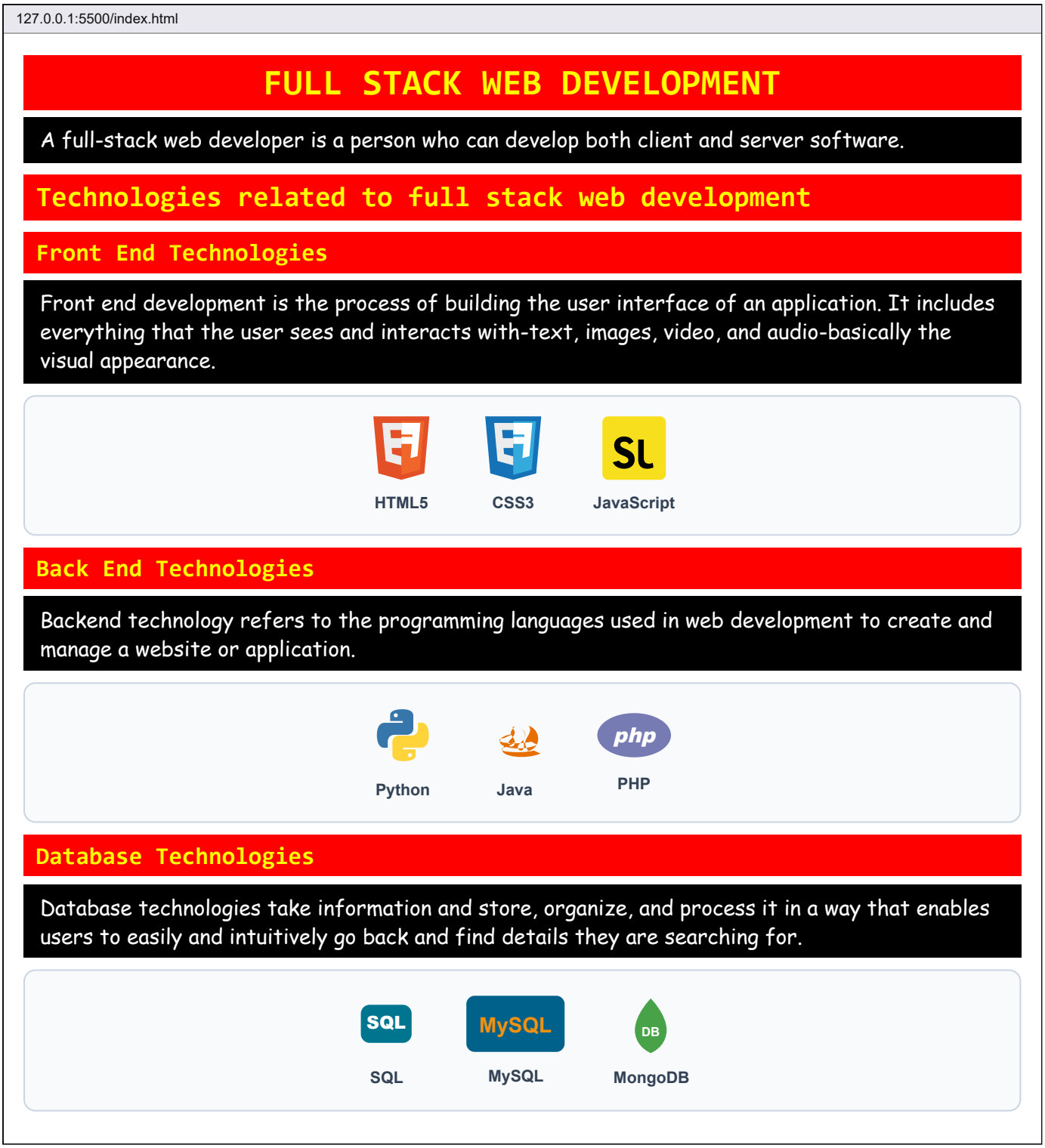


Figure 35: Output of index.html (html-3) with the class selectors .heading (red background, yellow monospace text) and .info (black background, white cursive text).

As we can see from the above example using class selector, we can group different elements into a certain class and apply styling.

ID SELECTOR

The id selector is used to set the style of given id. The id attribute is the unique identifier in HTML document. The id of an element is unique within a page, so the id selector is used to select one unique element. The id selector is used with # character. Let's now understand with an example.

style.css refer **(html-3)**

```
.heading{
  background-color: red;
  color: yellow;
  font-family: monospace;
}

.info{
  background-color: black;
  color: white;
  font-family: cursive;
}

#intro{
  background-color: cyan;
  color: black;
  font-family: fantasy;
}
```

OUTPUT:

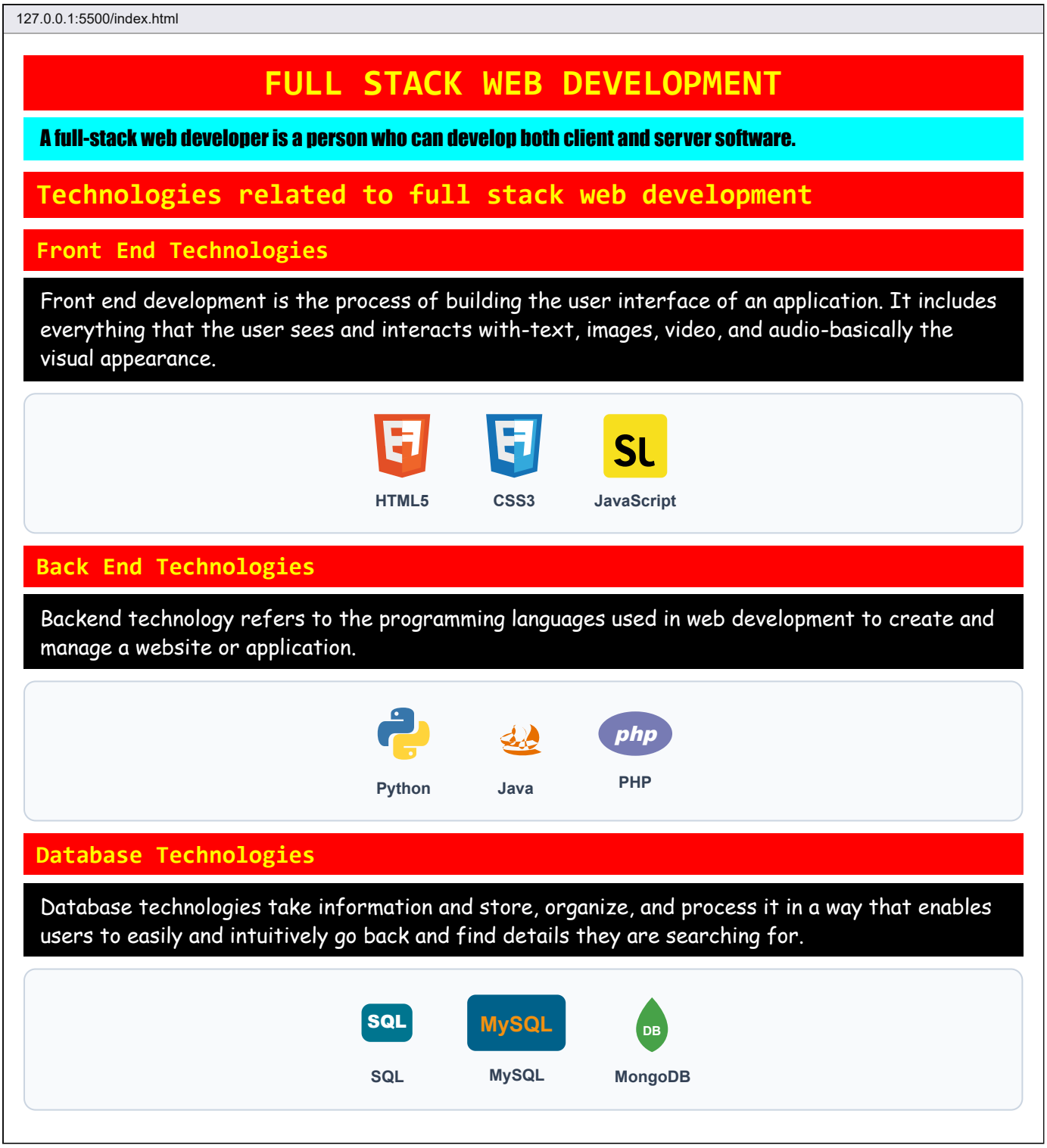


Figure 36: Output of index.html (html-3) with .heading, .info and #intro — the first paragraph (id="intro") now takes the id rule (cyan background, black text, fantasy font).

COMBINING SELECTORS

Combining element selector with class selector

index.html (html-4)

```
 1  <!DOCTYPE html>
 2  <html>
 3      <head>
 4          <title>Home Page</title>
 5          <link rel="stylesheet" href="style.css">
 6      </head>
 7      <body>
 8          <h1 class="web">Web Development</h1>
 9          <p class="bg1 font1">Web development is the work involved in developing a website for the Internet
(World Wide Web) or an
            intranet
10          (a private network).[1] Web development can range from developing a simple single static page of
plain text to
11          complex web applications, electronic businesses, and social network services. A more comprehensive
list of tasks
12          to which web development commonly refers, may include Web engineering, Web design, Web content
development,
13          client liaison, client-side/server-side scripting, Web server and network security configuration, and
14          e-commerce development.</p>
15          <h1 class="ai" id="header2">Artificial intelligence</h1>
16          <p class="bg2 font2">Artificial intelligence (AI), in its broadest sense, is intelligence exhibited
by machines, particularly
17          computer systems. It is a field of research in computer science that develops and studies methods and
software
18          which enable machines to perceive their environment and uses learning and intelligence to take
actions that
19          maximize their chances of achieving defined goals.[1] Such machines may be called AIs.</p>
20          <h1>Cyber Security</h1>
21          <p class="bg3 font3">Computer security, cybersecurity, digital security or information technology
security (IT security) is the
22          protection of computer systems and networks from attacks by malicious actors that may result in
unauthorized
23          information disclosure, theft of, or damage to hardware, software, or data, as well as from the
disruption or
24          misdirection of the services they provide.</p>
25      </body>
26  </html>
```

style.css refer **(html-4)**

```
.bg1{
  background-color: red;
}

.bg2{
  background-color: green;
}
.bg3{
  background-color: teal;
}
.font1{
  font-family: cursive;
  color: white;
}
.font3{
  font-family: fantasy;
  color: white;
}

h1.web{
  background-color: red;
  color: yellow;
}
```

OUTPUT

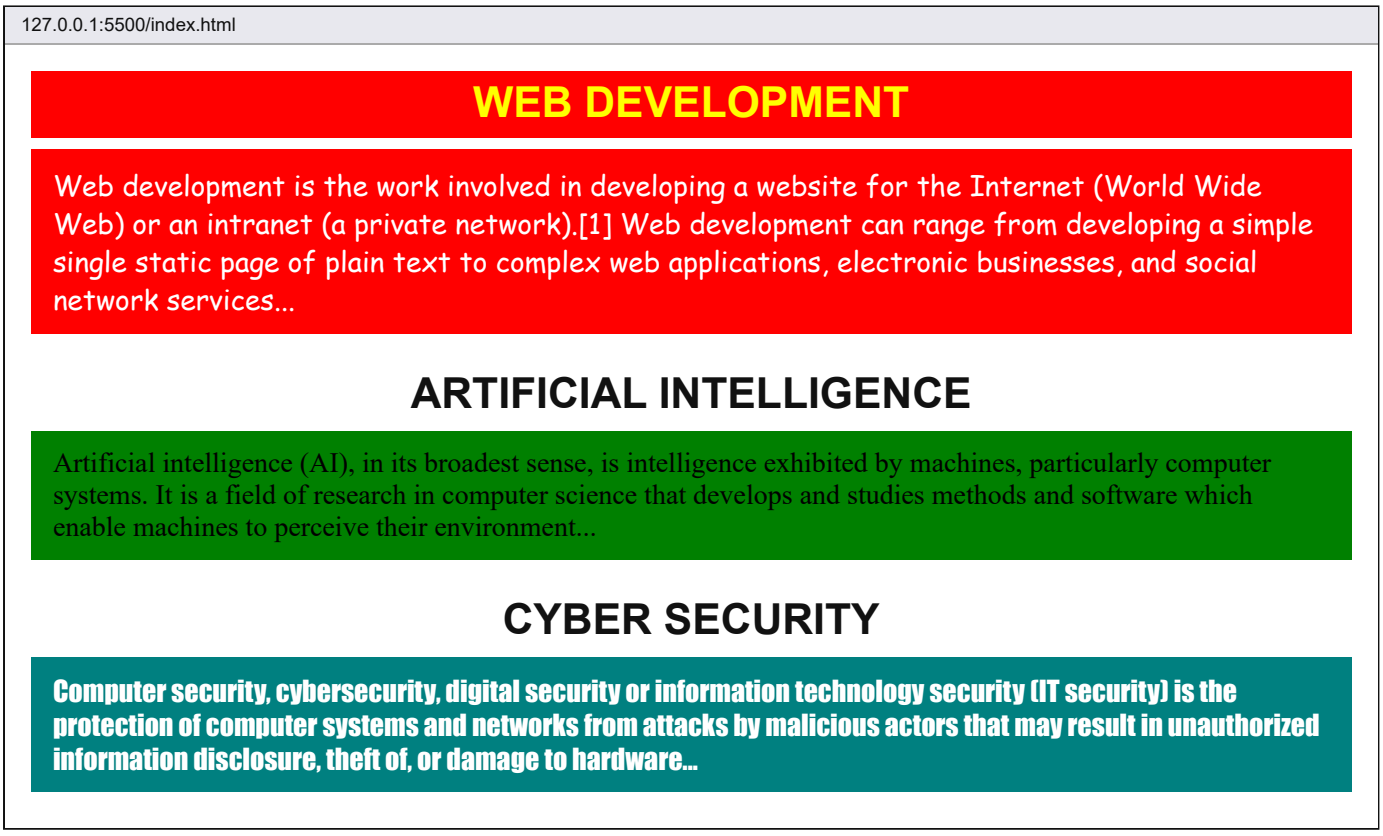


Figure 37: Output of index.html (html-4) — "Web Development" heading styled via h1.web (red background, yellow text); paragraphs styled via .bg1/.font1, .bg2 and .bg3/.font3.

As we can see from the above output styling of web page is done based on element and class i.e using element selector and class selector. In the above example h1 element is selected which belongs to the class web and styling of that particular element is done.

Combining class selector with another class selector

style.css refer **(html-4)**

```
.bg1{
  background-color: red;
}

.bg2{
  background-color: green;
}
.bg3{
  background-color: teal;
}
.font1{
  font-family: cursive;
  color: white;
}
.font3{
  font-family: fantasy;
  color: white;
}
h1.web{
  background-color: red;
  color: yellow;
}
.bg1 .font3{
  font-size: 25px;
}
```

OUTPUT

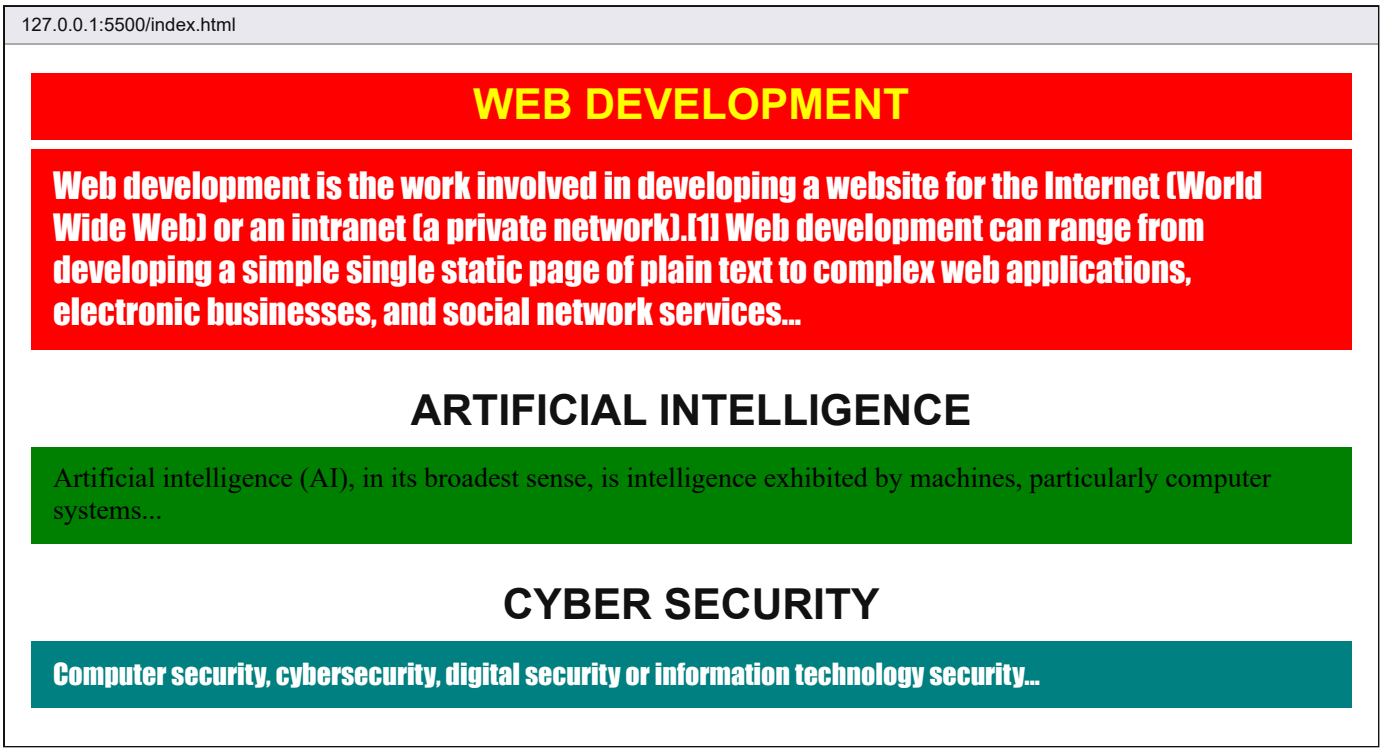


Figure 38: Output of index.html (html-4) with the combined selector .bg1 .font3 added to the style sheet.

In the above example, we have combined bg1 class with font1 class so html element that belongs to bg1 class and font1 class will be styled. If you can notice, first p tag is the only tag that belongs to both the classes and hence p tag gets styled.

Combining id and class selector

style.css refer **(html-4)**

```
.bg1{
  background-color: red;
}
.bg2{
  background-color: black;
}
.bg3{
  background-color: teal;
}
.font1{
  font-family: cursive;
  color: blueviolet;
}
.font3{
  font-family: fantasy;
  color: white;
}
h1.web{
  background-color: red;
  color: yellow;
}
.bg1 .font3{
  font-size: 25px;
}
#header2 .ai{
  background-color: grey;
}
```

OUTPUT

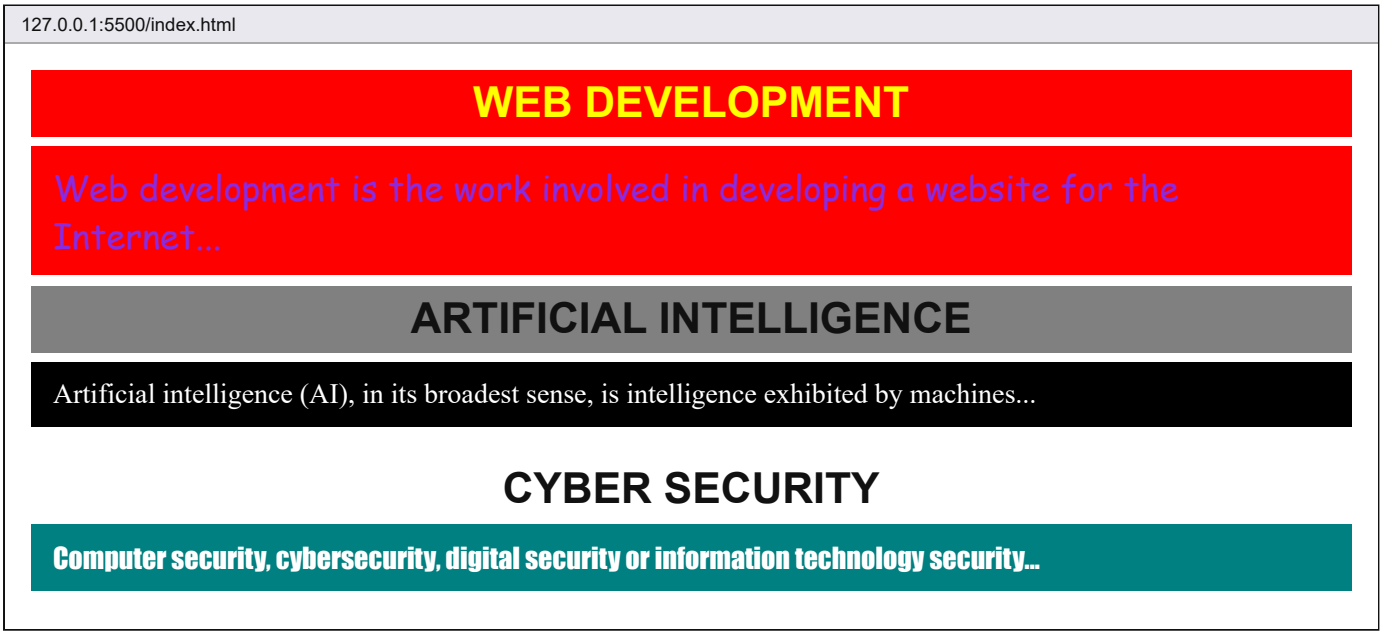


Figure 39: Output of index.html (html-4) with the combined id and class selector.

In the above example, clearly we can see styling is applied to that html element which belongs to header2 id selector and ai class selector

and tag

 tag is a block type element and tag is inline type element. The tag allows you to group a set of elements together in one block-level box. The element acts like an inline equivalent of the element.

index.html (html-5)

```
 1  <!DOCTYPE html>
 2  <html>
 3      <head>
 4          <title>Home Page</title>
 5          <link rel="stylesheet" href="style.css">
 6      </head>
 7      <body>
 8          <div class="web">
 9              <h1>Web Development</h1>
10              <p > <span class="firstline">Web development is the work involved in developing a website for the
Internet (World Wide Web) or an intranet
11                  (a private network).</span> [1] Web development can range from developing a simple single
static page of plain text to
12              complex web applications, electronic businesses, and social network services. A more
comprehensive list of tasks
13              to which web development commonly refers, may include Web engineering, Web design, Web content
development,
14              client liaison, client-side/server-side scripting, Web server and network security configuration,
and
15              e-commerce development.</p>
16          </div>
17          <div  class="ai">
18              <h2>Technologies to be learnt in Web Development</h2>
19              <ul>
20                  <li>Front-End: HTML,CSS,JS</li>
21                  <li>Back-End: Java,Spring,Hibernate</li>
22                  <li>Database: SQL,NOSQL</li>
23              </ul>
24              <h1>Artificial intelligence</h1>
25              <p> <span class="firstline">Artificial intelligence (AI), in its broadest sense, is intelligence
exhibited by machines, particularly
26                  computer systems.</span> It is a field of research in computer science that develops and
studies methods and software
27              which enable machines to perceive their environment and uses learning and intelligence to take
actions that
28              maximize their chances of achieving defined goals.[1] Such machines may be called AIs.</p>
29              <h2>Technologies to be learnt in AI</h2>
30              <ul>
31                  <li>Python</li>
32                  <li>statistics and probability  </li>
33                  <li>exploratory data analysis</li>
34                  <li>natural language processing</li>
35                  <li>Deep learning</li>
36                  <li>Machine learning</li>
37              </ul>
```

index.html (html-5) — continued

```
38          </div>
39          <div  class="cs">
40              <h1>Cyber Security</h1>
41              <p> <span class="firstline">Computer security, cybersecurity, digital security or information
technology security (IT security) is the
42                  protection of computer systems</span> and networks from attacks by malicious actors that may
result in unauthorized
43              information disclosure, theft of, or damage to hardware, software, or data, as well as from the
disruption or
44              misdirection of the services they provide.</p>
45              <h2>Technologies to be learnt in AI</h2>
46              <ul>
47                  <li>Networking and System Administration</li>
48                  <li>Knowledge of Operating Systems and Virtual Machines </li>
49                  <li>Network Security Control</li>
50                  <li>Cloud Security</li>
51                  <li>Blockchain Security</li>
52                  <li>The Internet of Things (IoT)</li>
53              </ul>
54          </div>
55      </body>
56  </html>
```

OUTPUT

127.0.0.1:5500/index.html

WEB DEVELOPMENT

Web development is the work involved in developing a website for the Internet (World Wide Web) or an intranet (a private network)...

Technologies to be learnt in Web Development

- Front-End: HTML,CSS,JS
- Back-End: Java,Spring,Hibernate
- Database: SQL,NOSQL

ARTIFICIAL INTELLIGENCE

Artificial intelligence (AI), in its broadest sense, is intelligence exhibited by machines, particularly computer systems...

Technologies to be learnt in AI

- Python
- statistics and probability
- exploratory data analysis
- natural language processing
- Deep learning
- Machine learning

CYBER SECURITY

Computer security, cybersecurity, digital security or information technology security (IT security)...

Technologies to be learnt in Cyber Security

- Network Security
- Ethical Hacking
- Cryptography

Figure 40: Output of index.html (html-5) before the style sheet is applied — three sections (Web Development, Artificial intelligence, Cyber Security), each with a paragraph and a list titled "Technologies to be learnt in ...".

As we can see in the output, we have grouped information related to web development has one element, AI has one element and cyber Security has another element using tag. Now, using this tag you can style each division differently as shown below.

style.css refer **(html-5)**

```
.web{
  background-color: red;
  font-family: cursive;
  color: yellow;
}

.ai{
  background-color: black;
  font-family: monospace;
  color: grey;
}

.cs{
  background-color: teal;
  font-family: fantasy;
  color: navy;
}
```

OUTPUT

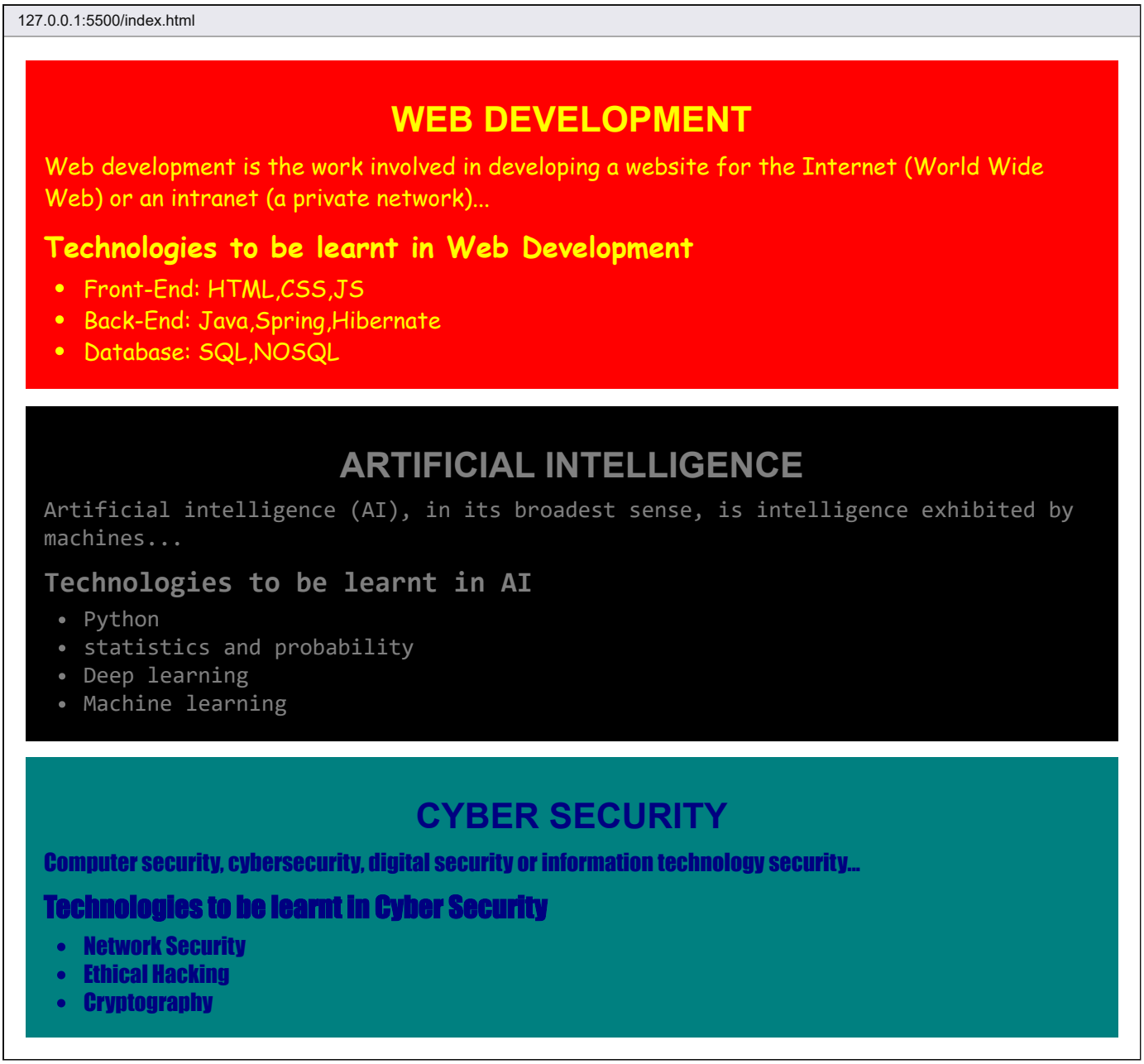


Figure 41: Output of index.html (html-5) with the division styles .web (red background, yellow cursive text), .ai (black background, grey monospace text) and .cs (teal background, navy fantasy text).

Now, if we want to style every first line of the paragraph without the line break in the above example then first you have to group those elements then apply styling to those elements. We can do this using the tag as shown below example.

style.css refer **(html-5)**

```
.web{
  background-color: red;
  font-family: cursive;
  color: yellow;
}

.ai{
  background-color: black;
  font-family: monospace;
  color: grey;
}

.cs{
  background-color: teal;
  font-family: fantasy;
  color: navy;
}

.firstline{
  background-color: burlywood;
}
```

OUTPUT

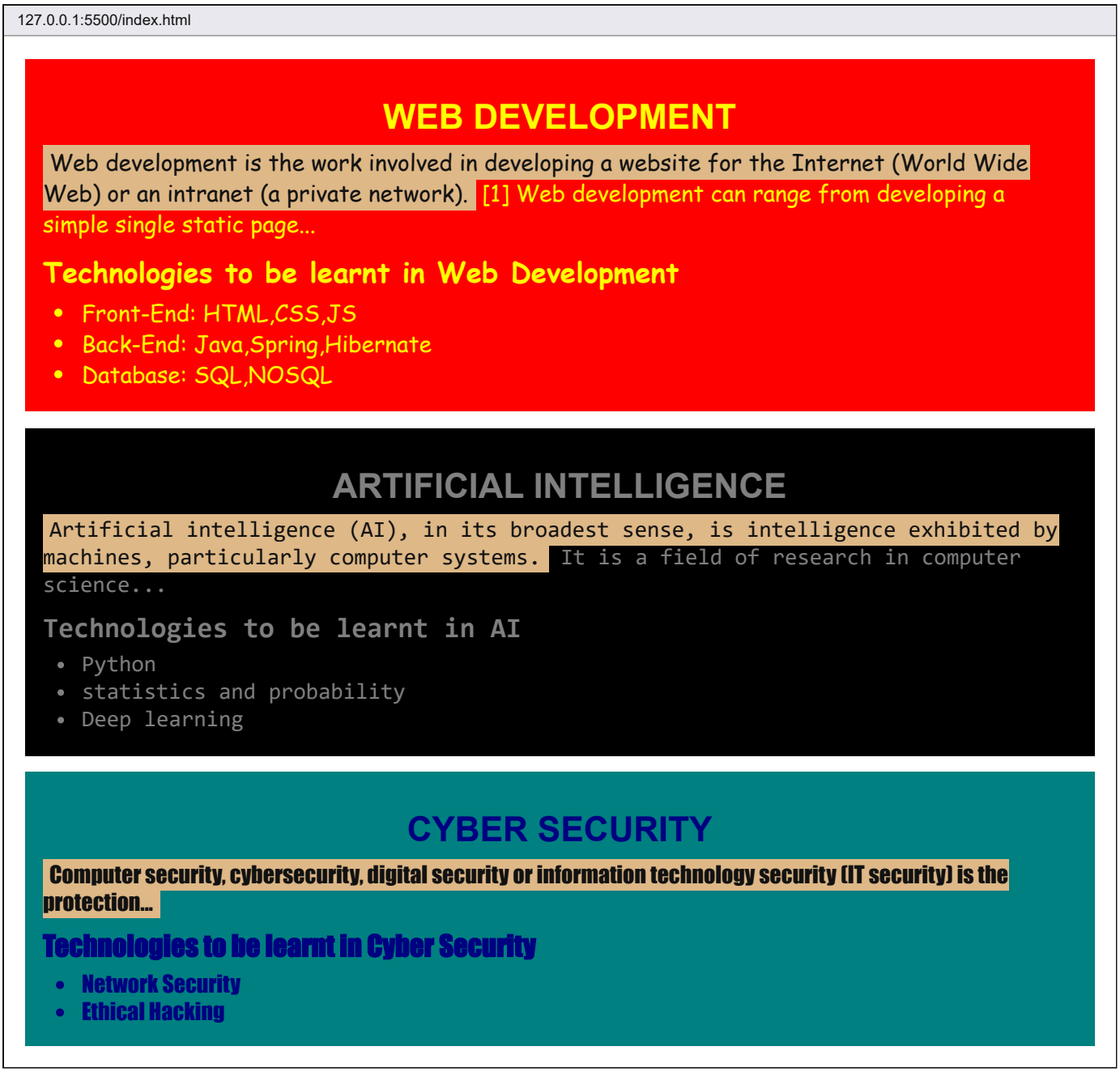


Figure 42: Output of index.html (html-5) with .firstline applied — the first line of each paragraph is highlighted (burlywood background).

Thus, using the tag we have styled the first line of every paragraph without the line break.

ADVANCED SELECTORS

COMBINATORS

A CSS selector can contain more than one simple selector. Between the simple selectors, we can include a combinator. There are four different combinators in CSS:

- descendant selector (space)
- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)

CHILD SELECTORS

The child selectors are again categorized into two types:

- Descendant
- Direct Child

DESCENDANT SELECTOR:

Any element which falls inside another HTML tag it is referred as its descendant. The combinator we use in a descendant selector is a whitespace(' ') character. Let's now understand descendant selector with the example.

style.css refer (html-5)

```
div h2{
background-color: yellow;
}
div li{
background-color: red;
}
```

Tree Representation

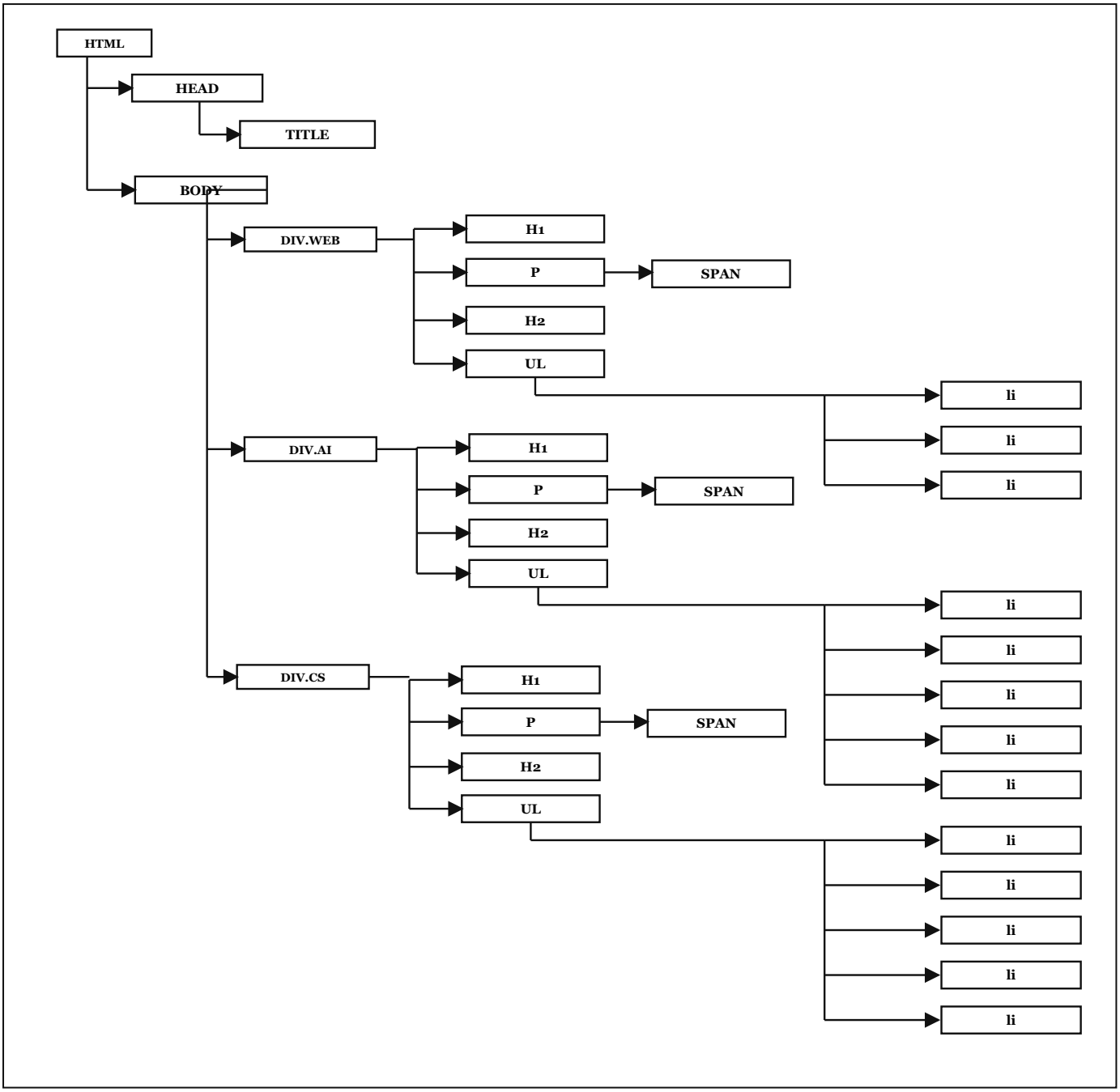


Figure 43: Tree representation of index.html (html-5) — HTML → HEAD (→ TITLE) and BODY; BODY → DIV.WEB, DIV.AI, DIV.CS; each DIV → H1, P (→ SPAN), H2, UL; each UL → li items (3 li under DIV.WEB, 5 li under each of DIV.AI and DIV.CS, as drawn in the source).

OUTPUT:

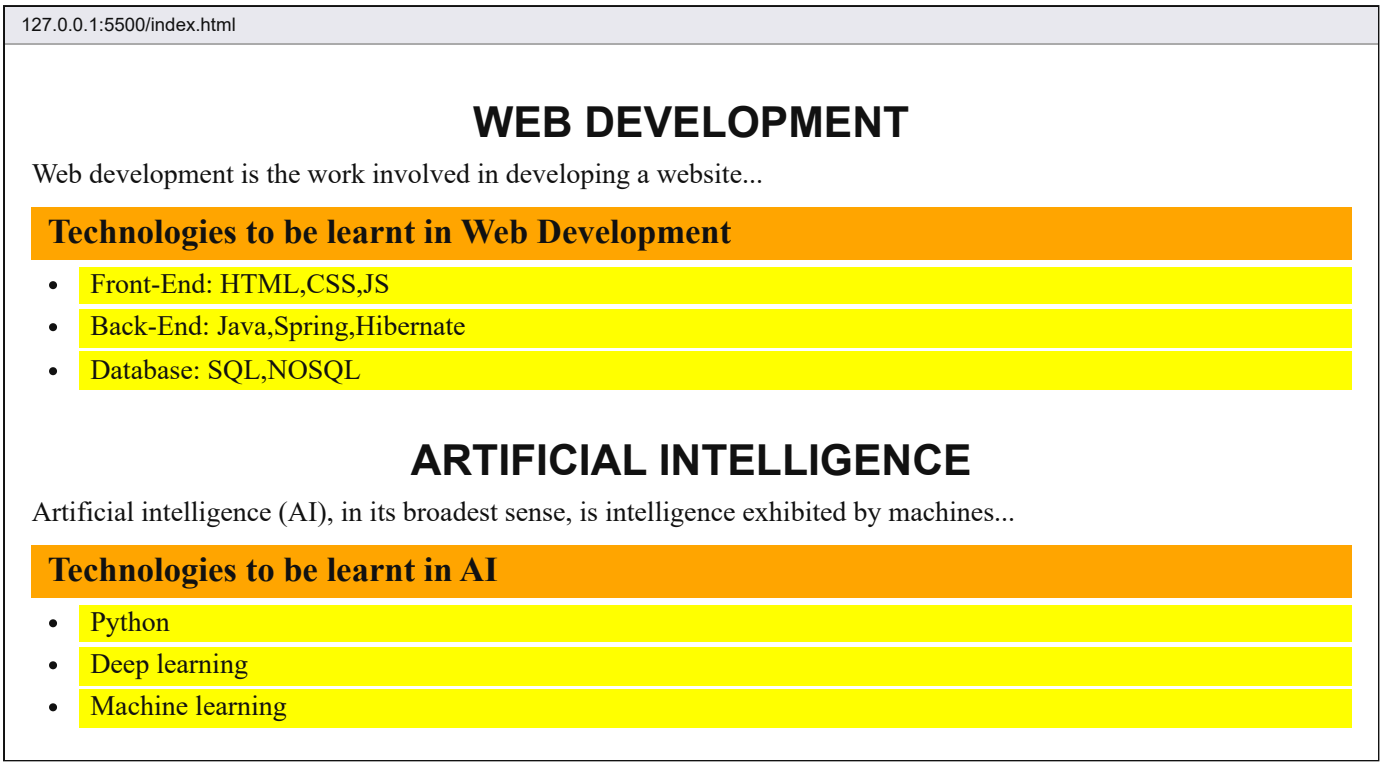


Figure 44: Output of index.html (html-5) with the descendant selectors `div h2` and `div li` applied.

In the above example the elements present inside the tag are descendent. If you can notice we have styled <li> and <h1> tag using descendant selector. This descendant selector will match all li and h1 elements in the example, because each of those elements has a div element as its ancestor.
The first simple selector within this selector represents the ancestor element a structurally superior element, such as a parent element, or the parent of a parent element, and so on. The second simple selector represents the descendant element we're trying to match. The combinator we use in a descendant selector is a whitespace character.

DIRECT CHILD:

In the above example, <h1> is directly present inside the tag so, <h1> is direct child. is not the direct child, if you observe it is not present directly inside the tag so it is descendent. Direct child is a Descendent but Descendent is not a direct child. Now let's see with the example, how to apply styling to direct child using child combinator.

style.css refer (html-5)

```
div > h1 {
  background-color: grey;
}
```

OUTPUT:

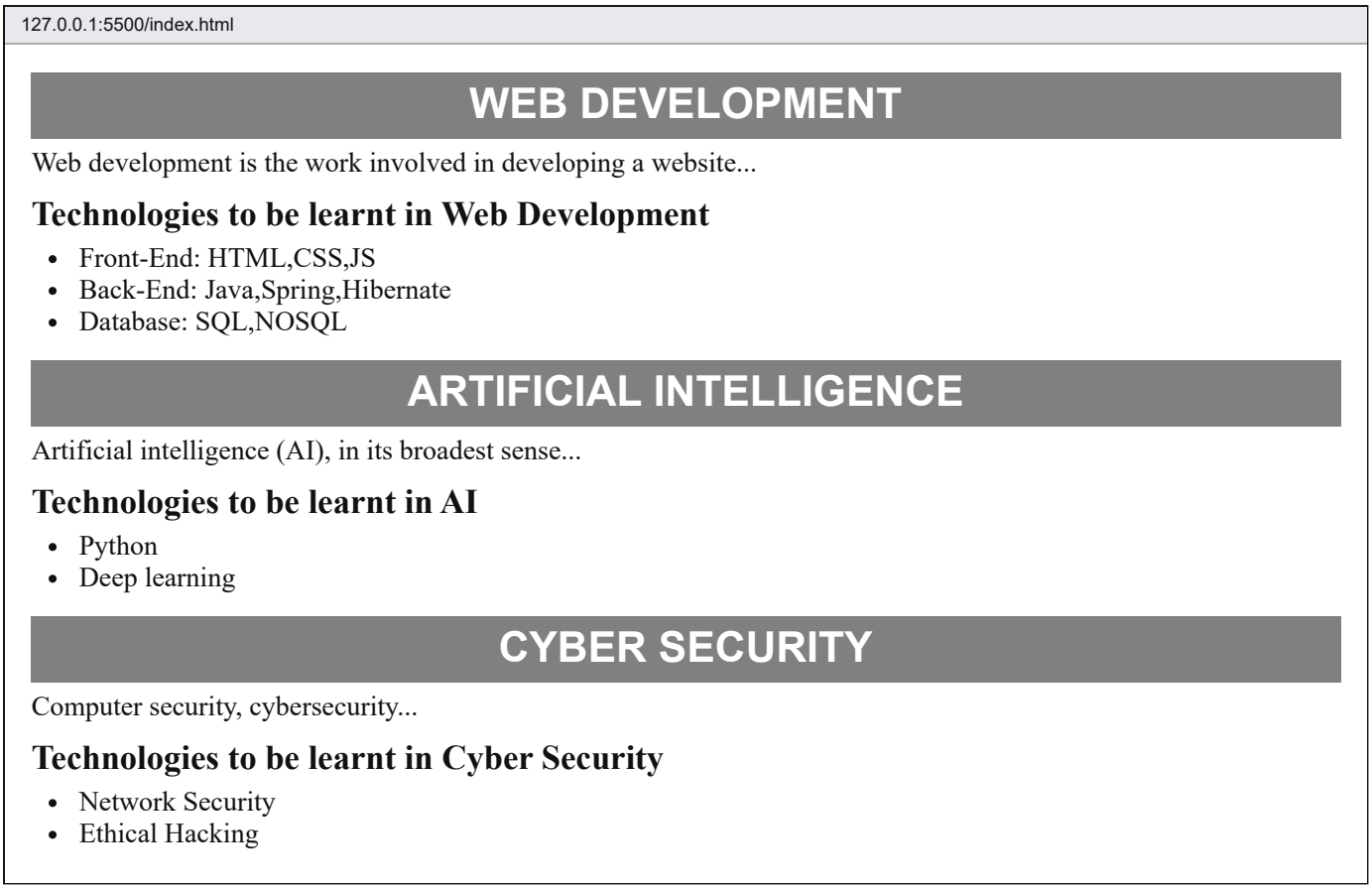


Figure 45: Output of index.html (html-5) with `div > h1` applied — the three h1 headings (Web Development, Artificial intelligence, Cyber Security) have a grey background.

In the above example, <h1> is direct child of tag. The child combinator (>) is placed between two CSS selectors. It matches only those elements matched by the second selector that are the direct children of elements matched by the first.
Example: Style the background color of body as grey.

index.html (html-6)

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Home Page</title>
        <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <h1>h1 HEADER</h1>
        <p class="firstp">p outside div</p>
        <div>
            <p id="fodp">p directly inside div</p>
            <span><p>p inside span inside div</p></span>
            <span><p>p inside span inside div</p></span>
            <p>p directly inside div</p>
            <div class="nesteddiv">
                <p>p inside nested div</p>
                <span><p>p inside span inside nested div</p></span>
                <span><p>p inside span inside nested div</p></span>
                <p>p inside nested div</p>
            </div>
            <p>p directly inside div</p>
        </div>
        <p>p outside div</p>
    </body>
</html>
```

style.css refer **(html-6)**

```css
body{
  background-color: grey;
}
```

OUTPUT:



Figure 46: Output of index.html (html-6) with the body background set to grey — "h1 HEADER" followed by the eleven paragraphs listed in the code.

To style the background color of body as grey we have to make use of element selector body.
Example: Style the background color h1 header as yellow and colour as red.
style.css refer (html-6)

```
body{
  background-color: grey;
}

h1{
  background-color: yellow;
  color: red;
}
```

OUTPUT

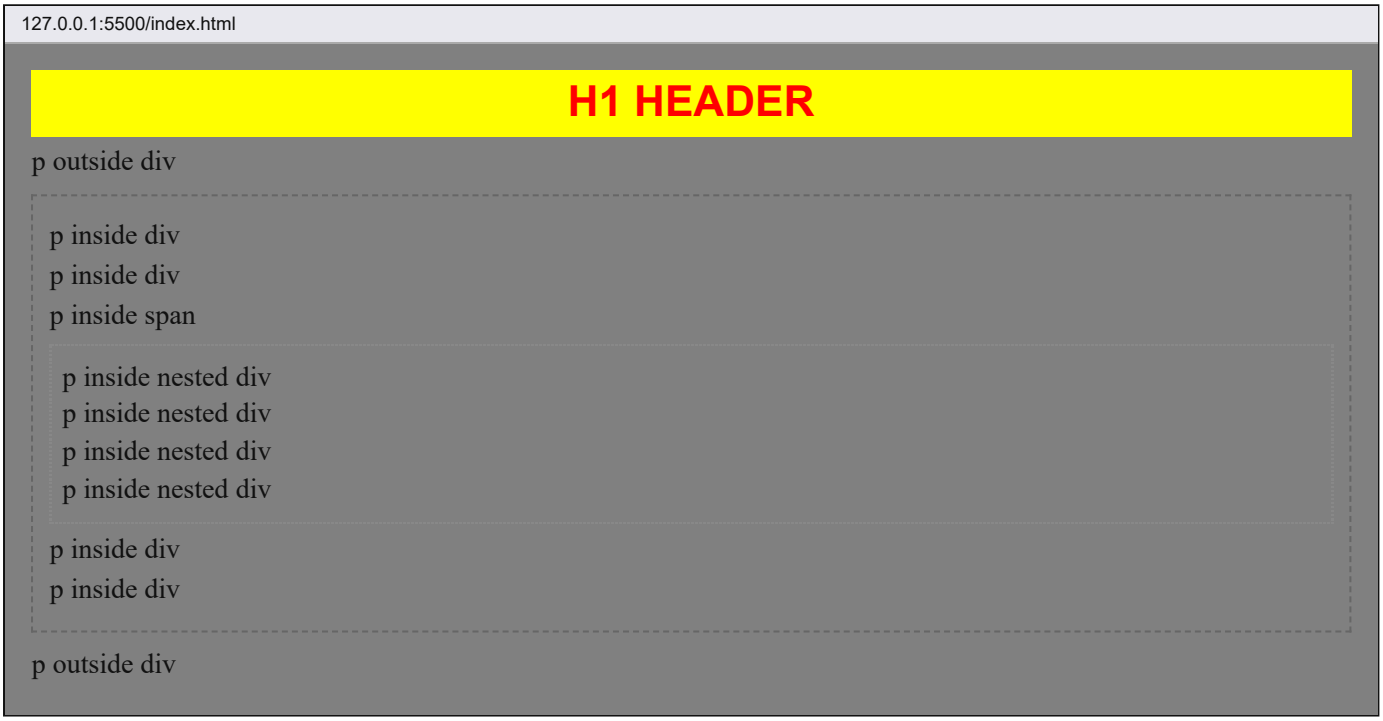


Figure 47: Output of index.html (html-6) with body{grey} and h1{yellow background, red text} — only the "h1 HEADER" heading is highlighted.

To style the h1 tag in this example you have to make use of element selector.
Example: Style the background color of all the p's outside div as powderblue and color as black.
style.css refer (html-6)

```
body{
  background-color: grey;
}
h1{
  background-color: yellow;
  color: red;
}
body > p{
  background-color: powderblue;
  color: black;
}
```

OUTPUT

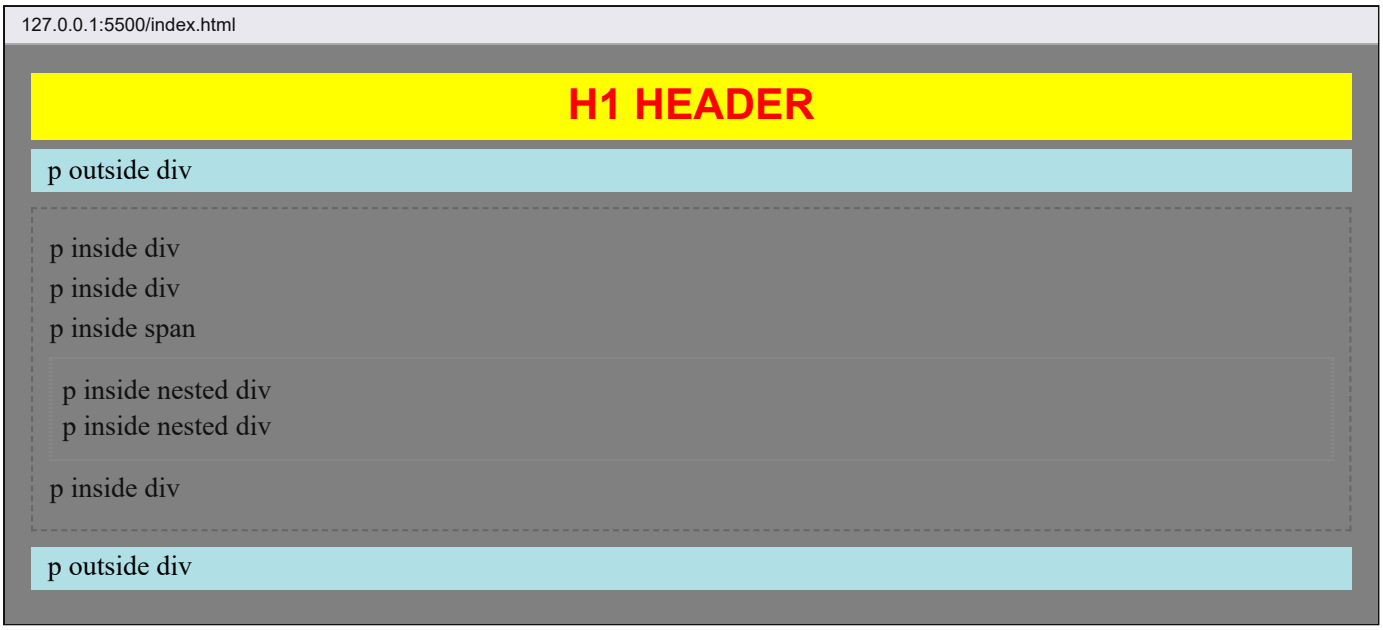


Figure 48: Output of index.html (html-6) with `body > p` applied — only the paragraphs that are direct children of body ("p outside div", the first and the last) are restyled.

Here we want to style p tag inside body tag, if you observe the above example there is a direct child relationship between body and p tag. Thus p tag is styled using direct child selector.
Example: Style the background color of all the p's inside div as orange and color as blue.

style.css refer (html-6)

```
body{
  background-color: grey;
}
h1{
  background-color: yellow;
  color: red;
}
body > p{
  background-color: powderblue;
  color: black;
}
div p{
  background-color: orange;
  color: blue;
}
```

OUTPUT

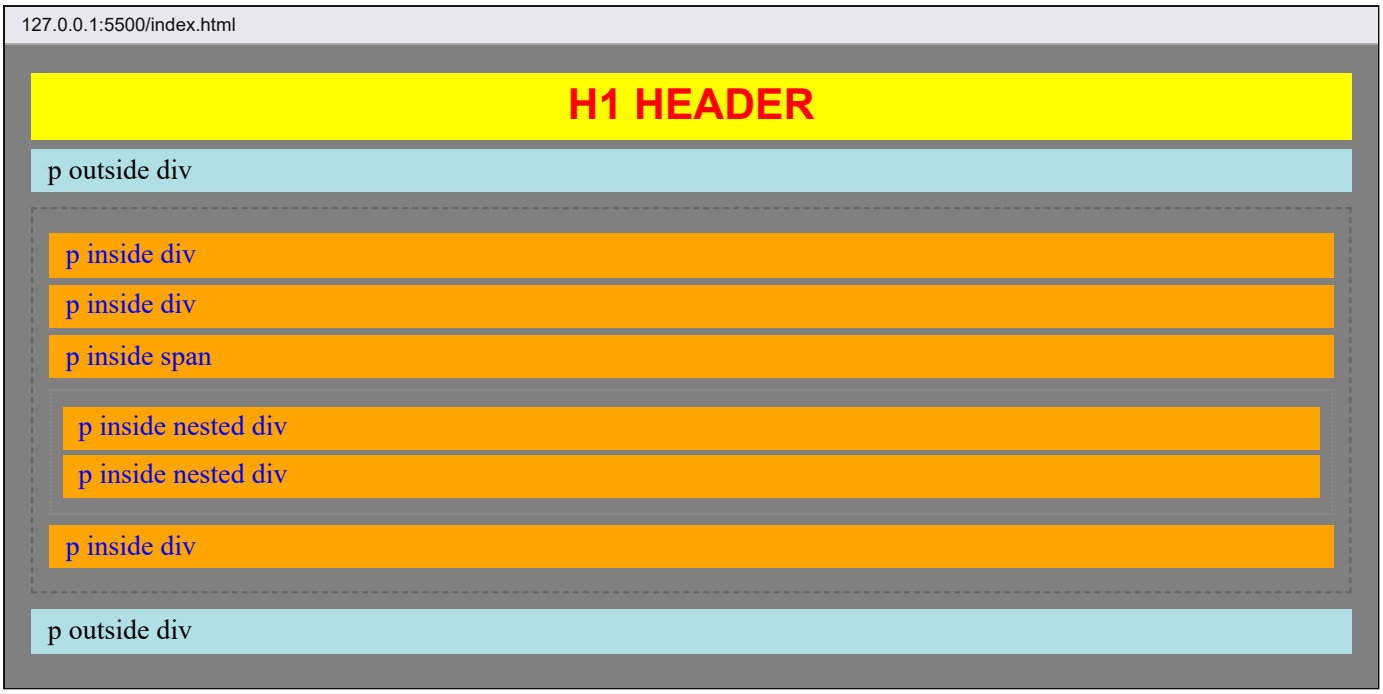


Figure 49: Output of index.html (html-6) with `div p` applied — all paragraphs inside the div (including those inside spans and in the nested div) are restyled orange with blue text.

Here we have to style p's inside div tag, if we observe there is descendant relationship between div and p tag. Thus styling is done using descendant selector.
Example: Style the background color of all p's directly inside div as cyan and color as brown.
style.css refer (html-6)

```
body{
  background-color: grey;
}
h1{
  background-color: yellow;
  color: red;
}
```

```
body > p{
  background-color: powderblue;
  color: black;
}
div p{
  background-color: orange;
  color: blue;
}
div > p{
  background-color: cyan;
  color: brown;
}
```

(Continuation of the style.css listing begun at the bottom of the previous page.)

OUTPUT



Figure 50: Output of index.html (html-6) with `div > p` applied — only the paragraphs directly inside the div are restyled (cyan background, brown text).

To style p tag which is directly inside div tag direct child selector is used.

Example: Style the background color of all p's inside nested div as navy and color as pink.

style.css refer (html-6)

```
body{
  background-color: grey;
}
h1{
  background-color: yellow;
  color: red;
}
body > p{
  background-color: powderblue;
  color: black;
}
```

```
div p{
  background-color: orange;
  color: blue;
}
div > p{
  background-color: cyan;
  color: brown;
}
.nesteddiv p{
  background-color: navy;
  color: pink;
}
```

(Continuation of the style.css listing begun at the bottom of the previous page.)

OUTPUT

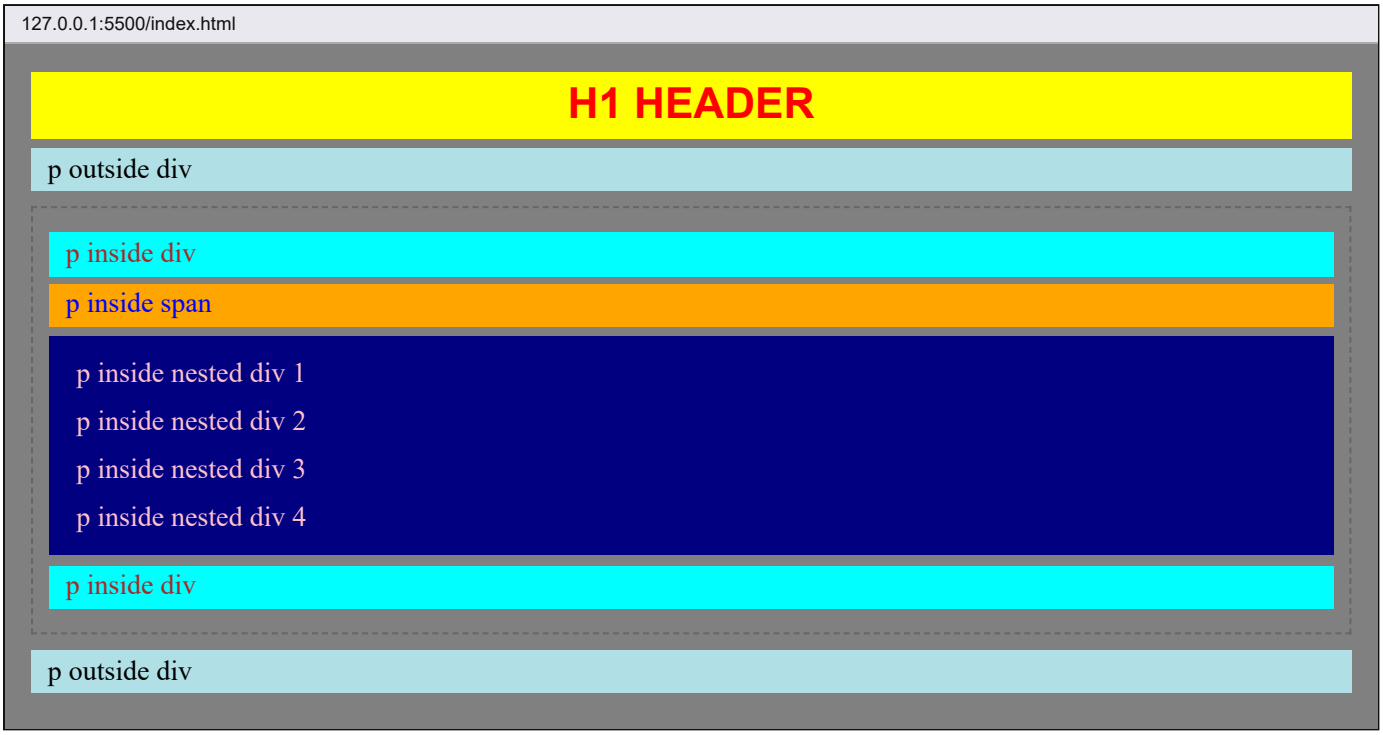


Figure 51: Output of index.html (html-6) with `.nesteddiv p` applied — the four paragraphs inside the nested div have a navy background and pink text.

To style p's inside nested div select based on class name, because nested div as some more information about class.

Example: Select the background color of only the first p inside outer div as teal and color as marron.

style.css refer (html-6)

```
body{
  background-color: grey;
}
h1{
  background-color: yellow;
  color: red;
}
body > p{
  background-color: powderblue;
  color: black;
}
```

```
div p{
  background-color: orange;
  color: blue;
}
div > p{
  background-color: cyan;
  color: brown;
}
.nesteddiv p{
  background-color: navy;
  color: pink;
}
#fodp{
  background-color: teal;
  color: maroon;
}
```

(Continuation of the style.css listing begun at the bottom of the previous page.)

OUTPUT



Figure 52: Output of index.html (html-6) with `#fodp` applied — the first paragraph inside the outer div (id="fodp") is restyled teal with maroon text.

To select that specific p you can assign it with unique id using id selector then style that specific element.

SIBLING SELECTOR:

Sibling selector further classified into two types

- General sibling
- Adjacent sibling

Let's understand sibling selector with an example.

index.html (html-7)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>H1 outside div</h1>
    <p>P1 outside div</p>
    <div>
        <h2>H2 inside div</h2>
        <p>P1 inside div</p>
        <p>P2 inside div</p>
        <h3>H3 inside div</h3>
    </div>
    <p>P2 outside div</p>
</body>
</html>
```

Tree Representation:

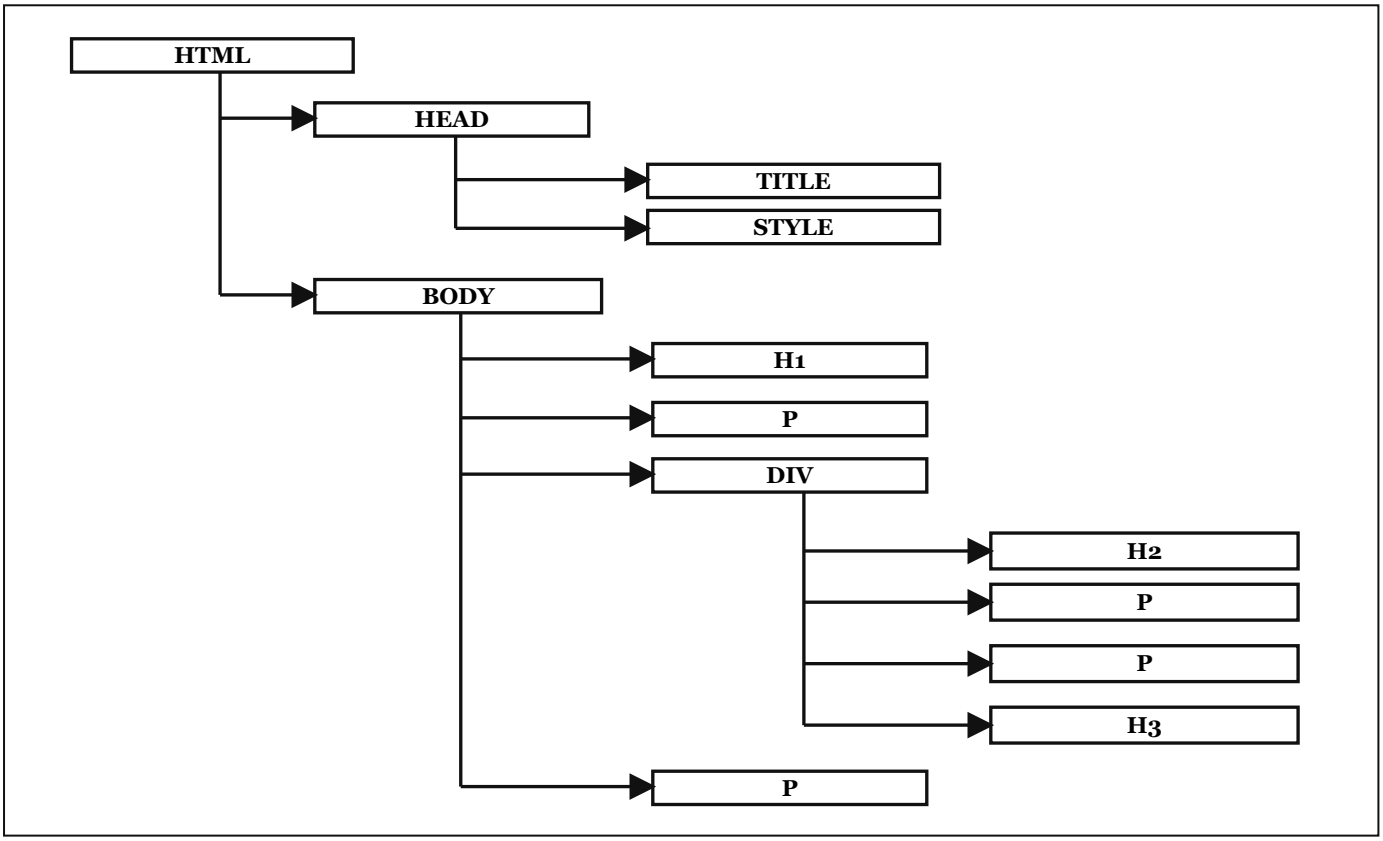


Figure 53: Tree representation of index.html (html-7) — HTML → HEAD (→ TITLE, STYLE) and BODY; BODY → H1, P, DIV and a final P; DIV → H2, P, P, H3. (In the photograph the first box under DIV is hard to read; it is drawn as H2 to match the code.)

In the above example, h1, p, div and p tag which is directly inside body tag is known as sibling, h2,p,p and h3 tag which is directly inside div tag is also known as sibling. These sibling which belongs to same parent is only known as General sibling. Now let's understand how to style these General sibling using css.

Example: Style all the general sibling of h1 and h2 tag which are p tag
style.css refer (html-7)

```
h1~p{
  background-color: orange;
}
h2~p{
  background-color: cyan;
}
```

OUTPUT:

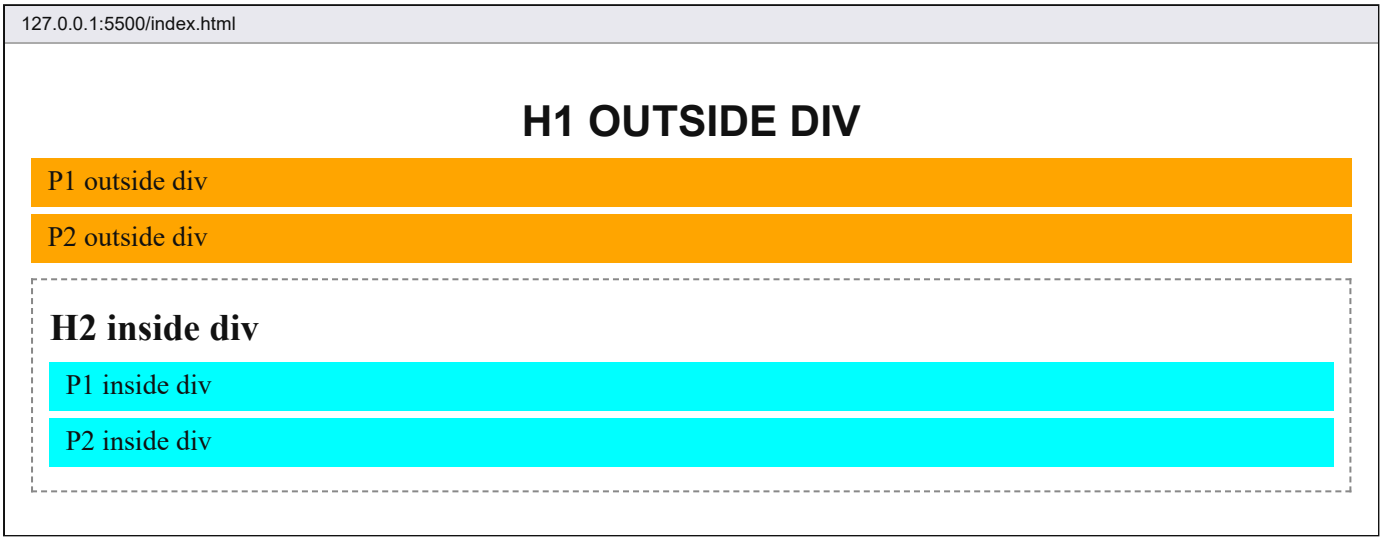


Figure 54: Output of index.html (html-7) with the general sibling selectors `h1~p` and `h2~p` — the paragraphs that follow h1 (P1 and P2 outside the div) and h2 (P1 and P2 inside the div) are shaded; the headings are not.

In this example, we applied styling for p tag which are general sibling of h1 and h2 tag. The general sibling selector selects all elements that are siblings of a specified element. The combinator used for general sibling is ~.

Adjacent sibling:

The adjacent sibling selector is used to select an element that is directly after another specific element. Sibling elements must have the same parent element, and "adjacent" means "immediately following". Let's now understand adjacent sibling with an example.

Example-1: Style the p tag which is adjacent sibling of h1 and h2 tag.

style.css (html-7)

```
h1+p{
   background-color: orange;
}
h2+p{
   background-color: cyan;
}
```

OUTPUT:

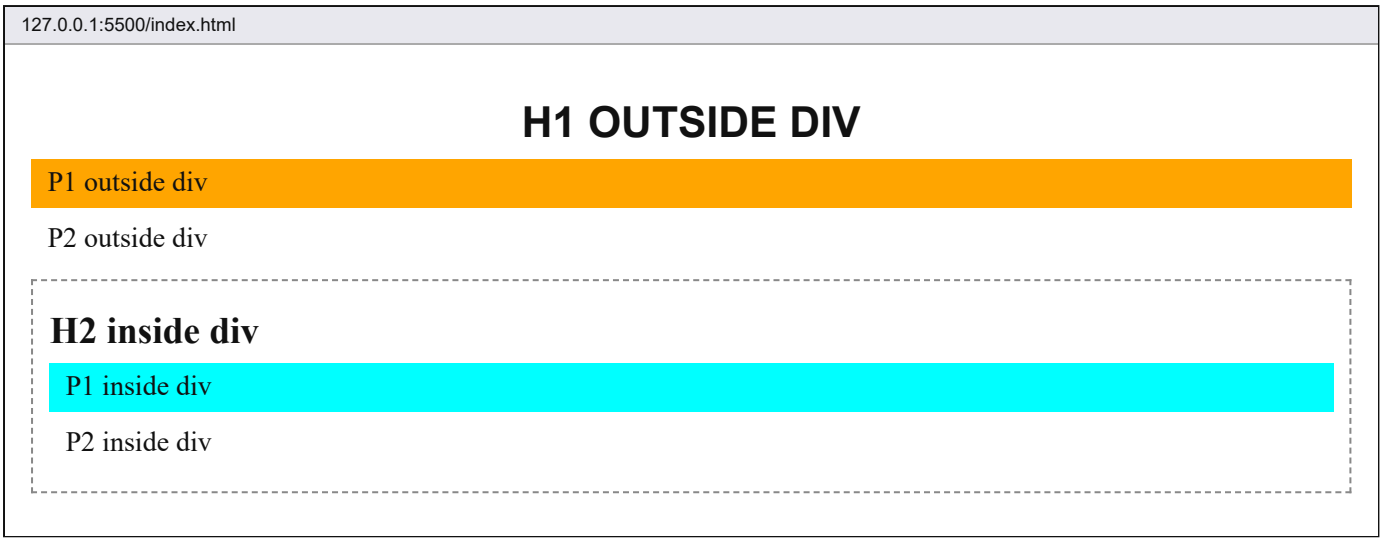


Figure 55: Output of index.html (html-7) with the adjacent sibling selectors `h1+p` and `h2+p` — only the paragraph immediately after the h1 (P1 outside div) and the paragraph immediately after the h2 (P1 inside div) are shaded.

In this example, styling is applied to p tag which is immediate sibling of h1 and h2 tag. The combinatory used for adjacent sibling is +.

Example-2: color the ordered list

index.html (html-8)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Lists Available on the webpage</h1>
    <ol class="firstline">
        <li>Popular langauages </li>
        <li>Popular Frameworks</li>
        <li>Highest Paying Jobs</li>
    </ol>
    <h3>Popular langauages</h3>
    <ul>
        <li class="first" >C</li>
        <li>Java   </li>
        <li>Python</li>
        <li>C++</li>
        <li>C#</li>
        <li>JavaScript</li>
    </ul>
    <h4>Popular Frameworks</h4>
    <ul>
        <li class="first" >Spring</li>
        <li>DJango  </li>
        <li>Node.JS</li>
        <li>.NET Core</li>
        <li>Flask</li>
        <li>React.JS</li>
    </ul>
    <h3>Highest Paying Jobs</h3>
    <ul>
        <li class="first" >Devops Engineer</li>
        <li>Full Stack Developer</li>
        <li>Data Scientist</li>
        <li>AI-Ml Engineer</li>
        <li>Cloud Engineer</li>
        <li>Cyber Security Expert</li>
    </ul>
</body>
</html>
```

style.css refer **(html-8)**

```css
ol{
  background-color: orange;
}
```

Tree Representation:

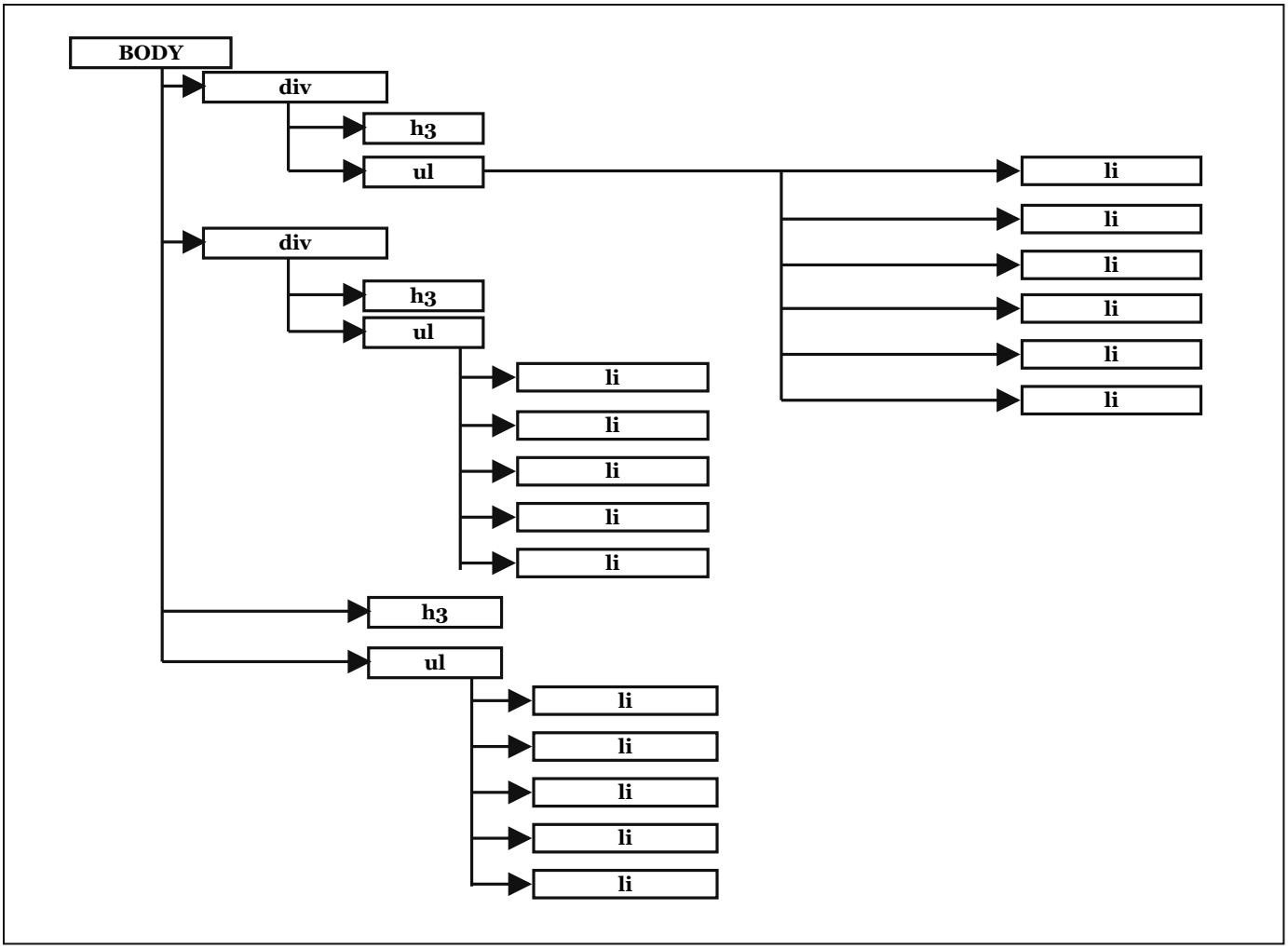


Figure 56: Tree representation of the body of index.html (html-8) — BODY → div (→ h3, ul → six li); div (→ h3, ul → five li); and a further h3 and ul (→ five li) drawn directly under BODY, exactly as in the source drawing.

Output:

127.0.0.1:5500/index.html

Lists Available in HTML

1. HTML
2. CSS
3. JavaScript
4. React

Unordered List

- Java
- Python
- C++
- Ruby

1. SQL
2. MySQL
3. MongoDB
4. PostgreSQL

- Git
- Docker
- AWS
- Linux

Figure 57: Output of index.html (html-8) with `ol{background-color: orange}` — only the ordered list ("Popular langauages / Popular Frameworks / Highest Paying Jobs") is shaded; the three unordered lists below are unchanged.

Example-3: Color all the unordered list.

style.css refer (html-8)

```
ul{
  background-color: cyan;
}
```

OUTPUT:



Figure 58: Output of index.html (html-8) with `ul{background-color: cyan}` — the three unordered lists are shaded; the ordered list at the top is not.

Example-4: color all li except first li inside ul. style.css refer (html-8)

```
.first ~ li{
  background-color: powderblue;
}
```

OUTPUT:

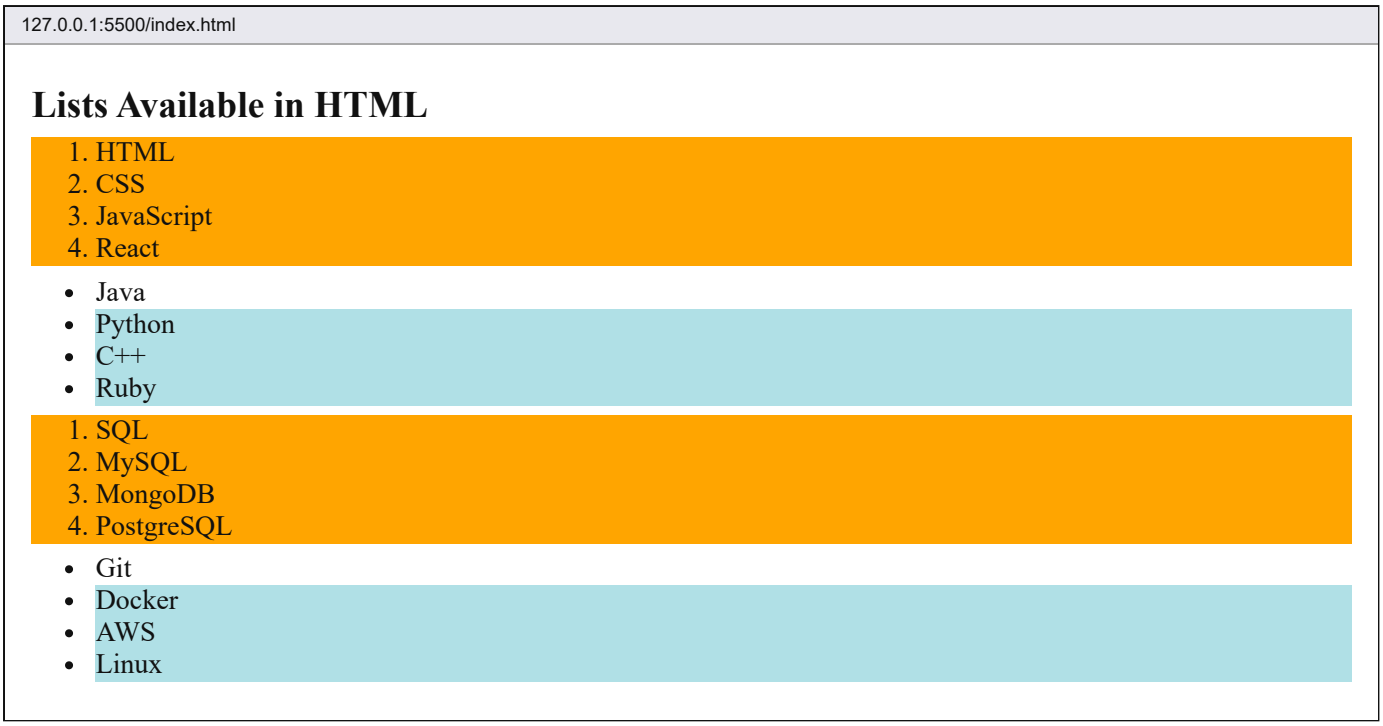


Figure 59: Output of index.html (html-8) with `.first ~ li` — in each unordered list all items except the first are shaded.

If we observe the li inside ul, the first li and other li have general sibling relationship and first li have class name thus using class selector and general sibling combinator we have styled the li except the first li.

Example-5: Color only the second li in ul

style.css refer **(html-8)**

```
.first + li{
  background-color: red;
}
```

OUTPUT:

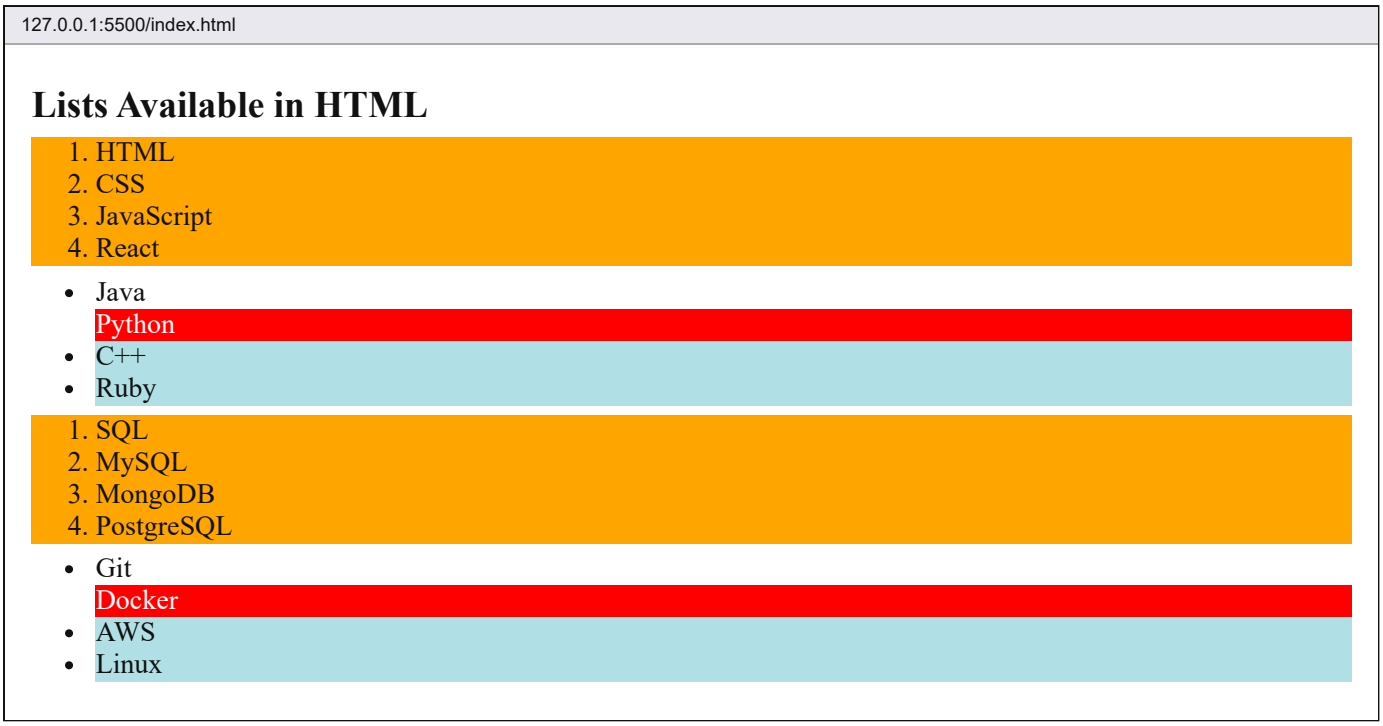


Figure 60: Output of index.html (html-8) with `.first + li` — only the second item of each unordered list is shaded.

Here we want to style the second li inside ul, if we notice html code clearly there is an adjacent sibling relationship between first li and second li thus styling of second li is done using Adjacent sibling selector.

Example-6: Color all li after second li in ul.

style.css refer **(html-8)**

```
.first + li ~ li{
  background-color: navy;
}
```

OUTPUT:

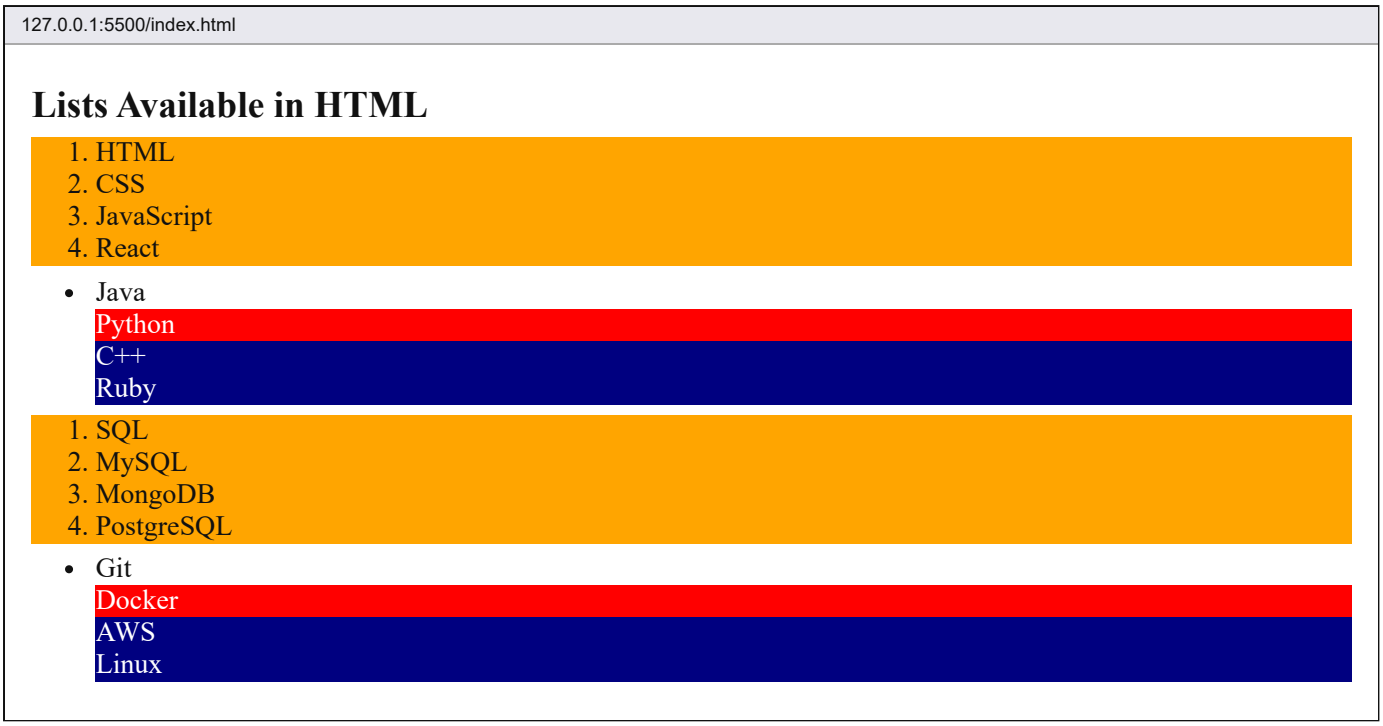


Figure 61: Output of index.html (html-8) with `.first + li ~ li` — in each unordered list the items after the second are shaded navy.

In the above example, for that we have to select second li first by using adjacent sibling relationship. Second li and other li have general sibling relationship so styling will be done using general sibling combinator.

Example-7: Color only the first ul

style.css refer **(html-8)**

```
ol + h3 + ul{
   background-color: teal;
}
```

OUTPUT:

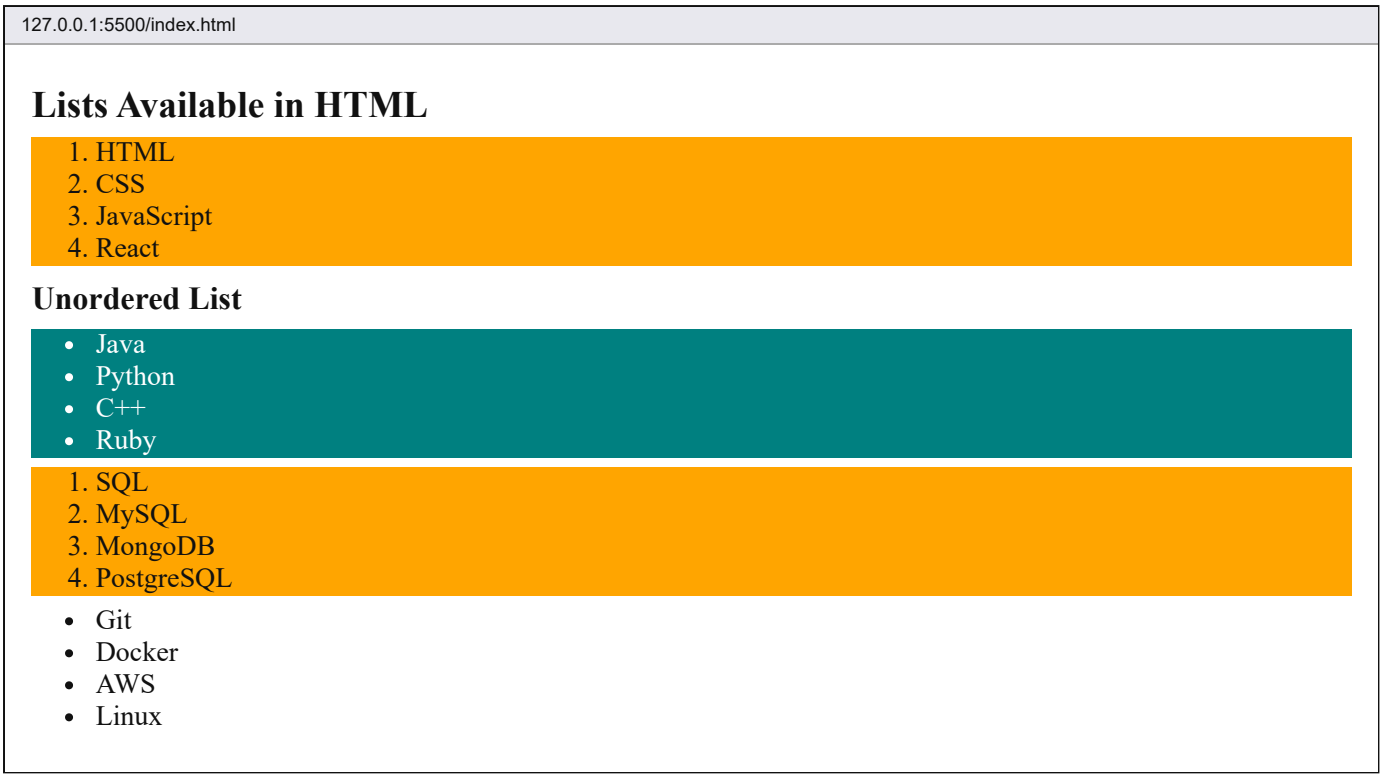


Figure 62: Output of index.html (html-8) with `ol + h3 + ul` — only the first unordered list (under "Popular langauages") is shaded.

To style first ul, we have adjacent relationship between ul and h3 has there are multiple h3 and ul available. Here, we have selected first h3 by finding adjacent relationship between ol and h3 then selected first ul using adjacent combinator.

Example-8: Color only first two li in every ul.

style.css refer **(html-8)**

```
.first{
  background-color: yellow;
}
```

OUTPUT:



Figure 63: Output of index.html (html-8) with `.first` (yellow) applied together with the earlier sibling rules. The shading is too faint to make out in the photograph.

Here, first li is styled using class name. Second li is styled by generating adjacent sibling relationship between first li and second li.

Attribute Selector:

The CSS Attribute Selector is used to select an element with some specific attribute or attribute value. It is an excellent way to style the HTML elements by grouping them based on some specific attributes and the attribute selector will select those elements with similar attributes.

There are three different types of attribute selector:

- Simple Attribute selector
- Exact value Attribute selector
- Partial value Attribute selector

Simple Attribute Selector:

If you want to select elements that have a certain attribute, regardless of that attribute's value, you can use a simple attribute selector. Let us now understand this with an example.

Example: To select all img tag that have an alt attribute and add the solid border to the image.

index.html (html-9)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <img src="Java.png" height="200" width="200" alt="Image not found"> <br>
    <img src="bootstrap.svg" height="200" width="200" > <br>
    <img src="Python.svg" height="200" width="200" alt="Image not found"> <br>
    <img src="js.svg" height="200" width="200" > <br>
</body>
</html>
```

style.css refer **(html-9)**

```
img[alt]{
  border: red solid 2px;
}
```

OUTPUT:

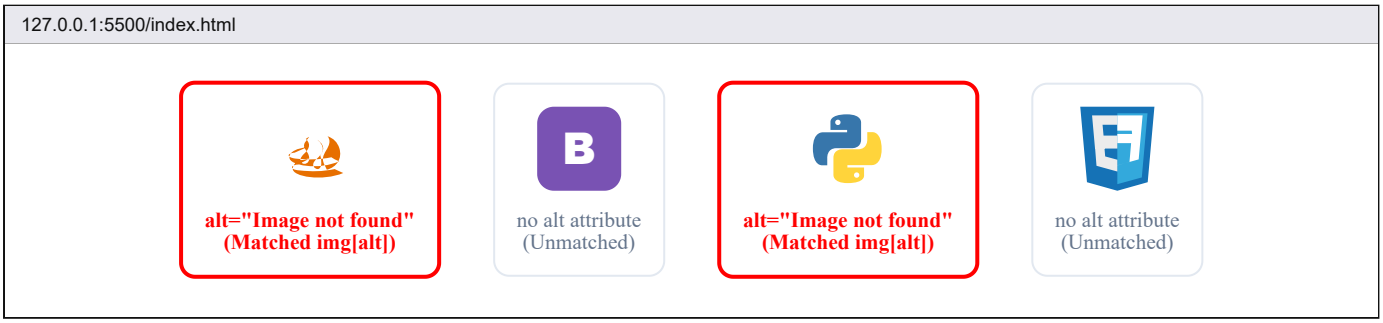


Figure 64: Output of index.html (html-9) with `img[alt]` — the Java and Python images (which have an alt attribute) get the red solid border; the Bootstrap and JS images (no alt) do not.

Example: To select all input tags based on multiple attributes.

index.html (html-10)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <fieldset>
        <legend>Sign Up Form</legend>
        <form>
            <label> Name: </label>
            <input type="text" name="name" placeholder="Rohit"><br>
            <label> Mobile: </label>
            <input type="tel" name="mob"><br>
            <label> Email: </label>
            <input type="email" name="email" placeholder="rohit@gmail.com"><br>
            <label> Password: </label>
            <input type="password" name="pwd"><br>
            <label> Confirm Password: </label>
            <input type="password" name="cpwd"><br>
            <input type="submit">
        </form>
    </fieldset>
</body>
</html>
```

style.css refer **(html-10)**

```css
input[type][placeholder]{
  border: 2px solid red ;
}
```

OUTPUT:

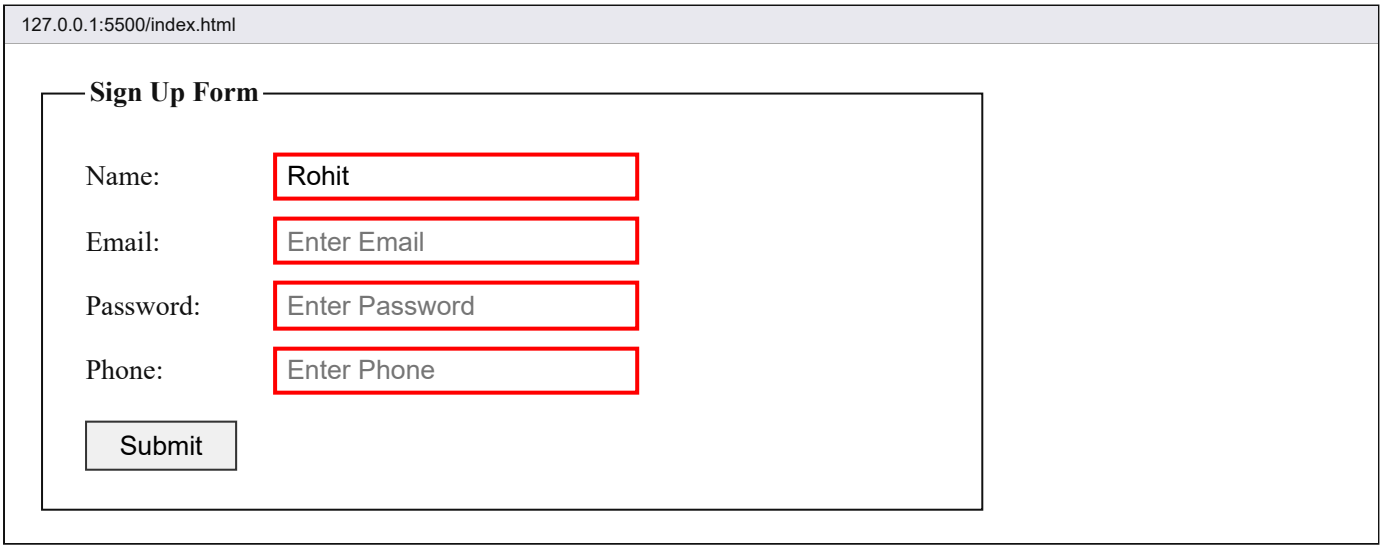


Figure 65: Output of index.html (html-10) with `input[type][placeholder]` — the "Sign Up Form" fieldset; only the Name and Email inputs (which have both type and placeholder attributes) get the red border.

Example: To select an attribute based on exact value, partial value

index.html (html-11)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="main font1">
        <h1>Web Development</h1>
        <p>
            Web development is the work involved in developing a website for the
            <a href="https://www.britannica.com/technology/Internet" target="_blank"> Internet</a> (World Wide Web)
            or an intranet (a private network).[1] Web development can range from developing a simple single
            <a href="https://en.wikipedia.org/wiki/Static"  target="_blank">static page</a> of plain text to
            complex web applications, electronic businesses, and social network services. A more comprehensive
            list of tasks to which Web development commonly refers, may include Web engineering, Web design,
            Web content development, client liaison, client-side/server-side scripting,
            <a href="https://en.wikipedia.org/wiki/Web_server" target="_blank">Web server</a> and network
            security configuration, and e-commerce development.
        </p>
        <a href="https://www.google.com/" target="_blank">To google about development click here</a><br><br>
        <h3>Layers of a web application:</h3>
        <a href="#FE">Front-end</a><br>
        <a href="#BE">Back-end</a><br>
        <a href="#Data">Database</a><br>
    </div>
    <div class="sub font2">
        <h2 id="FE">Front-End</h2>
        <p>
          Front-end development is the process of building components that interact with users. Examples are the
          user interface, buttons, user-entered data, websites, and user experience (UX) features. The front
          end aims at meeting user requirements and delivering a positive user experience.
        </p>
        <p>
          The HyperText Markup Language (HTML) programming language, which defines the structure and meaning of
          web content, is a building block for front-end development. Through HTML, browsers display text or load
          elements, rendering webpages, which contain hyperlinks and links to other webpages, for users.</p>
        <p>
          Cascading style sheets (CSS) is the standard language that specifies how to display HTML content: fonts,
           foreground and background colors, etc. With CSS, you can control the design layout and its components
           for various devices like desktops, tablets, and smartphones. Examples of the components are the header,
           body, footer, content, asides, and sections.
        </p>
        <p>
          JavaScript (JS) extends the functionality of websites beyond HTML and CSS.Webpages can refresh themselves
          dynamically and respond to user actions without page reloads or other changes.You can model animated UI
          components, such as pop-ups, image sliders, and extensive navigation menus.
        </p>
        <img src="frontend.png" alt="pic not found">
    </div>
    <div class="sub font2">
        <h2 id="BE">Back-End</h2>
        <p>
          Back-end development refers to the development of server-side logic that powers websites and apps from
          behind the scenes. It includes all the code needed to build out the database, server, and application.
          From database migrations to API integrations to setting up the server-side technologies that make a
```

```
57            website tick, a back-end web developer may be the talent you need to get your next web project off
the ground.
58          </p>
59          <p>
60            Inorder to handle backend of a given application the developers should have extensive knowledge on
backend
61            languages like Java, Python, JavaScript, C++ etc
62          </p>
63          <img src="backend.png" alt="pic not found">
64      </div>
65      <div class="sub font2">
66          <h2 id="Data">Database</h2>
```

index.html (html-11) — continued

```
67          <p>
68            A database is an organized collection of data stored in a computer system and usually controlled by
a database
69            management system (DBMS). The data in common databases is modeled in tables, making querying and
processing
70            efficient. Structured query language (SQL) is commonly used for data querying and writing.
71          </p>
72          <p>
73            Relational databases gained popularity in the 1980s, and object-oriented databases in the 1990s.
NoSQL
74            databases emerged more recently in response to the internet's rapid expansion and the demand for
unstructured
75            data processing at a faster rate. These days, self-driving databases and cloud databases are
revolutionizing
76            the ways in which data is gathered, stored, handled, and used.
77          </p>
78          <p>
79            As a developer, we have lot of choices to manage database we have mainly two types of databases.
80            The most popular databases are
81          </p>
82          <p>
83            SQL- Examples: Oracle, MySQL, PostgreSQL
84          </p>
85          <p>
86            NoSQL- Examples: MongoDB, Redis
87          </p>
88          <img src="database.png" alt="pic not found">
89      </div>
90      <div class="end font3">
91          <p><b>Also read about</b></p>
92          <a href="">Artificial Intelligence</a><br>
93          <a href="">Software Testing</a><br>
94          <p><b>Usefull PDF's for download</b></p>
95          <a href="">Full Stack Java Developer</a><br>
96          <a href="">Web Development Intro</a><br>
97      </div>
98  </body>
99  </html>
```

style.css refer (html-11)

```css
.main{
    background-color: #DC3D24 ;
    color: #232B2B ;
}
.font1{
    font-family: cursive;
    font-weight: bolder;
}
.sub{
    background-color: #221E1D;
    color: #ECEAE0;
}
.font2{
    font-family: monospace;
    font-weight: bold;
}
.end{
    background-color: #AC2832;
    color: #DFD297;
}
.font3{
    font-family: sans-serif;
    font-weight:normal ;
}
a[href="https://www.google.com/"]{
    color: yellow;
}
```

style.css (html-11) — continued

```css
a[href *= "wikipedia"]{
    color: cyan;
}
a [href ^= "#"]{
    color: #f5ad42;
}
a[href $= ".html"]{
    color: #0ec2eb;
}
a[href $= ".pdf"]{
    color: grey;
}
a[href $= ".pdf" i]{
    color: grey;
}
div [class ~= "font1"]{
    border: 2px solid blue;
}
```

OUTPUT:



Figure 66: Output of index.html (html-11) with the attribute selectors applied — the sections use the .main, .sub and .end styles and the links are recoloured by the a[href...] rules.

In this example,

The [attribute = "value"] selector is used to select elements with a specified attribute and value.

The [attribute *= "value"] selector is used to select elements whose attribute value contains a specified value.

The [attribute ^= "value"] selector is used to select elements whose attribute value begins with a specified value.

The [attribute $= "value"] selector is used to select elements whose attribute value ends with a specified value.

The [attribute ~= "value"] selector is used to select elements with an attribute value containing a specified word.

Example-1:

index.html (html-12)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2 lang="en">Hello!</h2>
    <h2 lang="en-us">Greetings!</h2>
    <h2 lang="en-au">G'day!</h2>
    <h2 lang="fr">Bonjour!</h2>
    <h2 lang="hi">Namaste!</h2>
    <h2 lang="es">Hola!</h2>
</body>
</html>
```

style.css refer **(html-12)**

```
h2[lang^="en"]{
   color: navy;
}
```

OUTPUT:

Hello!

Greetings!

G'day!

Bonjour!

Namaste!

Hola!

Figure 67: Output of index.html (html-12) with `h2[lang^="en"]` — the three headings whose lang value begins with "en" (Hello!, Greetings!, G'day!) are navy; the others (Bonjour!, Namaste!, Hola!) keep the default colour. (The photograph is black-and-white, so navy and black look alike there.)

Example-2:

style.css refer (html-12)

```
h2[lang|="en"]{
   color: red;
}
```

OUTPUT:

Hello!

Greetings!

G'day!

Bonjour!

Namaste!

Hola!

Figure 68: Output of index.html (html-12) with `h2[lang|="en"]` — Hello!, Greetings! and G'day! are red (shown as grey in the black-and-white photograph); Bonjour!, Namaste! and Hola! are unchanged.

In the above two examples, starts with (^) and pipe character (|) both will give same result but actually ^ matches only en and | matches en-. Depending on the requirement you can make use of ^ or |.

Pseudo-classes

A pseudo-class can be defined as a keyword which is combined to a selector that defines the special state of the selected elements. It is added to the selector for adding an effect to the existing elements based on their states. A pseudo-class starts with a colon **(:)**.

There are three types of pseudo-classes available:

- Tree structural based pseudo-classes
- Dynamic pseudo classes
- UI States pseudo classes

Tree structural based pseudo-classes:

CSS structural pseudo classes are part of the CSS pseudo classes. Let's understand different CSS structural pseudo classes with an examples.

➤ **Only-child pseudo class:** Selects a specified element if it is the only child element under the parent element.

Example: selects elements when they are the only child element of another element.

index.html (html-13)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
</head>
<body>
    <div>
     <p >Only Child p</p>
    </div>
    <div>
     <p>First Child p</p>
     <p>Second Child p</p>
    </div>
    <div>
     <p>First Child p</p>
     <div>
         <p >Only Child p</p>
     </div>
     <p>Second Child p</p>
    </div>
</body>
</html>
```

style.css refer **(html-13)**

```
p:only-child{
  background-color: orange;
}
```

Tree representation:

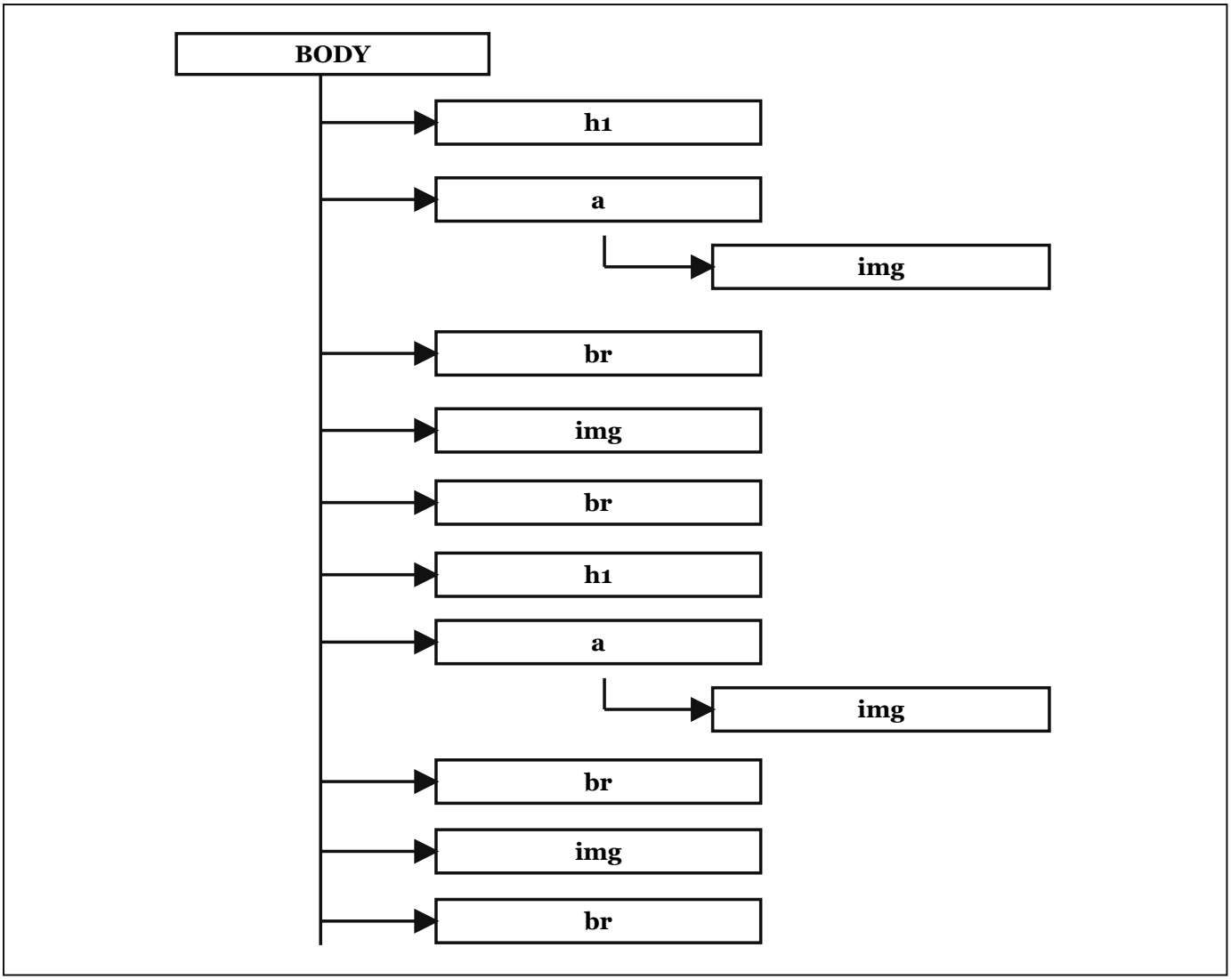


Figure 69: Tree representation of the body of index.html (html-13) — BODY → h1; a (→ img); br; img; br; h1; a (→ img); br; img; br, as drawn in the source.

OUTPUT:

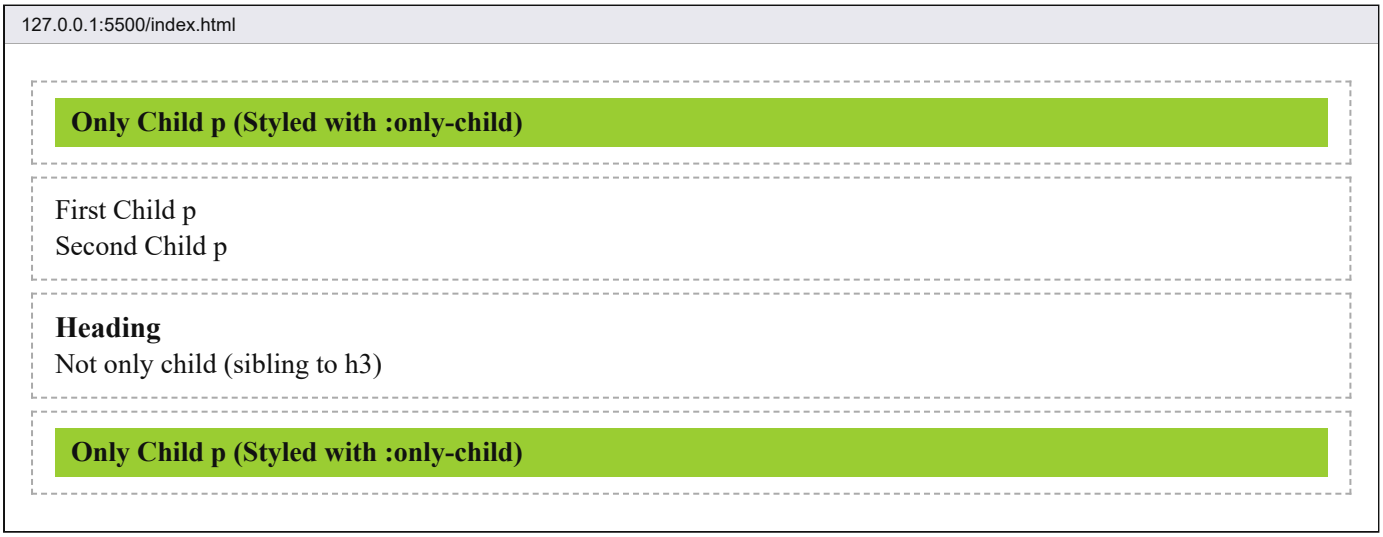


Figure 70: Output of index.html (html-13) with `p:only-child` — the two "Only Child p" paragraphs are shaded; the "First Child p" and "Second Child p" paragraphs are not.

Example: To add a border to any image that's the only child of another element.

index.html (html-14)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Java</h1>
    <a href="https://en.wikipedia.org/wiki/Java_(programming_language)">
     <img src="java1.png" height="250px" width="250px" alt="">
    </a><br>
    <img src="James.jpg" alt="" height="250px" width="250px" >
    <br>
    <h1>Python</h1>
    <a href="https://en.wikipedia.org/wiki/Python_(programming_language)">
     <img src="python1.svg" alt="" height="250px" width="250px" >
     <img src="Guido.jpg" alt="" height="250px" width="250px" ></a><br>
</body>
</html>
```

style.css refer **(html-14)**

```
img:only-child{
  border: 4px red solid;
}
```

Tree Representation:

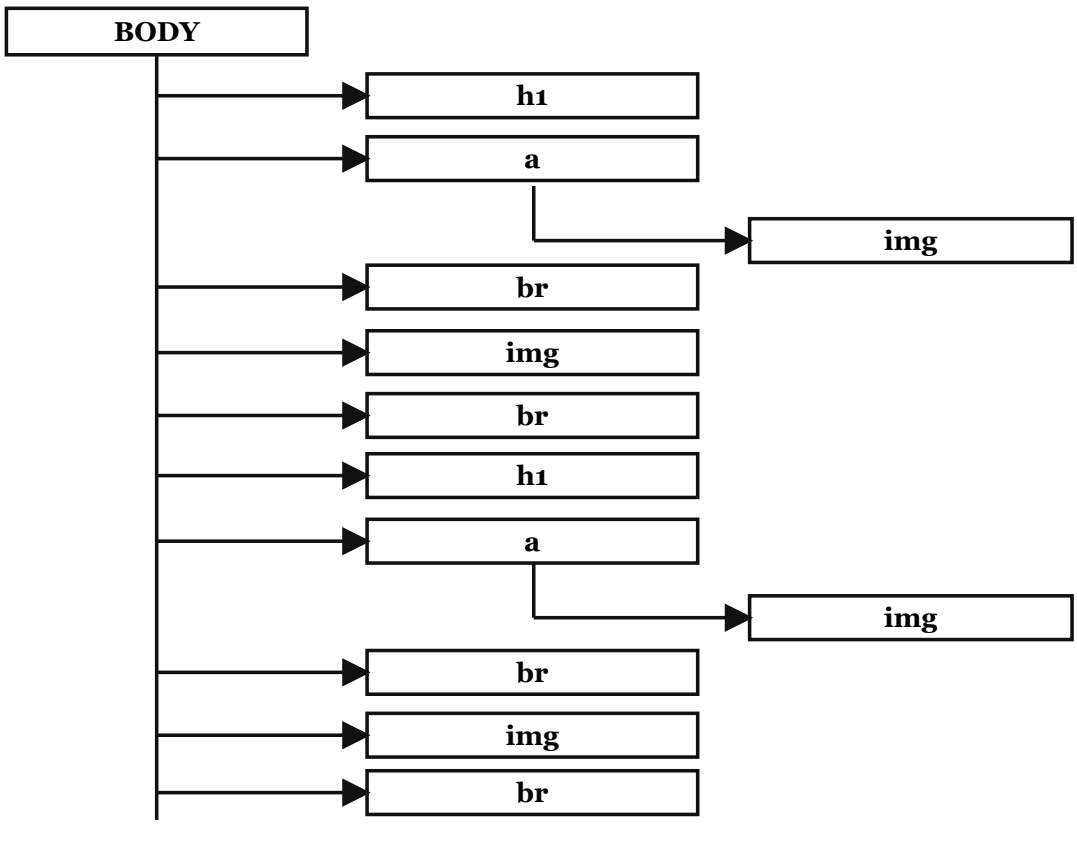


Figure 71: Tree representation of the body of index.html (html-14) — BODY → h1; a (→ img); br; img; br; h1; a (→ img); br; img; br, as drawn in the source (the second h1 box has no arrow in the original drawing).

OUTPUT:

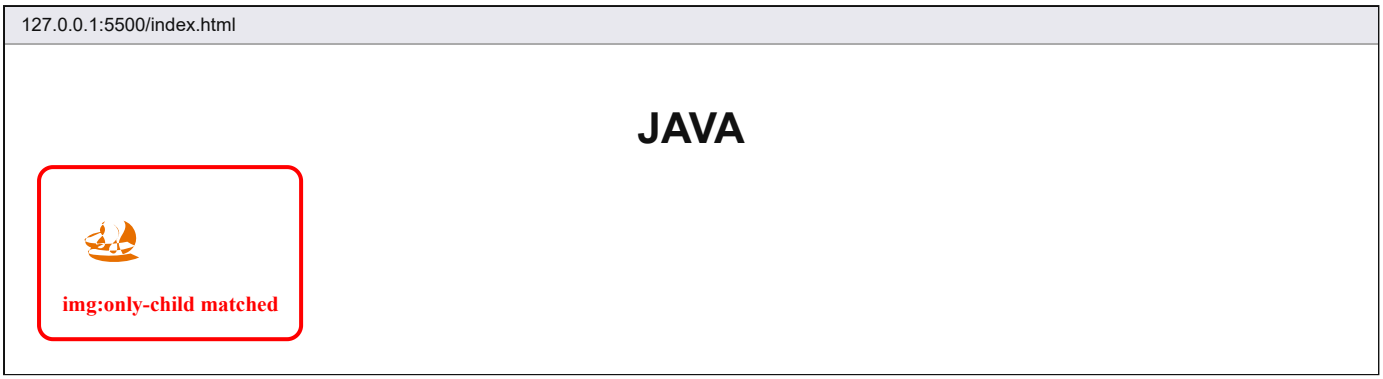


Figure 72: Output of index.html (html-14) with `img:only-child` — under the headings "Java" and "Python", the logo image (the only child of its link) has the red border, while the portrait photograph that sits directly in the body does not.

Example: To select any image that's the only-child and is direct child of an element.

index.html (html-15)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Java</h1>
    <a href="https://en.wikipedia.org/wiki/Java_(programming_language)">
     <span>
         <img src="java1.png" height="250px" width="250px" alt="">
     </span>
    </a><br>
    <img src="James.jpg" alt="" height="250px" width="250px" ><br>
    <h1>Python</h1>
    <a href="https://en.wikipedia.org/wiki/Python_(programming_language)">
     <img src="python1.svg" alt="" height="250px" width="250px" ></a><br>
    <img src="Guido.jpg" alt="" height="250px" width="250px" ><br>
</body>
</html>
```

style.css refer **(html-15)**

```
a[href]> img:only-child{
  border: 4px red solid;
}
```

Tree Representation:

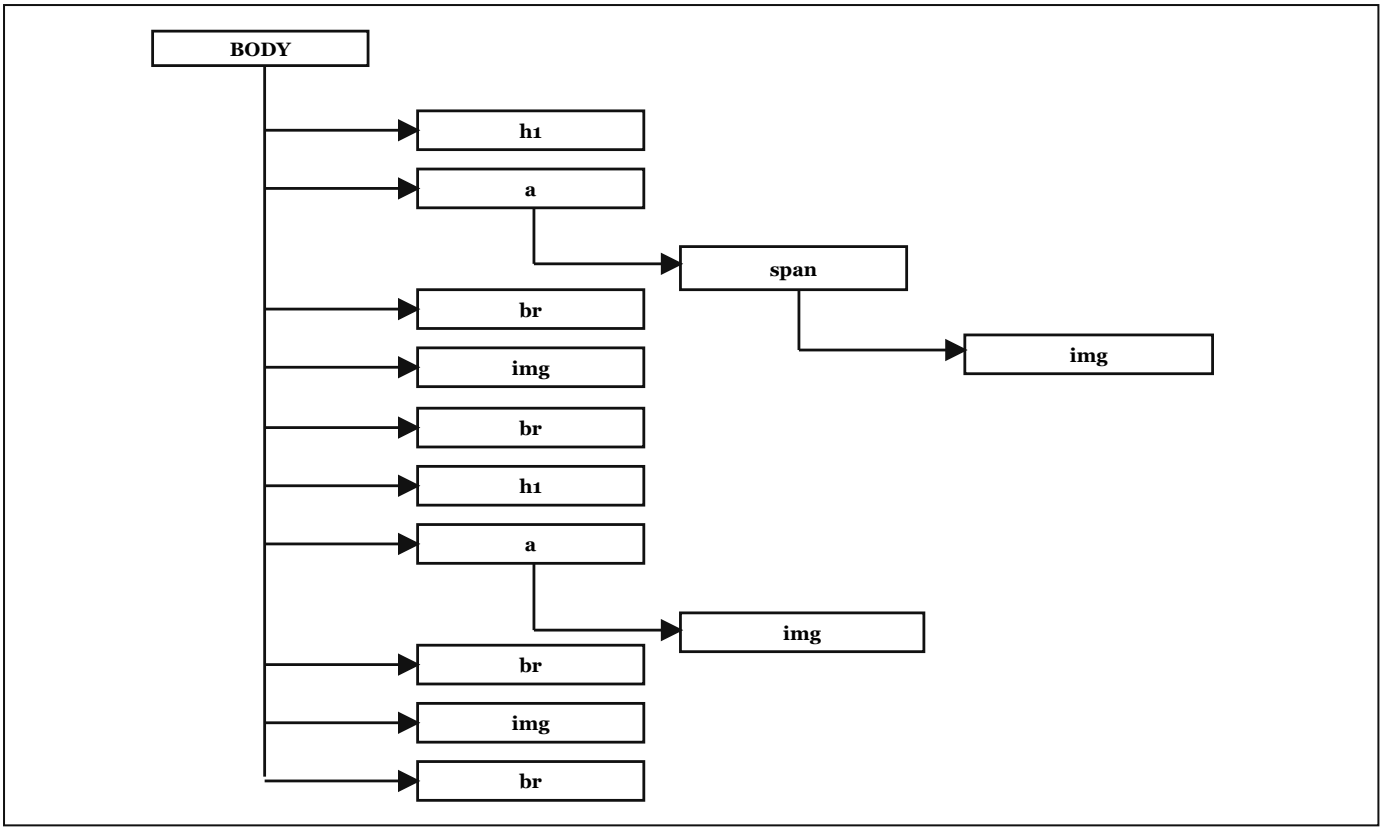


Figure 73: Tree representation of the body of index.html (html-15) — BODY → h1; a (→ span → img); br; img; br; h1; a (→ img); br; img; br, as drawn in the source.

OUTPUT:

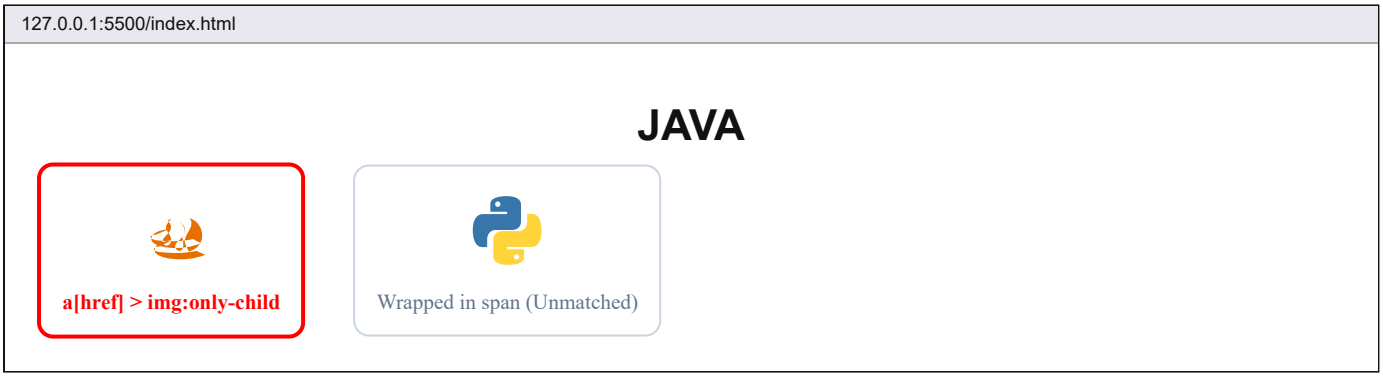


Figure 74: Output of index.html (html-15) with `a[href]> img:only-child` — only the image that is the only child directly inside the link is bordered (visible on the Python logo; the Java logo is inside a span, so it is not a direct child of the link and shows no border).

➤ **Only-of-type pseudo class:** Selects a specified child element if it is the only child element of its type under the parent element.

Example:

index.html (html-16)

```
 1  <!DOCTYPE html>
 2  <html lang="en">
 3  <head>
 4      <title>Document</title>
 5      <link rel="stylesheet" href="style.css">
 6  </head>
 7  <body>
 8      <p>Python is an <a href="https://en.wikipedia.org/w/index.php?
title=Interpreted_language&redirect=no">interpreted,</a>high-level and general-purpose programming language.
Python programming language (latest Python 3) is being used in web development, and Machine Learning
applications, along with all cutting-edge technology in Software Industry. Python language is being used by
almost all tech-giant companies like - Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.</p>
 9      <p>Java is a high-level, class-based, object-oriented programming language that is designed to have as
few implementation dependencies as possible. It is a <a href="https://en.wikipedia.org/wiki/General-
purpose_programming_language">general-purpose programming language</a> intended to let programmers write once,
run anywhere (WORA),[16] meaning that compiled Java code can run on all platforms that support Java without the
need to recompile.[17] Java applications are typically compiled to bytecode that can run on any <a
href="https://en.wikipedia.org/wiki/Java_virtual_machine">Java virtual machine</a> (JVM) regardless of the
underlying computer architecture. The syntax of Java is similar to C and C++, but has fewer low-level facilities
than either of them. The Java runtime provides dynamic capabilities (such as reflection and runtime code
modification) that are typically not available in traditional <a
href="https://en.wikipedia.org/wiki/Compiled_language">compiled</a> languages.</p>
10      <p>JavaScript is a high-level, often <a href="https://en.wikipedia.org/wiki/Just-in-
time_compilation">just-in-time compiled</a> language that conforms to the ECMAScript standard.[11] It has <a
href="https://en.wikipedia.org/wiki/Dynamic_programming_language">dynamic typing</a>, prototype-based object-
orientation, and first-class functions. It is multi-paradigm, supporting event-driven, functional, and imperative
programming styles. It has application programming interfaces (APIs) for working with text, dates, regular
expressions, standard data structures, and the Document Object Model (DOM).</p>
11  </body>
12  </html>
```

style.css refer **(html-16)**

```
a:only-of-type{
  background-color: orange;
}
```

OUTPUT:

127.0.0.1:5500/index.html

Python is an interpreted, high-level and general-purpose programming language. Learn more at **Official Python Site** (only anchor in this paragraph).

Java is a class-based, object-oriented language. Links: Oracle Java and OpenJDK (two anchors, unmatched).

JavaScript is the programming language of the Web. Learn more at **MDN Web Docs** (only anchor in this paragraph).

Figure 75: Output of index.html (html-16) with `a:only-of-type` — three paragraphs (Python, Java, JavaScript); in the Python paragraph, which contains a single link ("interpreted,"), that link is highlighted.

➤ **First-child:** Selects first child element under the parent element when first child element is a specified element.

Example:

index.html (html-17)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <h3>Popular langauages</h3>
    <ul>
        <li >C</li>
        <li>Java  </li>
        <li>Python</li>
        <li>C++</li>
        <li>C#</li>
        <li>JavaScript</li>
    </ul>
    </div>
    <div>
        <h3>Popular Frameworks</h3>
    <ul>
        <li >Spring</li>
        `<li>DJango  </li>
        <li>Node.JS</li>
        <li>.NET Core</li>
        <li>Flask</li>
        <li>React.JS</li>
    </ul>
    <h3>Highest Paying Jobs</h3>
    <ul>
        <li >Devops Engineer</li>
        <li>Full Stack Developer</li>
        <li>Data Scientist</li>
        <li>AI-Ml Engineer</li>
        <li>Cloud Engineer</li>
        <li>Cyber Security Expert</li>
    </ul>
    </div>
</body>
</html>
```

Tree Representation:

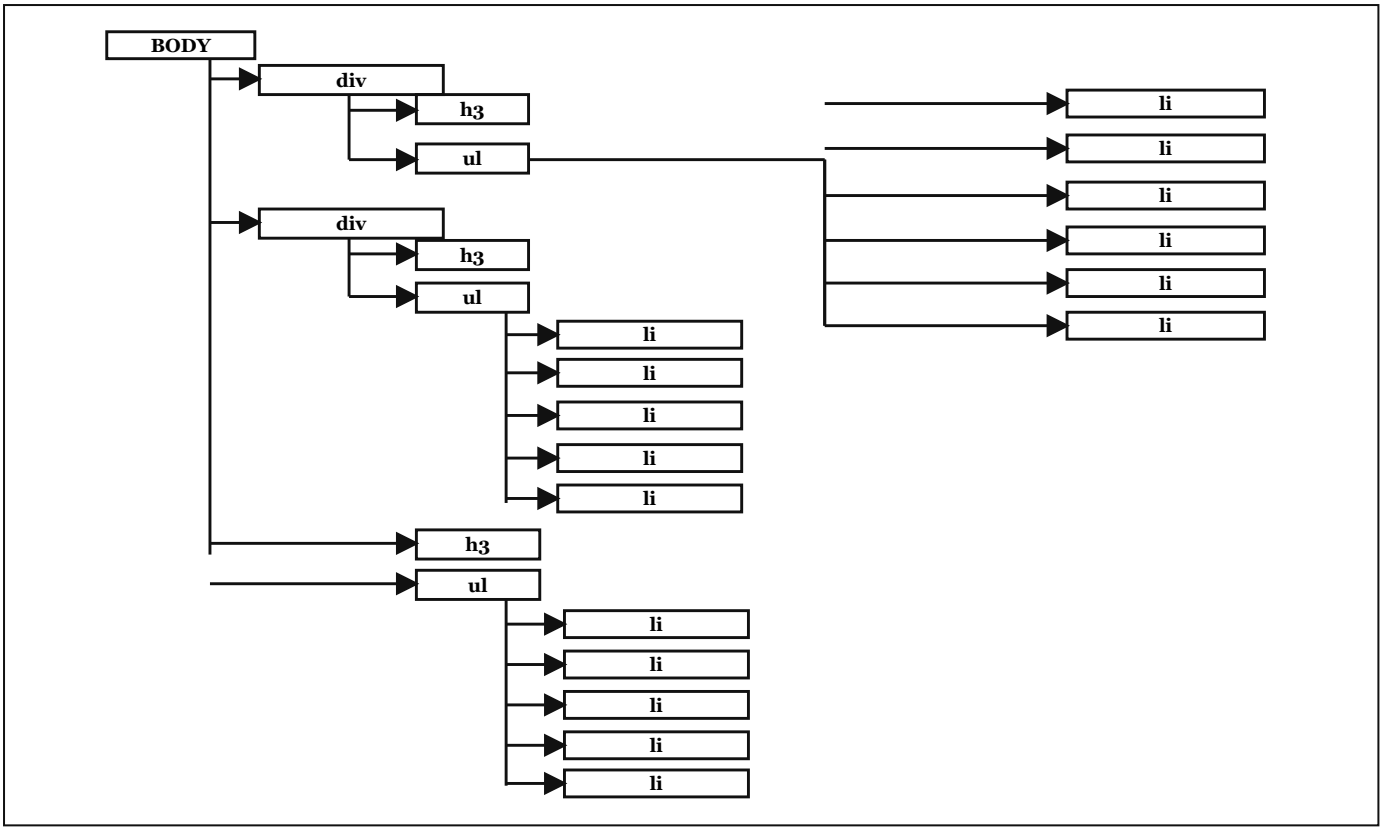


Figure 76: Tree representation of the body of index.html (html-17) — BODY → div (→ h3, ul → six li); div (→ h3, ul → five li); and a further h3 and ul (→ five li) drawn directly under BODY, as in the source drawing.

style.css refer **(html-17)**

```
li:first-child{
  background-color: orange;
}
```

OUTPUT:

127.0.0.1:5500/index.html

Popular Languages

- **C (first-child)**
- Java
- Python
- C++
- JavaScript

Popular Frameworks

- **Spring (first-child)**
- Django
- Node.JS
- Angular
- React.JS

Popular Profiles

- **Web Developer (first-child)**
- AI Engineer
- Data Scientist
- Cloud Architect
- Security Analyst

Figure 77: Output of index.html (html-17) with `li:first-child` — the first item of each of the three lists (C, Spring, Devops Engineer) is shaded.

➤ **Last-child:** Selects last child element under the parent element when last child element is a specified element.

Example:

style.css refer **(html-17)**

```
li:last-child{
  background-color:orangered;
}
```

OUTPUT:

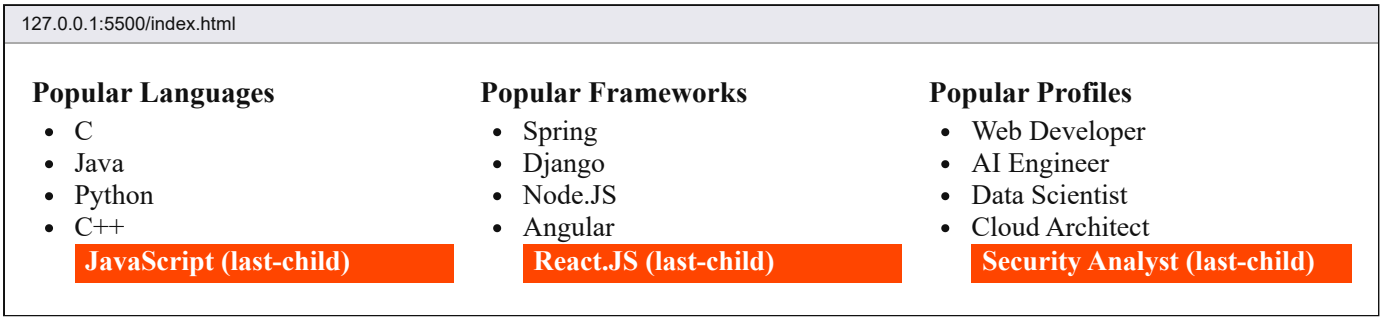


Figure 78: Output of index.html (html-17) with `li:last-child` — the last item of each list (JavaScript, React.JS, Cyber Security Expert) is shaded.

➤ **nth child:** Using nth child you can select elements that are the first, last, or only children of other elements. Let's understand how this nth- child works with an example.

Example:

style.css refer **(html-17)**

```
li:nth-child(3){
  background-color: aqua;
}
```

OUTPUT:

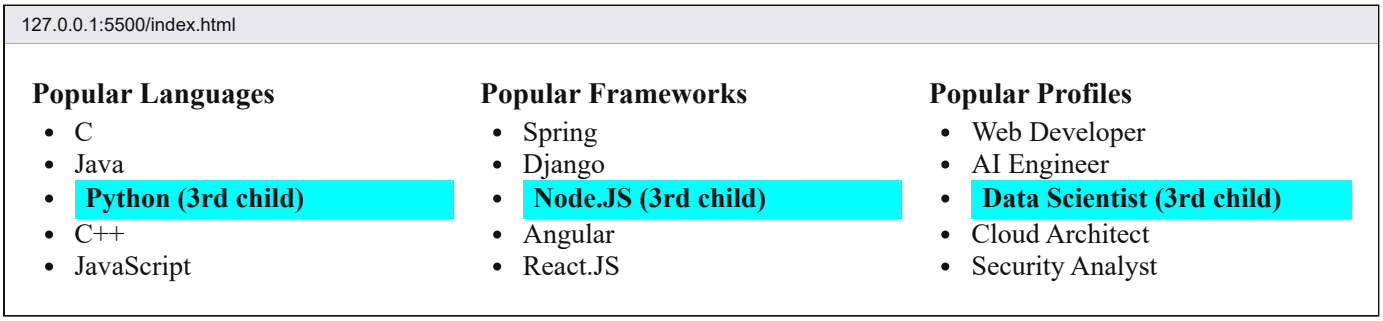


Figure 79: Output of index.html (html-17) with `li:nth-child(3)` — the third item of each list (Python, Node.JS, Data Scientist) is shaded.

Combining first-child and last-child pseudo class:

Example:

index.html (html-18)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <h3>Popular langauages</h3>
    <ul>
        <li >C</li>
        <li>Java  </li>
        <li>Python</li>
        <li>C++</li>
        <li>C#</li>
        <li>JavaScript</li>
    </ul>
    </div>
    <div>
        <h3>Popular Frameworks</h3>
    <ul>
        <li >Spring</li>
        <li>DJango  </li>
        <li>Node.JS</li>
        <li>.NET Core</li>
        <li>Flask</li>
        <li>React.JS</li>
    </ul>
    <h3>Highest Paying Jobs</h3>
    <ul>
        <li >Devops Engineer</li>
        <li>Full Stack Developer</li>
        <li>Data Scientist</li>
        <li>AI-Ml Engineer</li>
        <li>Cloud Engineer</li>
        <li>Cyber Security Expert</li>
    </ul>
    </div>
    <h3>Best Ed-tech companies</h3>
    <ul>
        <li>TechBridge</li>
    </ul>
</body>
</html>
```

style.css refer **(html-18)**

```css
li:first-child:last-child{
  background-color: aquamarine;
}
```

OUTPUT:

127.0.0.1:5500/index.html

Best Ed-tech Companies

- **TechBridge (Only child in list!)**

Popular Languages

- C
- Java
- Python
- C++
- JavaScript

Popular Frameworks

- Spring
- Django
- Node.JS
- Angular
- React.JS

Figure 80: Output of index.html (html-18) with `li:first-child:last-child` — only the single item "TechBridge" under "Best Edtech companies" is shaded.

In this example, only-child is selected using first-child and last-child pseudo class because only child is nothing but first and last child.

Example: Style the odd and even elements in the list using nth child pseudo class

style.css refer **(html-18)**

```
li:nth-child(2n){
  background-color: orange;
}
li:nth-child(2n+1){
  background-color: orangered;
}
```

OUTPUT:

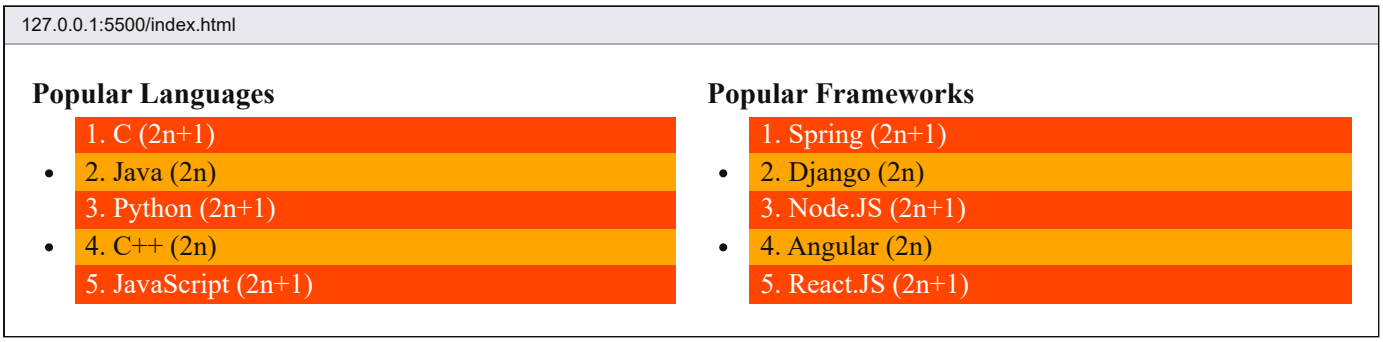


Figure 81: Output of index.html (html-18) with `li:nth-child(2n)` (orange) and `li:nth-child(2n+1)` (orangered) — every list item is shaded, even and odd items in alternating shades (the two colours look like two greys in the black-and-white photograph).

Here, nth-child(2n) will select all list items with even index numbers(2, 4, 6..etc) and nth-child(2n + 1) will select all list items with odd index numbers(1, 3, 5..etc). Instead of saying 2n and 2n + 1 in general you can say as even and odd as shown in below example.

style.css refer **(html-18)**

```
li:nth-child(odd){
  background-color: orange;
}
li:nth-child(even){
  background-color: orangered;
}
```

OUTPUT:

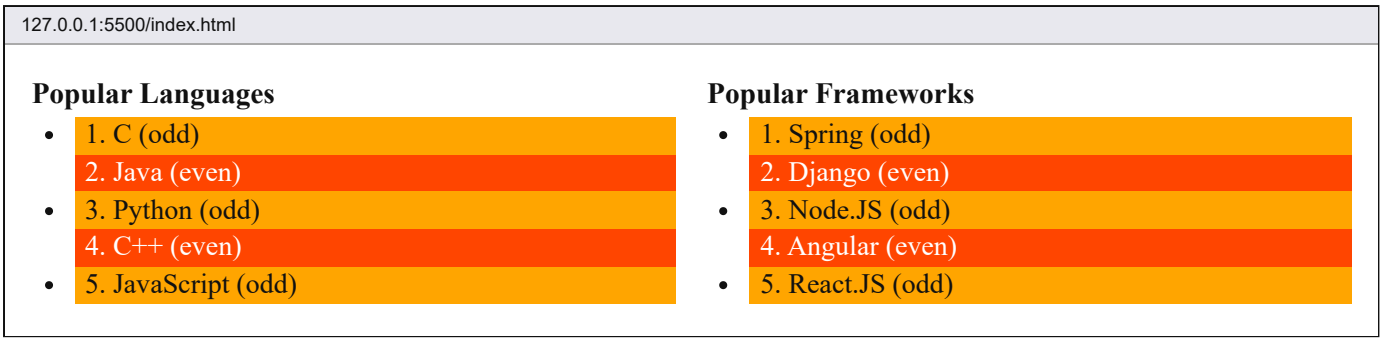


Figure 82: Output of index.html (html-18) with `li:nth-child(odd)` (orange) and `li:nth-child(even)` (orangered) — the same alternating shading as the 2n / 2n+1 version.

Example: Select range of values using nth-child

style.css refer **(html-18)**

```
li:nth-child(n+3){
  background-color: orangered;
}
```

OUTPUT:

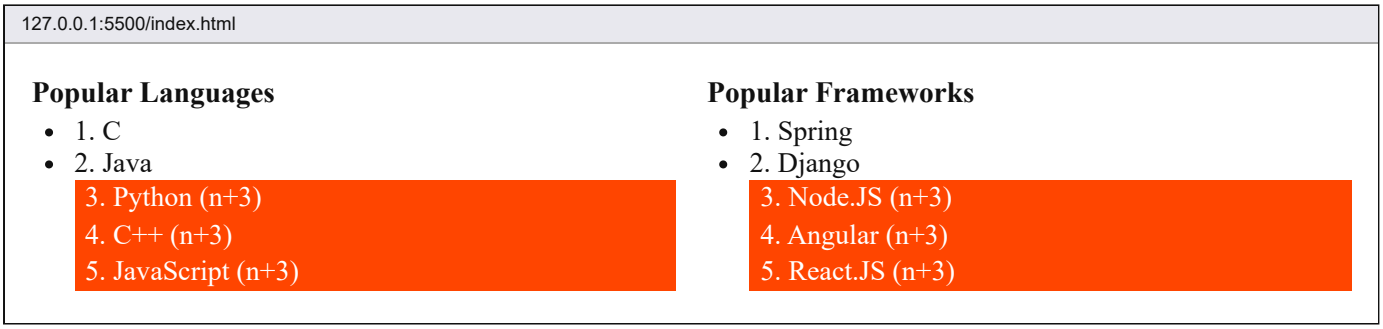


Figure 83: Output of index.html (html-18) with `li:nth-child(n+3)` — from the third item onward every item is shaded; the first two items of each list (and the single "TechBridge" item) are not.

In this example, (n + 3) selects every row starting from 3. If you want to select every row starting from 5 means you can write as (n + 5).

Example: Selecting the values from 3rd index to 5th index.

style.css refer **(html-18)**

```
li:nth-child(n+3):nth-child(-n+5){
  background-color: orange;
}
```

OUTPUT:

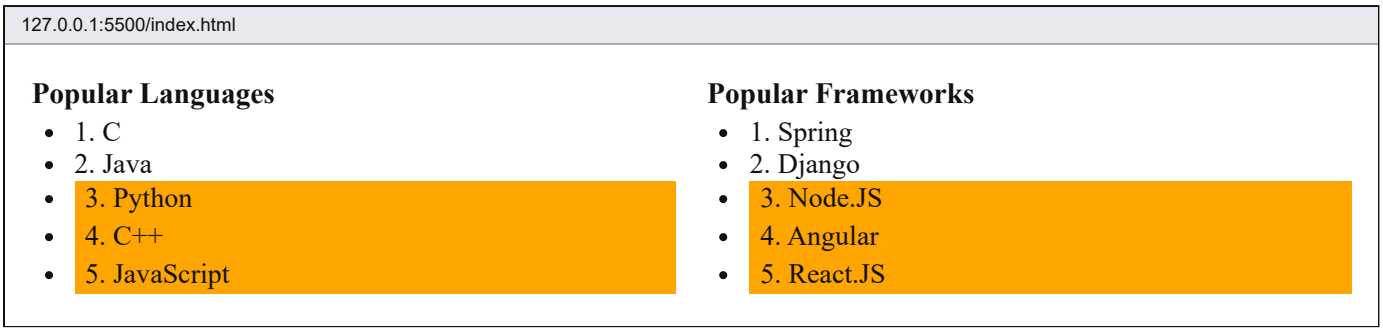


Figure 84: Output of index.html (html-18) with `li:nth-child(n+3):nth-child(-n+5)` — only items 3 to 5 of each list are shaded.

In this example, we have applied 2 nth-child selectors to one-group of siblings.

index.html (html-19)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <h3>Popular langauages</h3>
    <ul>
        <li >C</li>
        <li>Java  </li>
        <li>Python</li>
        <li>C++</li>
        <li>C#</li>
        <li>JavaScript</li>
    </ul>
    </div>
    <div>
        <h3>Popular Frameworks</h3>
    <ul>
        <li >Spring</li>
        <li>DJango  </li>
        <li>Node.JS</li>
        <li>.NET Core</li>
        <li>Flask</li>
        <li>React.JS</li>
    </ul>
    <h3>Highest Paying Jobs</h3>
    <ul>
        <li >Devops Engineer</li>
        <li>Full Stack Developer</li>
        <li>Data Scientist</li>
        <li>AI-Ml Engineer</li>
        <li>Cloud Engineer</li>
        <li>Cyber Security Expert</li>
    </ul>
    </div>
</body>
</html>
```

style.css refer **(html-19)**

```
ul:only-of-type >li:first-child,
ul:only-of-type >li:last-child {
  background-color: orange;
}
```

Tree Representation:

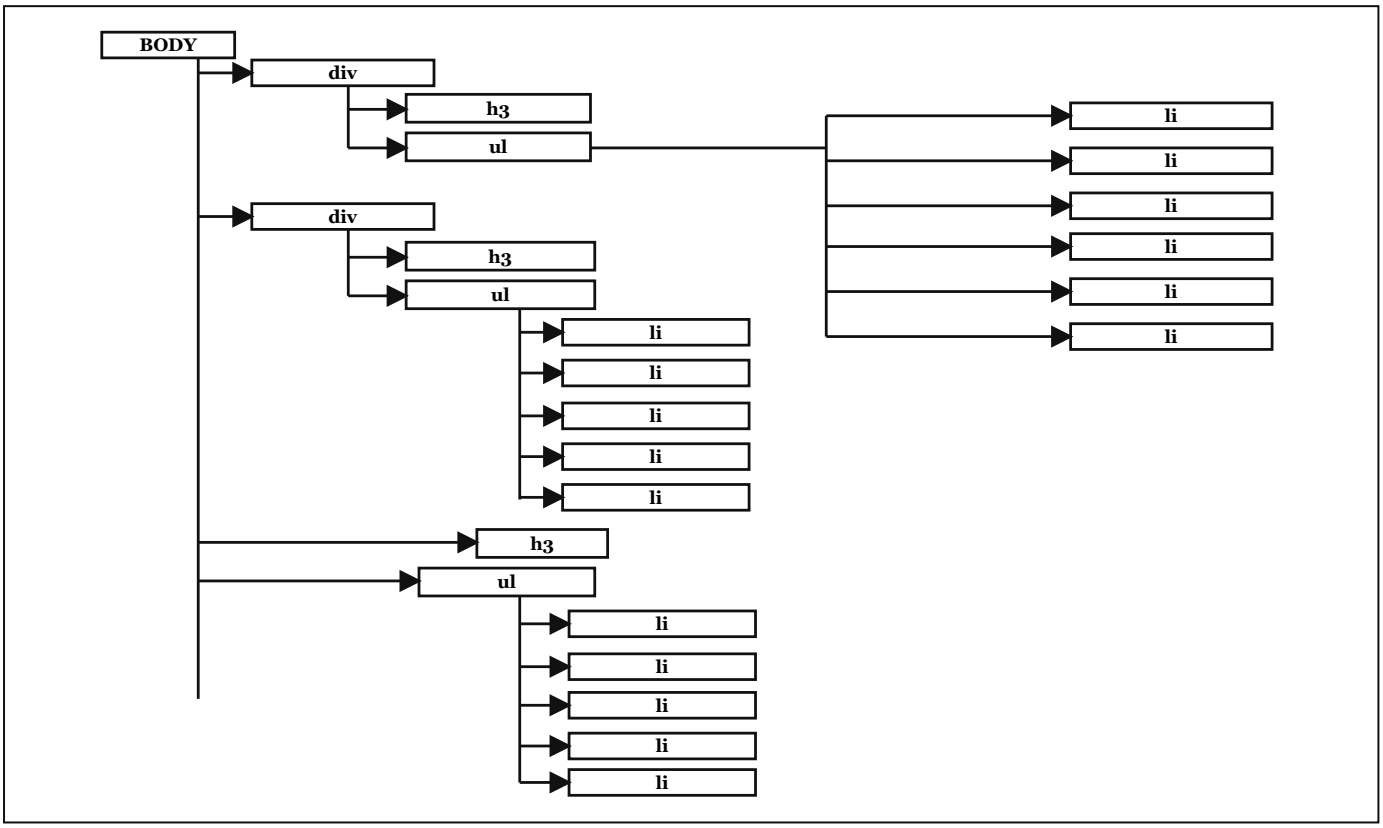


Figure 85: Tree representation of the body of index.html (html-19) — BODY → div (→ h3, ul → six li); div (→ h3, ul → five li); and an h3 and ul (→ five li) drawn directly under BODY, as in the source drawing.

OUTPUT:

127.0.0.1:5500/index.html

Container 1 (Only UL)

- Item A (ul:only-of-type)
- Item B
- Item C

Container 2 (Multiple lists: UL and OL)

- List item 1
- List item 2

1. Ordered item 1
2. Ordered item 2

Figure 86: Output of index.html (html-19) with the `ul:only-of-type` rule — only the first and last items (C and JavaScript) of the first list are shaded.

In this example, styling is applied to first and last li present inside first ul. Only-of-type is used to select first ul, first-child and last-child is used to select first and last li inside ul.

first-of-type: first-of-type pseudo-class selects an element that is the first element of its type in the list of children of its parent. Let's understand first-of-type pseudo class with an example.

Example:

style.css refer **(html-19)**

```
div>ul:first-of-type{
  background-color: violet;
}
```

OUTPUT:

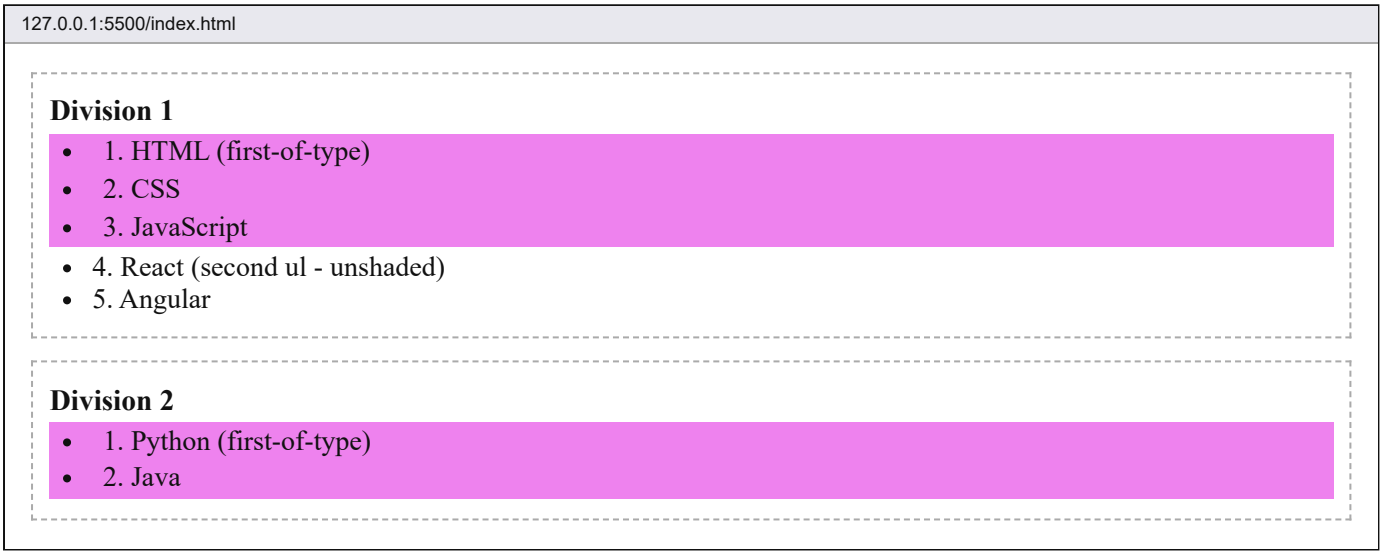


Figure 87: Output of index.html (html-19) with `div>ul:first-of-type` — the first ul inside each div (under "Popular langauages" and "Popular Frameworks") is shaded violet (grey in the black-and-white photograph); the second ul in the second div ("Highest Paying Jobs") is not.

Example: Styling of list inside ul using firs-of-type, last-of-type and nth-of-type

index.html (html-20)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <ul>
            <li>ul-1-A</li>
            <li>ul-1-B</li>
            <li>ul-1-C</li>
            <li>ul-1-D</li>
        </ul>
        <ol>
            <li>ol-1-A</li>
            <li>ol-1-B</li>
            <li>ol-1-C</li>
            <li>ol-1-D</li>
        </ol>
        <dl>
            <li>dl-1-A</li>
            <li>dl-1-B</li>
            <li>dl-1-C</li>
            <li>dl-1-D</li>
        </dl>
        <ul>
            <li>ul-2-A</li>
            <li>ul-2-B</li>
            <li>ul-2-C</li>
            <li>ul-2-D</li>
        </ul>
        <ol>
            <li>ol-2-A</li>
            <li>ol-2-B</li>
            <li>ol-2-C</li>
            <li>ol-2-D</li>
        </ol>
        <dl>
            <li>dl-2-A</li>
            <li>dl-2-B</li>
            <li>dl-2-C</li>
            <li>dl-2-D</li>
        </dl>
```

index.html (html-20) — continued

```
45          <ul>
46              <li>ul-3-A</li>
47              <li>ul-3-B</li>
48              <li>ul-3-C</li>
49              <li>ul-3-D</li>
50          </ul>
51          <ol>
52              <li>ol-3-A</li>
53              <li>ol-3-B</li>
54              <li>ol-3-C</li>
55              <li>ol-3-D</li>
56          </ol>
57          <dl>
58              <li>dl-3-A</li>
59              <li>dl-3-B</li>
60              <li>dl-3-C</li>
61              <li>dl-3-D</li>
62          </dl>
63      </div>
64      <div>
65          <ul>
66              <li>ul-1-One</li>
67              <li>ul-1-Two</li>
68              <li>ul-1-Three</li>
69              <li>ul-1-Four</li>
70          </ul>
71          <ol>
72              <li>ol-1-One</li>
73              <li>ol-1-Two</li>
74              <li>ol-1-Three</li>
75              <li>ol-1-Four</li>
76          </ol>
77          <dl>
78              <li>dl-1-One</li>
79              <li>dl-1-Two</li>
80              <li>dl-1-Three</li>
81              <li>dl-1-Four</li>
82          </dl>
```

index.html (html-20) — continued

```
 83          <ul>
 84              <li>ul-2-One</li>
 85              <li>ul-2-Two</li>
 86              <li>ul-2-Three</li>
 87              <li>ul-2-Four</li>
 88          </ul>
 89          <ol>
 90              <li>ol-2-Two</li>
 91              <li>ol-2-Three</li>
 92              <li>ol-2-One</li>
 93              <li>ol-2-Four</li>
 94          </ol>
 95          <dl>
 96              <li>dl-2-One</li>
 97              <li>dl-2-Two</li>
 98              <li>dl-2-Three</li>
 99              <li>dl-2-Four</li>
100          </dl>
101      </div>
102  </body>
103  </html>
```

style.css refer **(html-20)**

```
div > ul:first-of-type{
  background-color: blueviolet;
}
div > ol:last-of-type{
  background-color: orangered;
}
div > dl:nth-of-type(2){
  background-color: grey;
}
```

OUTPUT:

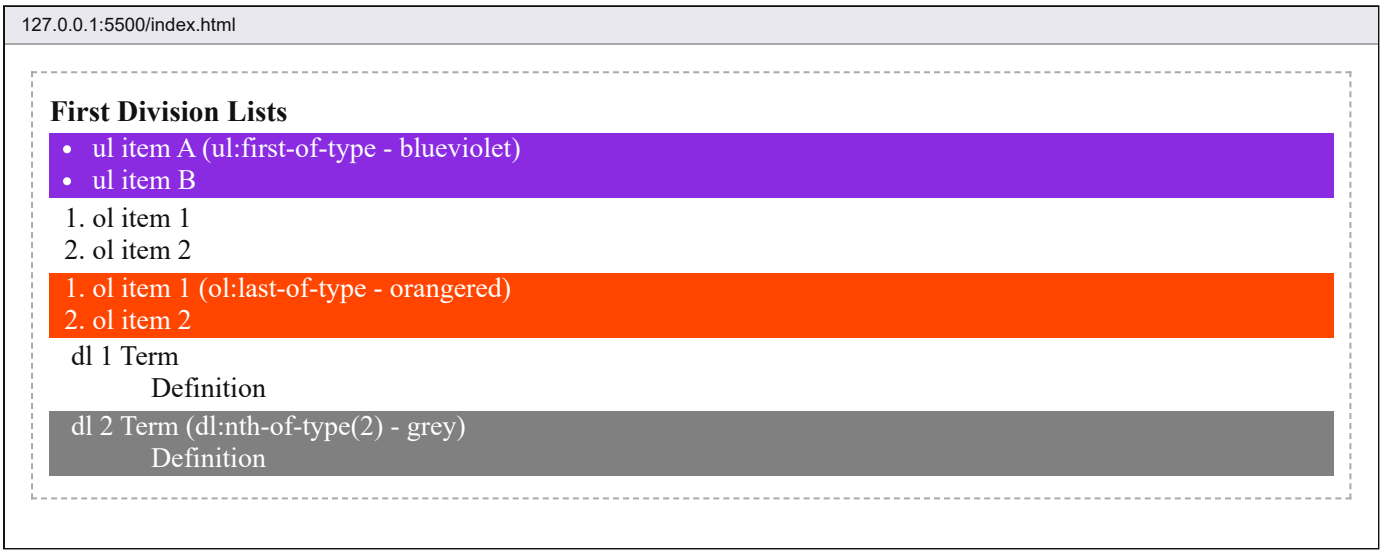


Figure 88: Output of index.html (html-20) with `div > ul:first-of-type` (blueviolet), `div > ol:last-of-type` (orangered) and `div > dl:nth-of-type(2)` (grey).

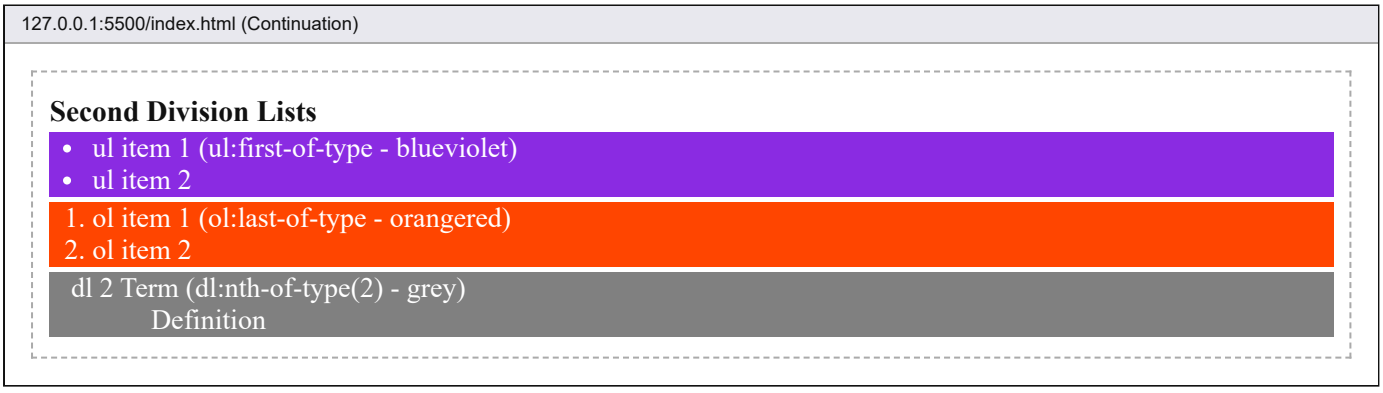


Figure 89: Continuation of the output of index.html (html-20) from the previous page (the lower part of the page, with the second div's lists).

Example: Styling of even and odd ul inside the div. (refer index.html of above example)

style.css refer**(html-20)**

```
div > ul:nth-of-type(2n){
  background-color: grey;
}
div > ul:nth-of-type(2n+1){
  background-color: grey;
}
```

OR

```
div > ul:nth-of-type(even){
  background-color: grey;
}
div > ul:nth-of-type(odd){
  background-color: grey;
}
```

OUTPUT:

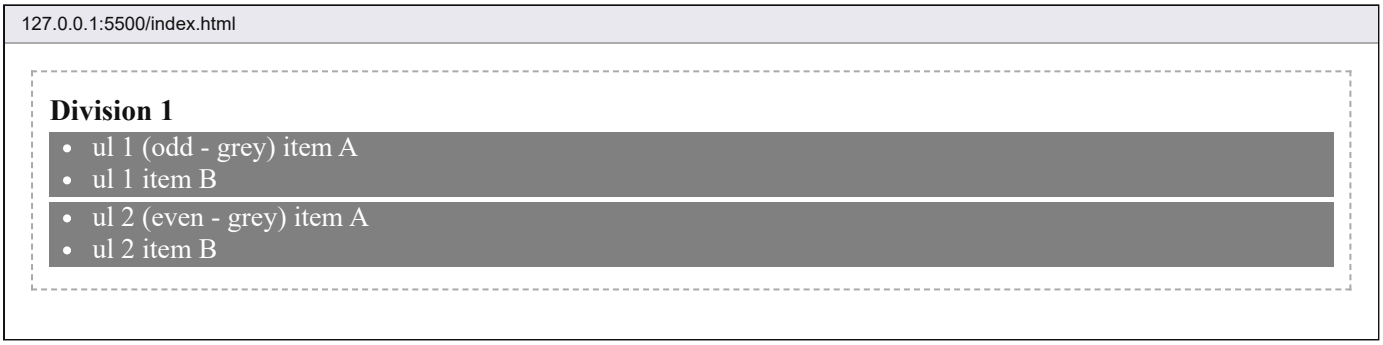


Figure 90: Output of index.html (html-20) with `div > ul:nth-of-type(2n)` and `(2n+1)` (or `even` / `odd`) — the ul blocks are shaded grey.

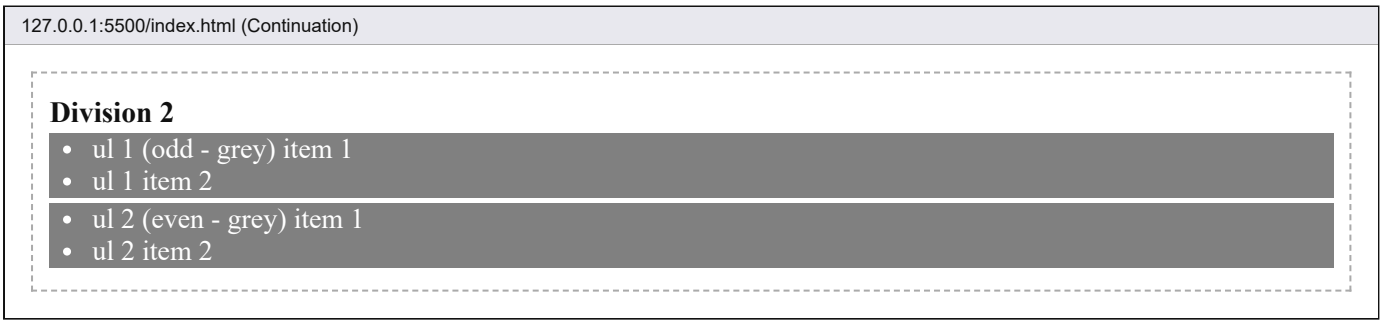


Figure 91: Continuation of the output of index.html (html-20) from the previous page.

Dynamic Pseudo class:

There are four types of Dynamic pseudo classes they are:

- **Link:** Refers to any anchor that is a hyperlink (i.e., has an href attribute) and points to an address that has not been visited.
- **Visited:** Refers to any anchor that is a hyperlink to an already visited address
- **Hover:** Refers to any element over which the mouse pointer is placed—e.g., a hyperlink over which the mouse pointer is hovering.
- **Active:** Refers to any element that has been activated by user input—e.g., a hyperlink on which a user clicks during the time the mouse button is held down.

Example:

index.html (html-21)

```
<!DOCTYPE html>
<html >
<head>
    <title>Dynamic pseudo classes</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2>Good NGO's</h2>
    <a href="https://bhartifoundation.org">Bharti Foundation</a> <br>
    <a href="https://cuddlesfoundation.org">Cuddles Foundation</a> <br>
    <a href="https://goonjfoundation.org">Goonj Foundation</a> <br>
</body>
</html>
```

style.css refer **(html-21)**

```
a:link{
  background-color: orange;
}
```

OUTPUT:

Good NGO's

Bharti Foundation
Cuddles Foundation
Goonj Foundation

Figure 92: Output of index.html (html-21) with `a:link` — the three unvisited links (Bharti Foundation, Cuddles Foundation, Goonj Foundation) have an orange background (shown dark in the black-and-white photograph).

In this example, styling is applied to link pseudo class that has not been visited.

Example:

style.css refer **(html-21)**

```
a:visited{
  background-color: violet;
}
```

OUTPUT:

Good NGO's

Bharti Foundation
Cuddles Foundation
Goonj Foundation

Figure 93: Output of index.html (html-21) with `a:visited` — the three links are shown without a highlight because none has yet been visited.

In this example, styling to visited pseudo-class applies once the link has been visited by the user.

Example:

style.css refer **(html-21)**

```
a:hover{
  background-color: red;
}
```

OUTPUT:



Figure 94: Output of index.html (html-21) with `a:hover` — the mouse pointer is over "Bharti Foundation", so only that link has the red background (shown dark in the black-and-white photograph).

Here, if you observe the output styling is applied to BhartiFoundation because mouse cursor is hovered on that hyperlink.

Example:

style.css refer **(html-21)**

```
a:active{
  background-color: pink;
}
```

Output



Figure 95: Output of index.html (html-21) with `a:active` — the Good NGO's page with its three links (the pink background is applied only while a link is held down).

In this example, if you observe the output BhartiFoundation is active so styling is applied to that hyperlink only.

Target pseudo class:

When a URL includes a fragment identifier, the piece of the document at which it points is called (in CSS) the target. Thus, you can uniquely style any element that is the target of a URL fragment identifier with the :target pseudo-class. Let's understand target pseudo class with an example.

Example:

index.html (html-22)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
</head>
<body>
    <a href="#header1">Header1</a>
    <a href="#header2">Header2</a>
    <a href="#header3">Header3</a><br>
    <a href="#para1">Para1</a>
    <a href="#para2">Para2</a>
    <a href="#para3">Para3</a>
    <h1 id="header1">Heading 1</h1>
    <p id="para1">
        Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.
        It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).
    </p>
    <h1 id="header2">Heading 2</h1>
    <p id="para2">
        Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.
        It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).
    </p>
    <h1 id="header3">Heading 3</h1>
    <p id="para3">
        Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.
        It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).
    </p>
</body>
</html>
```

style.css refer **(html-22)**

```
h1:target{
  background-color: violet;
}
p:target{
  background-color: orange;
}
```

OUTPUT:

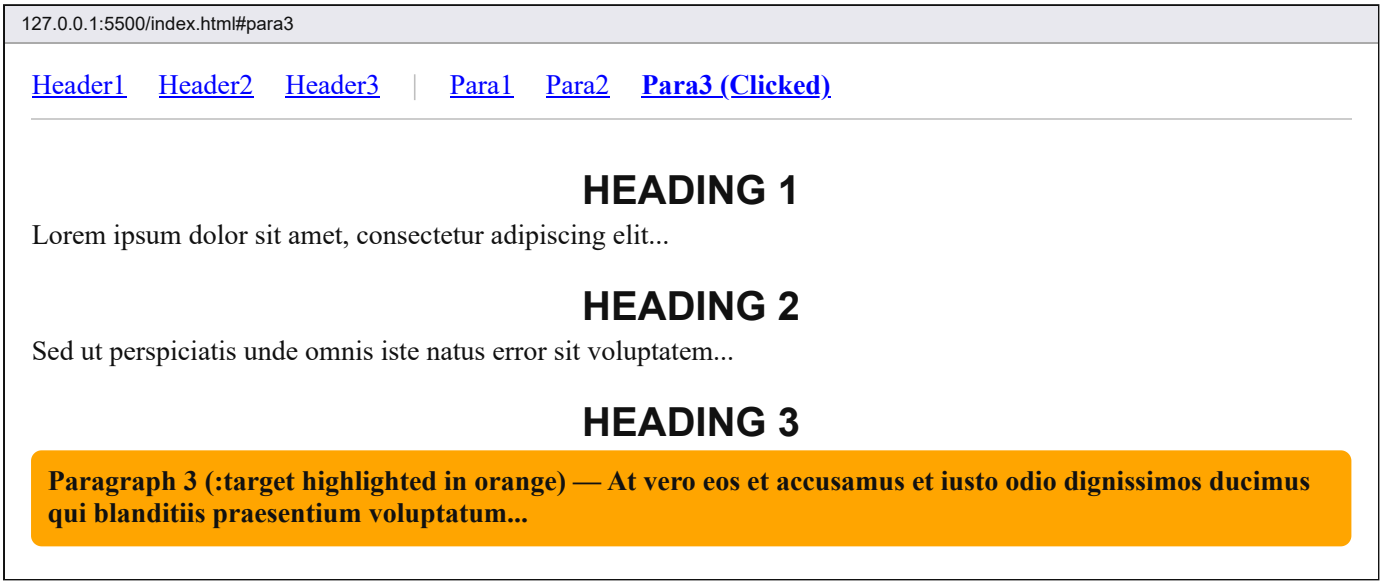


Figure 96: Output of index.html (html-22) with `h1:target` (violet) and `p:target` (orange) — the page after clicking the "Para3" link: the third paragraph (id="para3") is highlighted.

Here we have clicked on para3 hyperlink so styling is applied to that target hyperlink only.

Pseudo elements

Pseudo-element in CSS is used to add style in specified parts of an element. Example: Using style before or after an element. Under pseudo elements we will come across first-letter, firstline, before and after. Let's now understand this with an example.

Example: Styling is applied to first line in the paragraph using first pseudo elements.

index.html (html-23)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>HTML</h1>
    <p>
        HyperText Markup Language (HTML) is the standard markup language for documents
        designed to be displayed in a web browser. <br> It defines the content
        and structure of web content. <br>  It is often assisted by technologies such
        as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.
    </p>
</body>
</html>
```

style.css refer **(html-23)**

```
/* Style only the first line of the para */
p::first-line{
  color: orange;
  font-family: monospace;
  font-weight: bold ;
  font-size: 20px;
}
```

OUTPUT:

HTML

HyperText Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser.
It defines the content and structure of web content.
It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

*Figure 97: Output of index.html (html-23) with `p::first-line` — only the first line of the paragraph is orange, monospace and bold; the lines after the
 breaks keep the normal style.*

Example: Styling is applied to first letter in the paragraph using first-letter pseudo elements.

style.css refer **(html-23)**

```
/* Style only the first word of the para */
p::first-letter{
   color: purple;
   font-size: 22px;
   font-family: cursive;
   font-weight: bold;
}
```

OUTPUT:

HTML

HyperText Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser.
It defines the content and structure of web content.
It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Figure 98: Output of index.html (html-23) with `p::first-letter` — only the first letter "H" of the paragraph is larger, bold, cursive and purple (grey in the black-and-white photograph).

Example: use of before and after pseudo elements.

style.css refer **(html-23)**

```
/* Add [ before and ] after  HTML */
h1::before{
   content: "[";
   color: orange;
}
h1::after{
   content: "]";
   color: orange;
}
```

OUTPUT:

[HTML]

HyperText Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser.
It defines the content and structure of web content.
It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Figure 99: Output of index.html (html-23) with `h1::before` and `h1::after` — the heading is shown as "[HTML]" with the added brackets in orange (grey in the black-and-white photograph).

Example: Adding image using before and after pseudo elements

style.css refer **(html-23)**

```
/* Add image before and after HTML */
h1::before{
  content: url(HTML5.png) ;
  color: orange;
}
h1::after{
  content: url(HTML5.png);
  color: orange;
}
```

OUTPUT:

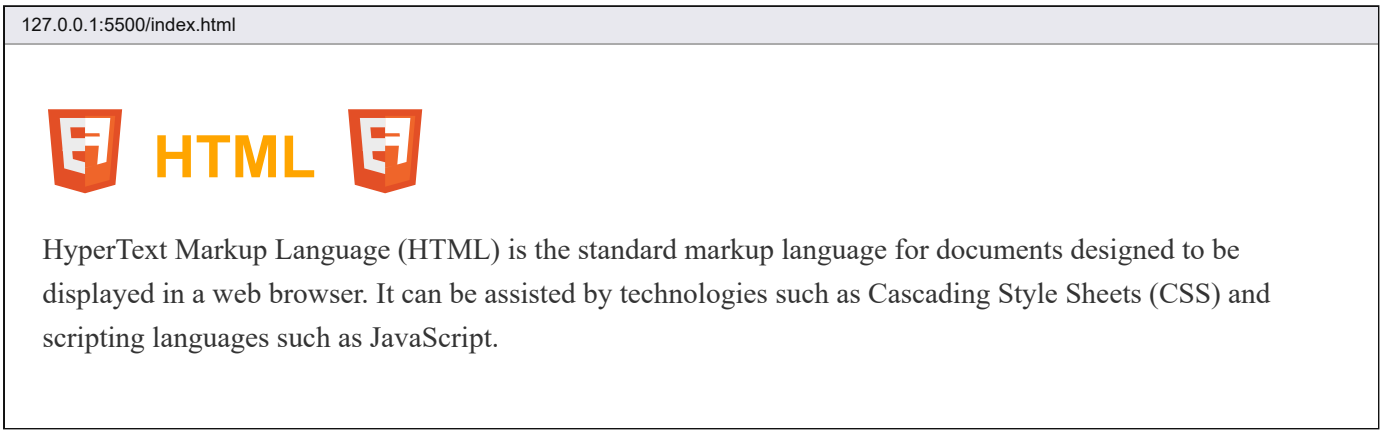


Figure 100: Output of index.html (html-23) with `content: url(HTML5.png)` in `h1::before` and `h1::after` — the HTML5 logo appears on both sides of the heading "HTML".

UI State Pseudo class:

There are different types of UI state pseudo classes available, let's understand with an example.

index.html (html-24)

```
<!DOCTYPE html>
<html >
<head>
    <title>UI based Dynamic pseudo classes</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2>Sign Up Form</h2>
    <form action="response.html">
        <table>
            <tr>
                <td><label>Name</label></td>
                <td><input type="text" name="name" size="15" required>
                    <p>Enter Your Name</p> </td><br>
            </tr>
            <tr>
                <td><label>College</label></td>
                <td>
                    <input type="text" name="college" size="15" disabled> <br>
            </tr>
            <tr>
                <td><label>Primary Email</label></td>
                <td>
                    <input type="email" name="pemail" size="15" required>
                    <p>Enter Your Email</p>
                    <img class="valid" src="right.png" >
                    <img class="invalid" src="warning.webp" ><br>
                </td>
            </tr>
            <tr>
                <td><label>Secondary Email</label></td>
                <td>
                    <input type="email" name="semail" size="15">
                    <p>Enter Your Optional Email</p>  <br>
                </td>
            </tr>
            <tr>
                <td><label>Password</label></td>
                <td>
                    <input type="password" name="pwd" size="15" required>
                    <p>Enter Your Password</p> <br>
                </td>
            </tr>
            <tr>
                <td><label>Gender</label></td>
                <td>
                    <input type="radio" name="gender" size="15" value="M" checked>
                    <label>Male</label>
                    <input type="radio" name="gender" size="15" value="F">
                    <label>Female</label>
                    <input type="radio" name="gender" size="15" value="O">
                    <label>Other</label> <br>
                </td>
            </tr>
```

index.html (html-24) — continued

```
            <tr>
                <td><label>Age</label></td>
                <td>
                    <input type="number" name="age" min="18" max="100" >
                    <p>Enter Your Age</p>
                    <img class="valid" src="right.png" >
                    <img class="invalid" src="warning.webp" ><br>
                </td>
            </tr>
            <tr>
                <td><label>MOD</label></td>
                <td>
                    <input type="text" name="mod" value="online" size="15" readonly>
                    <p>Only Online Mode of Delivery is Available</p>  <br>
                </td>
            </tr>
            <tr>
                <td><input type="submit"></td></tr>
        </table>
    </form>
</body>
</html>
```

style.css refer **(html-24)**

```
/* To change the colour and font of the entire page */
body{
    background-color: yellow;
    font-family: cursive;
}
/* To add a border aroud the form */
form{
    border: 3px solid black;
}
/* To style only such elements which are compulsory to be filled in order submit the form */
input:required {
    border: 2px groove violet;
}
/* To style only the such elements which are not compulsory to filled in order to submit the form */
input:not(:required){
    border: 2px dashed violet;
}
input:optional{
    border: 2px dashed violet;
}
/* To style only such elements which are disabled */
input:disabled{
    border: 2px dotted violet;
}
/* To style only  */
input:read-only{
    background-color: gray;
}
/* This can help us to overcome the problem */
input[readonly]{
    background-color: gray;
}
```

style.css (html-24) — continued

```
/* To display the paragraph when hovered */
input:hover + p{
    display: inline;
    font-family: monospace;
    background-color: red;
}
/* This will change the background colour of the input container */
input:focus{
    background-color: silver;
    font-weight: bold;
}
/* This will hide the img */
img{
    display: none;
}
/* To display the img when its valid */
input[type="email"]:focus:valid ~ img.valid{
    display: inline;
    width: 18px;
    height: 18px;
}
/* To display the img when its invalid */
input[type="email"]:focus:invalid ~ img.invalid{
    display: inline;
    width: 18px;
    height: 18px;
}
/* This will increase the size of the radio when its checked */
input[type="radio"]:checked{
    width: 20px;
    height: 20px;
}

/* This will change the colour of the label when its checked */
input[type="radio"]:checked + label{
    color: red;
}
/* This will add the border to the submit button */
input[type="submit"]{
    border: 2px solid violet;
}
/* To display the img when its valid */
input[name="age"]:focus:in-range:valid ~ img.valid{
    display: inline;
    width: 18px;
    height: 18px;
}

/* To display the img when its invalid */
input[name="age"]:focus:out-of-range:invalid ~ img.invalid{
    display: inline;
    width: 18px;
    height: 18px;
}
```

OUTPUT:

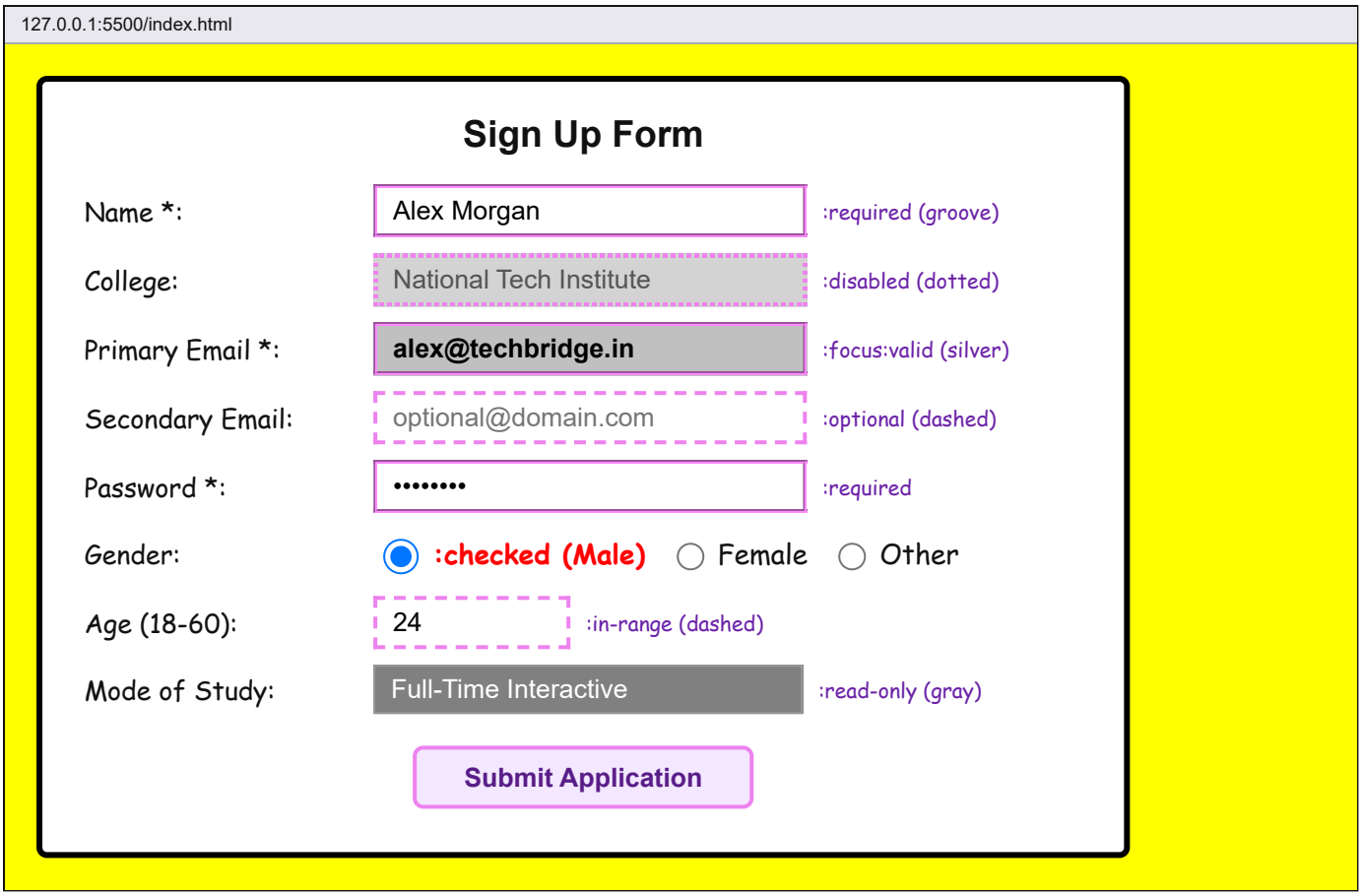


Figure 101: Output of index.html (html-24) with its style sheet — the "Sign Up Form" with the fields Name, College, Primary Email, Secondary Email, Password, Gender (radio buttons Male / Female / Other), Age, MOD and a Submit button; the required fields have a groove border, the optional ones a dashed border, the disabled College field a dotted border, and the read-only MOD field a grey background.

Let's explore all UI state pseudo class one by one

:required – styling is applied to a user input that must have a value set.
:optional – Styling is applied to a user input that does not need to have a value set
:read-only - Refers to a user input that is not editable by the user
:read-write - Refers to a user input that is editable by the user
:disabled - Refers to user-interface elements (such as form elements) that are disabled; that is, not available for input
:enabled - Refers to user-interface elements (such as form elements) that are enabled; that is, available for input.
:checked - Refers to radio buttons or checkboxes that have been selected, either by the user or by defaults within the document itself.
:default - Refers to the radio button, checkbox, or option that was selected by default.
:valid - Refers to a user input that meets all of its data validity semantics
:invalid - Refers to a user input that does not meet all of its data validity semantics
:in-range - Refers to a user input whose value is between the minimum and maximum values
:out-of-range - Refers to a user input whose value is below the minimum or above the maximum values allowed by the control
display:none - Hiding an element can be done by setting the display property to none. The element will be hidden, and the page will be displayed as if the element is not there.

Specificity

What is specificity?

If two or more CSS rules apply to the same element, the rule with the highest specificity will take priority, and its styles will be used.

Specificity measures how specific a CSS rule is.

Lets look into some examples to understand how specificity works.

index.html (html-25)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <p class="para1">
        Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the
industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled
it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic
typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets
containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker
including versions of Lorem Ipsum.
    </p>
</body>
</html>
```

style.css

```
/* Style using element selector */
p{
  background-color: orange;
  font-family: cursive;
}
/* Style using class selector */
p.para1{
  background-color: grey;
  font-family: monospace;
}
/* Style using child selector*/
body p{
  background-color: cyan;
  font-family: sans-serif;
}
```

OUTPUT:

```
Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's
standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make
a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting,
remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing
Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions
of Lorem Ipsum.
```

Figure 102: Output of index.html (html-25) — the paragraph is shown with a grey background and monospace font (the `p.para1` rule wins over `p` and `body p`).

In the example above, the background color is grey and the font family is monospace. This happens because the selector for those styles has a high specificity.

Specificity Hierarchy

Every CSS selector has a specific place in the specificity hierarchy. There are four categories that define the specificity level of a selector:

1. Inline styles – Example : <h1 style= "color:pink;">
2. ID's – Example:#navbar
3. Classes, pseudo-classes, attribute selectors:- Example: .test, :hover, [href].
4. Elements and pseudo-elements- Example: h1, ::before.

From the hierarchy, we can understand that inline CSS is the most specific. Next is the ID selector, followed by classes, pseudo-classes, and attribute selectors. The least specific are element selectors and pseudo-elements.

How to calculate Specificity?

When the browser calculates specificity, it gives each category a "score" or "weight." The scores are added up to determine which rule is most specific:

- Inline styles: will always have the higher specificity.
- IDs: 1,0,0 (hundred points)
- Classes, attributes, pseudo-classes: 0,1,0 (ten points)
- Elements, pseudo-elements: 0,0,1 (one point)

Let's understand this by the above html-25

- `p` :- one element selector (0,0,1)
- `p.para1` :- one element selector and class selector (0,1,1)
- `body p` :- two element selectors (0,0,2)

Thus, the higher specificity value belongs to `p.para1`. Therefore, the CSS related to it is being applied.

Rules for calculating specificity:

- If the selector contains an ID, 1 is added to the 100's place.
- If the selector contains a class, pseudo-class, or attribute, 1 is added to the 10's place.
- If the selector contains an element or pseudo-element, 1 is added to 1's place.

Note:- Specificity is always compared from left to right.

style.css refer **(html-25)**

```
/* Style using id selector*/
p#p1{
  background-color: violet;
  font-family: fantasy;
}
```

OUTPUT:

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.

Figure 103: Output of index.html (html-25) with the `p#p1` rule — the paragraph now has a violet background (grey in the photograph) and the fantasy font family.

In the above output, we can see that the background colour has changed to violet and the font family has changed to fantasy.

This is because the specificity of the above selector is higher than all the other selectors.

index.html (html-26)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2>Best websites for front-end development</h2>
    <ul>
        <li>
            <a class="firsta" href="https://www.w3schools.com/">W3Schools</a>
        </li>
        <li>
            <a href="https://codepen.io/">Codepen</a>
        </li>
        <li>
            <a href="https://cssbattle.dev/">CSSBATTLE</a>
        </li>
    </ul>
</body>
</html>
```

style.css refer **html-26**

```
/* Style all the a tags  */ /* Here specificity is 013 */
ul li > a[href]{
  background-color: orange;
}
/* Here specificity is 022 so this will be selected */
li a[href ^= "https"]:first-child{
  background-color: violet;
}
/* Here specificity is 004 */
body ul > li > a {
  background-color: cyan;
}
```

OUTPUT:



Figure 104: Output of index.html (html-26) — the heading "Best websites for front-end development" with the three links; the first link (W3Schools) has the violet background (grey in the photograph).

In the above example, we can see that the CSS rule with higher specificity has changed the background colour to violet.

style.css refer **html-26**

```
/* specificity = 011 */
a.firsta{
  background-color: orange;
}
/* specificity = 011 */
a[href = "https://www.w3schools.com/"]{
  background-color: cyan;
}
```

OUTPUT:



Figure 105: Output of index.html (html-26) with the two rules of equal specificity — the W3Schools link is highlighted (cyan in the original; grey in the black-and-white photograph).

In the above output, we can see that the background colour has changed to cyan, even though both CSS rules have the same specificity.

When CSS rules have the same specificity, the one that comes last will be applied.

style.css refer **html-26**

```
a{
  background-color: blue !important;
}
```

OUTPUT:



Figure 106: Output of index.html (html-26) with `background-color: blue !important;` — all three links get the blue background (dark in the black-and-white photograph).

The above example explains how we can make a CSS rule compulsory regardless of its specificity.

style.css refer **html-26**

```
a{
  background-color: blue !important;
}
a{
  background-color: red !important;
}
```

OUTPUT:



Figure 107: Output of index.html (html-26) with two `!important` rules — all links are red (dark grey in the photograph).

In the above output, we can see that the background colour is red despite both rules being marked as important.

This is because they have the same specificity, so the rule that appears last is applied.

index.html (html-27)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h2>Best websites for front-end development</h2>
    <ul>
        <li>
            <a class="firsta" href="https://www.w3schools.com/" >W3Schools</a>
        </li>
        <li>
            <a style="background-color: violet"; href="https://codepen.io/">Codepen</a>
        </li>
        <li>
            <a href="https://cssbattle.dev/">CSSBATTLE</a>
        </li>
    </ul>
</body>
</html>
```

OUTPUT:



Figure 108: Output of index.html (html-27) — the Codepen link, which carries the inline `style` attribute (violet), is the one that is not overridden by the style sheet. (The style sheet for this example is not shown in the notes.)

In the above output CSS rule of inline CSS will be applied. As it has considered as highest specificity.

Inheritance

In programming, inheritance means that one class or object can inherit properties and methods from another class or object. This helps in reusing code, making it more organized and easier to maintain.

In CSS, inheritance refers to certain style properties being passed down from a parent element to its child elements. This means if you apply a style to a parent element, the child elements will automatically inherit that style unless they have their own specific styles defined.

Inheritance in CSS is a powerful concept that reduces the amount of code you need to write, making your styles more efficient and easier to maintain.

How does inheritance work in CSS?

In CSS, inheritance works by passing down specific style property values from a parent element to its child elements. However, not all CSS properties are inherited by default. Some properties, like colour and font-family, are automatically inherited, while others require explicit inheritance using the `inherit` keyword.

index.html (html-28)

```
 1  <html>
 2  <head>
 3      <link rel="stylesheet" href="style.css">
 4  </head>
 5  <body>
 6      <h1>TechBridge</h1>
 7      <p>TechBridge is a technical training institute that incorporates technology into its training programs.
TechBridge has a unique approach to training, utilizing augmented reality (AR) to enhance the learning
experience.
 8      </p>
 9  </body>
10  </html>
```

style.css refer **html-28**

```
body {
  font-family: Arial, sans-serif;
  color:green;
}
h1 {
  font-size: 24px;
}
p{
  font-size: 16px;
}
```

OUTPUT:

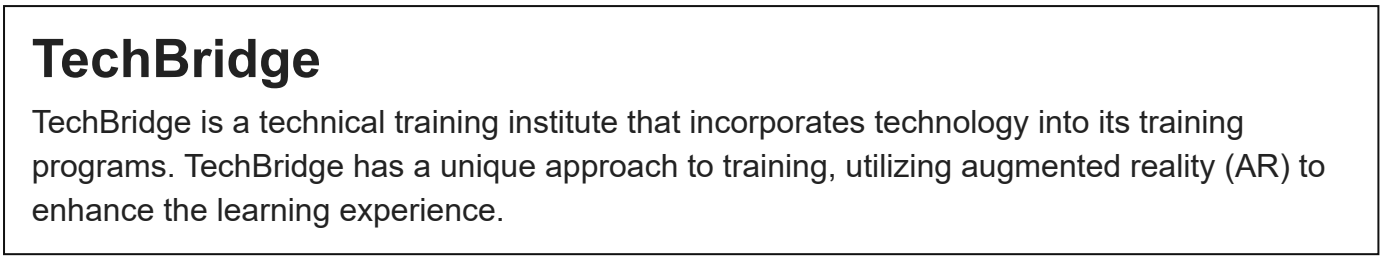


Figure 109: Output of index.html (html-28) — the heading "TechBridge" and the paragraph both appear in Arial, in green (dark in the black-and-white photograph).

In this example, we have defined a font family and colour for the body element. These properties will be inherited by all the child elements of the body, including the h1 and p tags.

Therefore, both the heading and the paragraph will have Arial as the font and a colour value of green.

Using the inherit keyword

Sometimes, some CSS properties won't be inherited by child elements. In such situations, we can use the `inherit` keyword to explicitly inherit the property from the parent element.

Lets look at an example.

style.css refer **html-28**

```
body {
  font-family: Arial, sans-serif;
  colour: green;
}
h1 {
  font-size: 24px;
  margin: inherit;
}
p{
  font-size: 16px;
  margin: inherit;
}
```

OUTPUT:

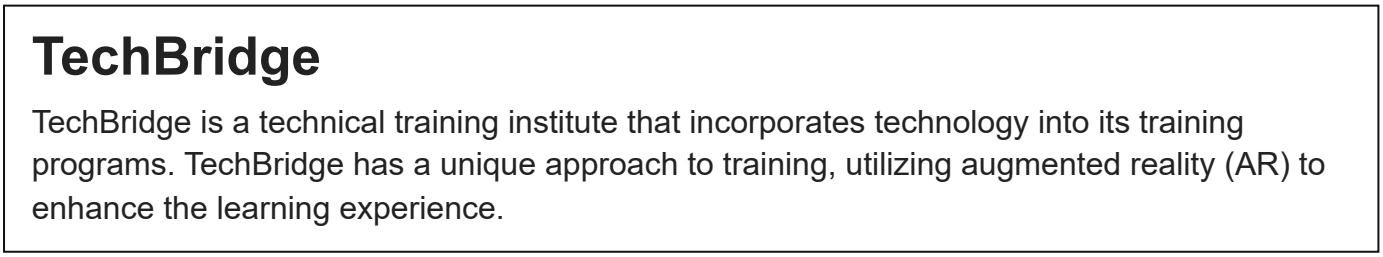


Figure 110: Output of index.html (html-28) with `margin: inherit` on the h1 and p tags — same heading and paragraph, now with the margin inherited from the body.

In this example, we have added the margin property to the h1 and p tags with the value `inherit`. This means that the margin of these elements will be inherited from their parent element, which is the body element in this case.

Typography

Typography is all about style, proportions, and spacing. Good typography is not only important for aesthetic appeal, but it also improves site usability by enhancing text legibility and readability.

Typography is also about emotion. Do the letters complement your content, or do they contradict it? Even when the same text is presented with different typographic styles, it can evoke different feelings in people. Therefore, being deliberate about typography allows us to control these emotions and ensure that the text aligns with the intended message and mood.

Adding fonts:-

Font selection is important choosing the right font has a huge impact on how readers experience a website . The right font can create a strong identity for your brand. Using a font that is easy to read is crucial. Th font adds value to your text. Additionally, its important to choose the correct colour and text size for the font.

Generic Font Families

In CSS, generic font families are broad categories of fonts that can be used as fallbacks if a specific font is not available. These families ensure that text is displayed in a readable manner, even if the exact font isn't supported by the browser or device.

1.**Serif**: Fonts with small lines or embellishments at the ends of characters (e.g., Times New Roman, Georgia).

2. **Sans-serif**: Fonts without the small lines or embellishments (e.g., Arial, Helvetica).

3. **Monospace**: Fonts where each character occupies the same amount of horizontal space (e.g., Courier New, Lucida Console).

4. **Cursive**: Fonts that mimic handwriting or calligraphy (e.g., Comic Sans, Brush Script).

5.**Fantasy**: Decorative fonts that are more playful and less conventional (e.g., Impact, Papyrus).

Using these generic font families ensures that your text remains legible and maintains a certain style, even when the preferred font isn't available.

Google Fonts

Google Fonts is a free library of web fonts that you can use to enhance the typography of your website. It offers a wide variety of font families, styles, and weights, making it easy to find the perfect font for your project. Here's how you can use Google Fonts:

1. **Browse and Select Fonts:**
   Visit the Google Fonts website and browse the available fonts. You can filter fonts by category (Serif, Sans-serif, Display, Handwriting, Monospace) and preview text in various styles.

2. **Choose Font Styles:**
   After selecting a font, you can choose the specific styles (weights and italics) you want to use. Google Fonts provides options like Light, Regular, Bold, and Italic.
3. **Generate Embed Code:**
   Once you've selected your fonts and styles, Google Fonts will generate an embed code. This code

can be added to the <head> section of your HTML document to link the chosen fonts.

4. **Include in CSS:**
   After embedding the fonts, you can use them in your CSS. For example:

HTML

```
<link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
```

CSS

```
body {
  font-family: 'Roboto', sans-serif;
}
```

Font-Styling

Font Weight:-

The font-weight property in CSS is used to set the thickness (weight) of the font. You can specify various numeric values or keywords to achieve different levels of boldness.

Here are some examples:

Numeric Values

- 100 : thin
- 200 : Extra Weight
- 300 : Light
- 400 : Normal(default)
- 500 : Medium
- 600 : Semi Bold
- 700 : Bold
- 800 : Extra Bold
- 900 : Black(Heavy)

Keywords

- normal : Equivalent to `400`
- bold : Equivalent to `700`
- bolder : Bolder than the inherited font weight
- lighter : Lighter than inherited font weight

index.html (html-29)

```html
<html>
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <p class="normal-text">This is normal text.</p>
    <p class="bold-text">This is bold text.</p>
    <p class="extra-bold-text">This is extra bold text.</p>
    <p class="italic-text">This is italic text.</p>
    <p class="fontsize">This text is 24 pixels in size.</p>
    <p class="lineheight">This text has a line height of 1.5.</p>
    <p class="letterspacing">This text has a letter spacing of 4 pixels.</p>
    <p class="texttransform1">This text is transformed to uppercase.</p>
    <p class="texttransform2">This text is transformed to lowercase.</p>
    <p class="texttransform3">This text is transformed to capitalize.</p>
    <p class="textdecoration1">This text is underlined.</p>
    <p class="textdecoration2">This text has a line through it.</p>
    <p class="textdecoration3">This text has an overline.</p>
    <p class="textalign1">This text is centered.</p>
    <p class="textalign2">This text is right-aligned.</p>
    <p class="textalign3">This text is justified.</p>
</body>
</html>
```

style.css refer **html-29**

```css
.normal-text {
  font-weight: normal;
}
.bold-text {
  font-weight: bold;
}
.extra-bold-text {
  font-weight: 800;
}
```

OUTPUT:

This is normal text.

This is bold text.

This is extra bold text.

Figure 111: Output of index.html (html-29) showing `font-weight` — normal, bold and extra bold (800) text. (The listing's other paragraphs are styled by the rules on the following pages.)

In the above output, we can observe that by using the `font-weight` property, we can control the thickness of the text. This allows for varying levels of boldness, enhancing the visual hierarchy and emphasis within the content.

Italics:

To italicize text using CSS, you use the font-style property with the value italic. Here are different ways to achieve this:

style.css refer **html-29**

```
.italic-text {
  font-style: italic;
}
```

OUTPUT:

This is normal text.

This is bold text.

This is extra bold text.

This is italic text.

Figure 112: Output of index.html (html-29) with `font-style: italic` — the fourth line is now italicised.

In the above output, we can observe that by adding the `font-style: italic;` property, we are able to make that line italicized.

Font-Size:

To specify the font size using only pixels (px) in CSS, you can use the font-size property followed by the desired pixel value.

Here are examples of how to use the font-size property:

style.css refer **html-29**

```
.fontsize {
  font-size: 24px;
}
```

OUTPUT:

This is normal text.

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

Figure 113: Output of index.html (html-29) with `font-size: 24px` — the fifth line is displayed at 24 pixels.

Using pixel values (px) for font size allows for precise control over the size of the text. However, keep in mind that pixel-based font sizes may not scale well across different devices and screen sizes, as they do not respond to changes in viewport size or user preferences.

Line-Height:

The line-height property in CSS is used to specify the height of each line of text within an element. It defines the amount of space above and below the text within the element's content area.

Here's an example :

style.css refer **html-29**

```
.lineheight {
  line-height: 1.5;
}
```

OUTPUT:

This is normal text.

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

This text has a line height of 1.5.

Figure 114: Output of index.html (html-29) with `line-height: 1.5` applied to the last line.

The line-height property accepts various units, such as pixels (px), em, percentages (%), or unitless values. A unitless value is a multiplier of the font size. For example, a line-height value of 1.5 means the line height is 1.5 times the font size of the element.

Setting an appropriate line-height can improve the readability and aesthetics of text, ensuring that lines of text are spaced optimally for comfortable reading.

Letter-Spacing:

The letter-spacing property in CSS is used to control the spacing between characters (letters) within a text. It can be used to increase or decrease the space between letters, altering the overall appearance of the text.

Here's how you can use it:

style.css refer **html-29**

```
.letterspacing {
  letter-spacing: 4px;
}
```

OUTPUT:

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

This text has a line height of 1.5.

This text has a letter spacing of 4 pixels.

Figure 115: Output of index.html (html-29) with `letter-spacing: 4px` *— the last line is spread out. (The first line of the original output, "This is normal text.", is cut off at the top of the photograph.)*

Using pixel values (px) for letter spacing allows for precise control over the spacing between characters. Adjusting the letter spacing can affect the readability and visual appearance of text, providing emphasis or enhancing the typography of your content.

Text Transform:

The text-transform property in CSS is used to control the capitalization of text. It allows you to transform the case of the text characters to uppercase, lowercase, capitalize, or revert to its original form.

The text-transform property accepts the following values:

- uppercase: Transforms the text to uppercase.
- lowercase: Transforms the text to lowercase.
- capitalize: Transforms the first character of each word to uppercase.
- none: Specifies no capitalization transformation (default).

Here's how you can use it:

style.css refer **html-29**

```
.texttransform1{
  text-transform: uppercase;
}
.texttransform2{
  text-transform: lowercase;
}
.texttransform3 {
  text-transform: capitalize;
}
```

OUTPUT:

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

This text has a line height of 1.5.

This text has a letter spacing of 4 pixels.

THIS TEXT IS TRANSFORMED TO UPPERCASE.

this text is transformed to lowercase.

This Text Is Transformed To Capitalize.

Figure 116: Output of index.html (html-29) with the three `text-transform` rules — the last three lines appear in UPPERCASE, lowercase and Capitalized Form.

Using the text-transform property, you can easily change the appearance of text by altering its capitalization style, providing consistency and enhancing readability across your web content.

Text-Decoration

The text-decoration property in CSS is used to control the decoration of text, such as underlining, overlining, striking through, or none (removing any decoration).

The text-decoration property accepts the following values:

- none: Specifies no decoration (default).
- underline: Adds an underline below the text.
- overline: Adds a line above the text.
- line-through: Adds a line through the middle of the text.
- blink: Makes the text blink (not widely supported and considered obsolete).

Here's how you can use it:

style.css refer **html-29**

```
.textdecoration1{
  text-decoration: underline;
}
.textdecoration2{
  text-decoration: line-through;
}
.textdecoration3{
  text-decoration: overline;
}
```

OUTPUT:

This is normal text.

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

This text has a line height of 1.5.

This text has a letter spacing of 4 pixels.

THIS TEXT IS TRANSFORMED TO UPPERCASE.

this text is transformed to lowercase.

This Text Is Transformed To Capitalize.

This text is underlined.

~~This text has a line through it.~~

This text has an overline.

Figure 117: Output of index.html (html-29) with the three `text-decoration` rules — the last three lines are underlined, struck through and overlined.

Using the text-decoration property, you can easily add or remove text decorations to enhance the visual appearance of your web content.

Text Alignment:

Text alignment in CSS is controlled using the text-align property. It allows you to specify how text should be aligned within its containing element.

The text-align property accepts the following values:

- left: Aligns text to the left (default).
- right: Aligns text to the right.
- center: Centers text horizontally within its container.
- justify: Stretches the lines of text to align with both the left and right edges of the container, creating even spacing between words.

Here's how you can use it:

style.css refer **html-29**

```
.textalign1{
  text-align: center;
}
.textalign1{
  text-align: right;
}
.textalign1{
  text-align: justify;
}
```

OUTPUT:

This is bold text.

This is extra bold text.

This is italic text.

This text is 24 pixels in size.

This text has a line height of 1.5.

This text has a letter spacing of 4 pixels.

THIS TEXT IS TRANSFORMED TO UPPERCASE.

this text is transformed to lowercase.

This Text Is Transformed To Capitalize.

This text is underlined.

~~This text has a line through it.~~

This text has an overline.

This text is centered

This text is justified

This text is right-aligned

Figure 118: Output of index.html (html-29) with the `text-align` rules — the last three lines are centered, justified (left edge) and right-aligned.

By using the text-align property, you can control the alignment of text elements within your web content, ensuring readability and visual consistency.

Box Model

The box model in CSS describes the layout of elements on a webpage in terms of rectangular boxes. Each element is considered a box, and the box model defines how these boxes are structured, sized, and positioned within the document layout. It consists of four main components: content, padding, border, and margin. Here's a brief overview of each component:

- **Content**: This is the actual content of the element, such as text, images, or other media. It is enclosed by the padding, border, and margin of the box.
- **Padding**: The padding is the space between the content and the border of the box. It provides additional spacing within the box and helps to separate the content from the border.
- **Border**: The border surrounds the padding and content of the box. It defines the outer edge of the box and can have various styles, widths, and colours.
- **Margin**: The margin is the space outside the border of the box. It creates a gap between the element and its neighbouring elements, controlling the layout and spacing of elements on the page.

General Box model

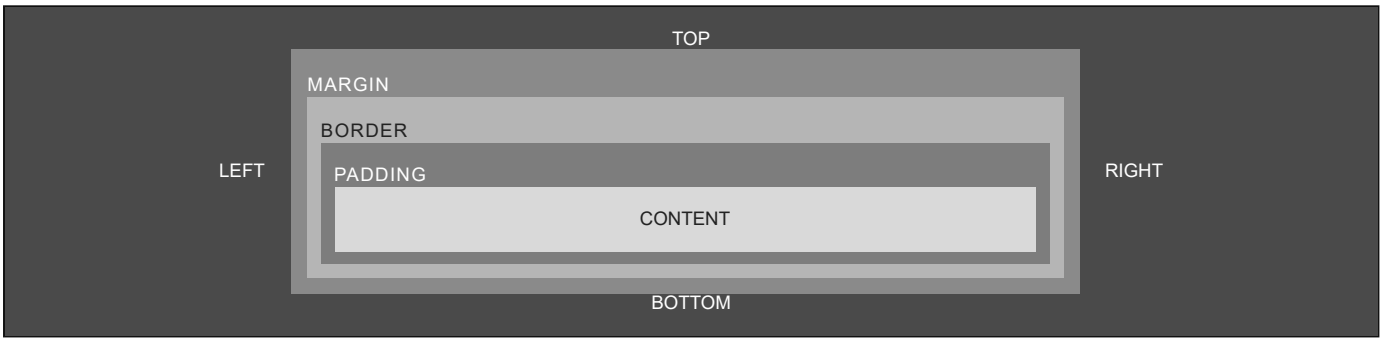


Figure 119: The general box model — the content sits in the centre, surrounded in turn by the padding, the border and the margin, with TOP, RIGHT, BOTTOM and LEFT marked on the four sides.

Padding

index.html (html-30)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>TechBridge</h1>
</body>
</html>
```

style.css refer **html-30**

```
h1{
  /* We can add padding differently like below */
  padding-top: 25px;
  padding-right: 25px;
  padding-bottom: 25px;
  padding-left: 25px;
  /* we can use below shorthand also to give padding */
  padding: 25px 25px 25px 25px;
  /* we can use below shorthand to mention padding when all four sides are equal */
  padding: 25px;
  /* Below shorthand will apply padding in clockwise direction 25 to top, 15 to right then 25 to bottom and 15 to left */
  padding:25px 15px ;
  /* if there are 3 values for padding then the same clockwise will work 4 value will be same as 2 value*/
  padding: 25px 20px 15px;
}
```

OUTPUT:

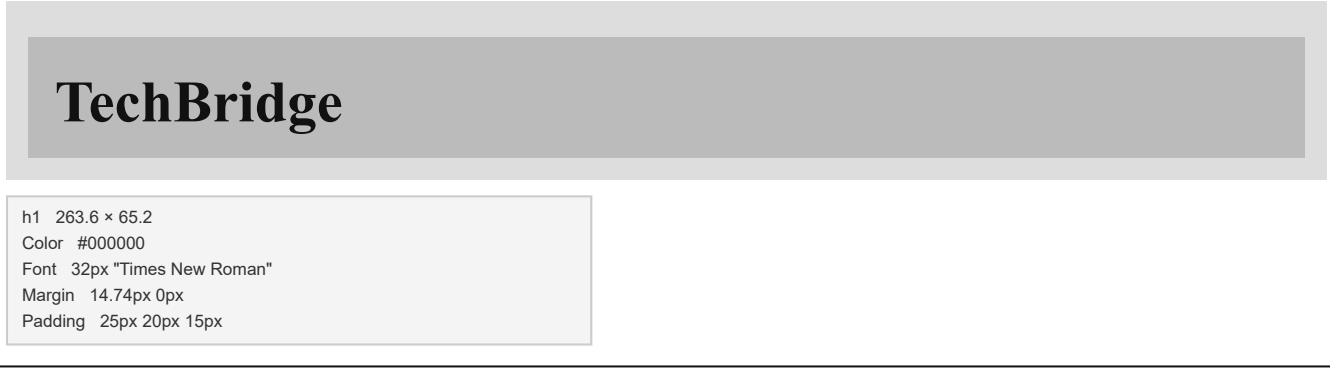


Figure 120: Output of index.html (html-30) with `padding: 25px 20px 15px` — the "TechBridge" heading with the padding area highlighted, and the browser inspector tooltip for the h1 (263.6 × 65.2, colour #000000, font 32px "Times New Roman", margin 14.74px 0px, padding 25px 20px 15px).

BORDER

index.html (html-31)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <p>This is frontend development</p>
</body>
</html>
```

style.css refer **html-31**

```
p{
  padding: 5px;
  border: 3px solid red;
  /* If we want to provide diff border for each side */
  border-top: 3px dashed teal;
  border-right: 3px ridge yellow;
  border-bottom: 3px dotted red;
  border-left: 3px spolid blue;
  /* How can we create a rounded border */
  border-top-right-radius:5px ;
  border-top-left-radius:5px ;
  border-bottom-right-radius:5px ;
  border-bottom-left-radius:5px ;
  /* To apply rounded border this is the shorthand */
  border-radius: 10px;
}
```

OUTPUT:

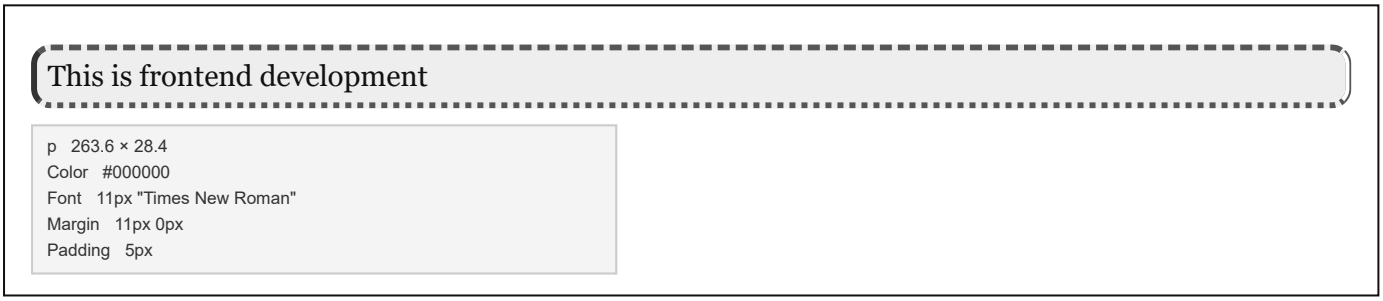


Figure 121: Output of index.html (html-31) — the paragraph "This is frontend development" with a different border style on each side and rounded corners, plus the browser inspector tooltip for the p (263.6 × 28.4, margin 11px 0px, padding 5px).

MARGIN:

index.html (html-32)

```
<!DOCTYPE html>
<html>
<head>
    <title>Document</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1 class="title1">Hello World</h1>
    <h1>Hello World</h1>
</body>
</html>
```

style.css refer **html-32**

```
/* remove all the margin  */
*{
  margin: 0;
}
/* Change the font size */
h1{
  font-size: 60px;
  margin: 40px;
}
/* apply margin for all sides space for 1st h1 */
.title1{
  margin-bottom: 100px;
  margin-top: 100px;
  margin-left :100px;
  margin-right :100px;
  /* Shorthand for margin */
  margin: 100px;
  /* We can also give 4 values */
  margin: 100px 100px 100px 100px;
}
```

OUTPUT:

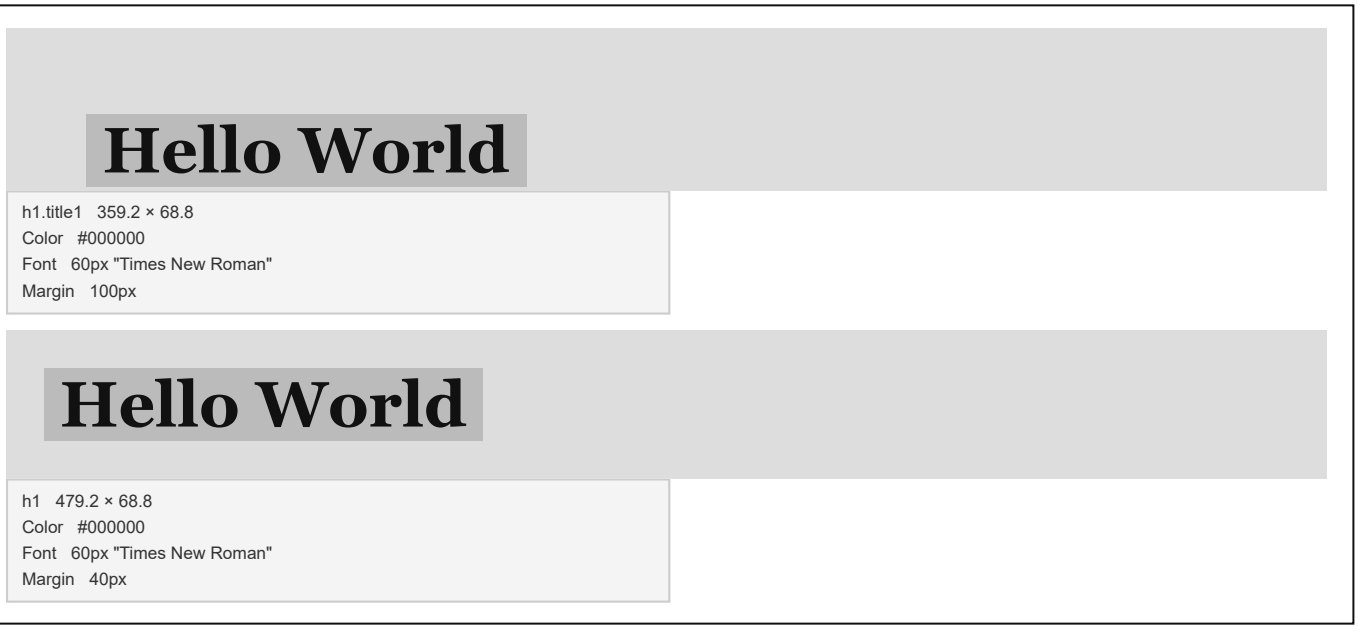


Figure 122: Output of index.html (html-32) — two "Hello World" headings: the first (`h1.title1`) with a 100px margin on all sides, the second with the 40px margin from the `h1` rule; the browser inspector tooltips show the margins.

BOX-SIZING

In CSS, the box-sizing property determines how the total width and height of an element are calculated. Specifically, it defines whether padding and borders should be included in the dimensions or not.

CONTENT BOX (Default):

- This is the default behaviour.
- When you set an element's width or height, it only applies to the content box (the inner area where the actual content resides).

- The padding and border are added to the specified width and height, making the element appear larger than expected.

index.html (html-33)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Content Box Example</title>
</head>
<body>
    <div class="box">TechBridge</div>
</body>
</html>
```

style.css refer **html-33**

```
.box {
  width: 200px;
  height: 200px;
  background-color: red;
}
```

OUTPUT (CONTENT BOX) :

OUTPUT:



Figure 123: Output of index.html (html-33) with the content box — a 200 × 200 red square (grey in the black-and-white photograph) and the inspector tooltip for `div.box`.

BORDER BOX:

- When you set an element's box-sizing to border-box, the padding and border are included in the total width and height.
- This ensures that the specified dimensions account for both content, padding, and border.
- Applying box-sizing: border-box; to all elements is often recommended for a more intuitive sizing experience.

style.css refer **html-33**

```
.box {
  width: 320px;
  height: 50px;
  padding: 10px;
  border: 5px solid gray;
  margin: 0;
}
```

OUTPUT (BORDER BOX) :

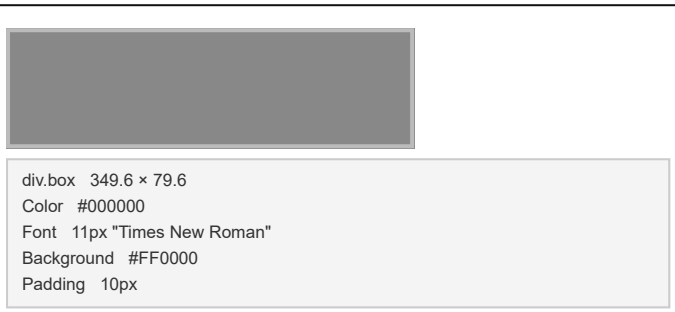


Figure 124: Output of index.html (html-33) for the border-box example — the red box (grey in the black-and-white photograph) with the inspector tooltip for `div.box` *(349.6 × 79.6, padding 10px).*

Flex Box

What is flexbox?

Flexbox (Flexible Box Layout) is inherently a one-dimensional layout model, which means it is designed to lay out items in a single direction, either horizontally or vertically.

Here are the dimensions of layout with respect to Flexbox:

One-Dimensional Layouts

Flexbox operates in one dimension at a time, which can be either:

1. **Horizontal (Row) Layout:** Items are placed in a row from left to right (or right to left in right-to-left languages).
2. **Vertical (Column) Layout:** Items are placed in a column from top to bottom (or bottom to top).

Properties and Their Impact

1. **Horizontal Layout (Row)**

Example for Horizontal Layout:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flexbox Horizontal Layout</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
        }
        .container {
            display: flex;
            flex-direction: row;/*Default flex direction*/
        }
        .item {
            background-color: #3498db;
            color: white;
            padding: 10px;
            margin: 5px;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="item">Item 1</div>
        <div class="item">Item 2</div>
        <div class="item">Item 3</div>
        <div class="item">Item 4</div>
        <div class="item">Item 5</div>
        <div class="item">Item 6</div>
    </div>
</body>
</html>
```

OUTPUT:



Figure 125: Output of the horizontal layout — Item 1 to Item 6 placed side by side in a row (the items are blue, `#3498db`, in the original; grey in the black-and-white photograph).

Main Axis: Horizontal (row direction)

- Main axis will always be along the flex direction.
- The flex direction will always be row.

Cross Axis: Vertical

- Always opposite to the main axis.

2.**Vertical Layout (column)**

Example for Vertical Layout:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flexbox Vertical Layout</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
        }
        .container {
            display: flex;
            flex-direction: column; /* Sets the main axis to be vertical */
            max-width: 100px;

        }
        .item {
            background-color: #3498db;
            color: white;
            padding: 10px;
            margin: 5px;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="item">Item 1</div>
        <div class="item">Item 2</div>
        <div class="item">Item 3</div>
        <div class="item">Item 4</div>
        <div class="item">Item 5</div>
        <div class="item">Item 6</div>
    </div>
</body>
</html>
```

OUTPUT:

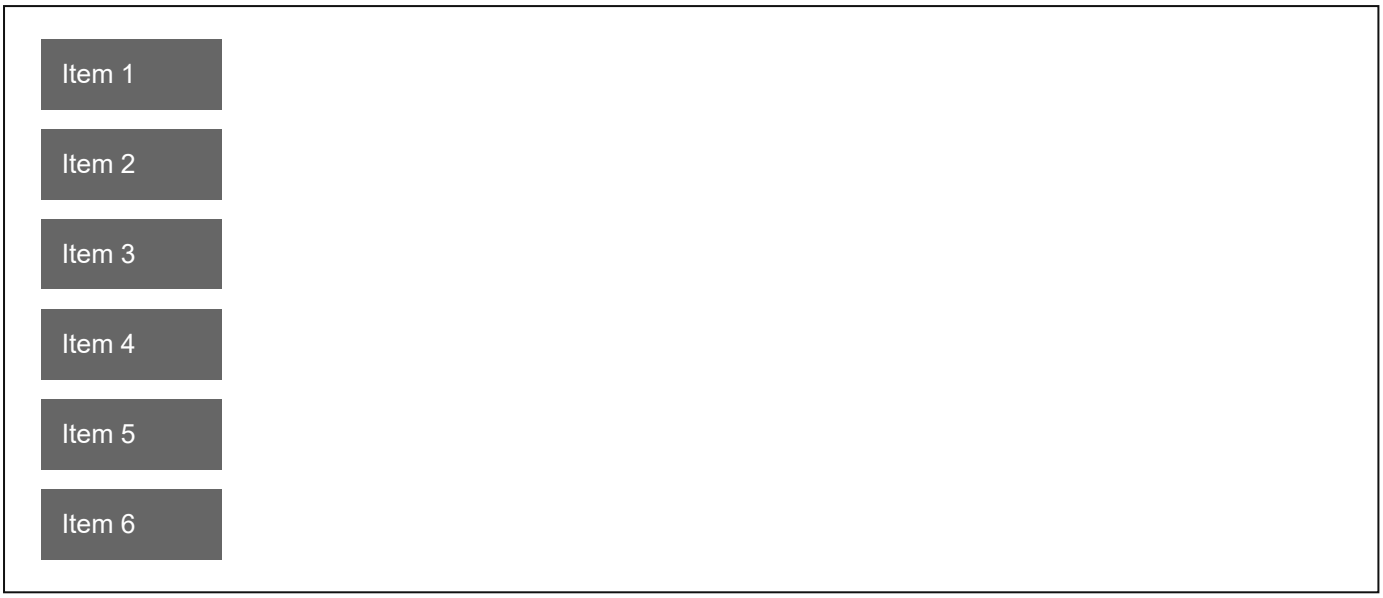


Figure 126: Output of the vertical layout — Item 1 to Item 6 stacked in a column (blue in the original; grey in the black-and-white photograph).

Main Axis: Vertical (column direction)

Cross Axis: Horizontal

Key Flexbox Properties for Layout

1. **flex-direction**

Specifies the direction of the flex container's main axis:

- row: Default value; horizontal left-to-right.
- row-reverse: Horizontal right-to-left.
- column: Vertical top-to-bottom.
- column-reverse: Vertical bottom-to-top.

2. **justify-content**

Aligns items along the main axis:

- flex-start: Default value; items are packed toward the start.
- flex-end: Items are packed toward the end.
- center: Items are centered along the main axis.
- space-between: Items are evenly distributed with space between them.
- space-around: Items are evenly distributed with space around them.
- space-evenly: Items are distributed so that the spacing between any two items (and the space to the edges) is equal.

3. **align-items**

Aligns items along the cross axis:

- stretch: Default value; items stretch to fill the container.
- flex-start: Items are packed toward the start.
- flex-end: Items are packed toward the end.
- center: Items are centered along the cross axis.

- baseline: Items are aligned such that their baselines align.

4. **flex-wrap**

Specifies whether items should wrap onto multiple lines:

- nowrap: Default value; all items are on a single line.
- wrap: Items will wrap onto multiple lines.
- wrap-reverse: Items will wrap onto multiple lines in reverse order.

5. **align-content**

Aligns lines within a flex container when there is extra space on the cross axis:

- flex-start: Lines are packed toward the start.
- flex-end: Lines are packed toward the end.
- center: Lines are packed toward the center.
- space-between: Lines are evenly distributed with space between them.
- space-around: Lines are evenly distributed with space around them.
- stretch: Lines stretch to take up the remaining space.

3.**Two-Dimensional Layout (column)**

Placing elements both in horizontal and vertical direction.

Example for Two Dimension Layout:

index.html (htm-34)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flexbox Two-Dimensional Layout</title>
</head>
<body>
    <div class="container">
        <div class="item"><img src="1.jpg" alt="Image 1"></div>
        <div class="item"><img src="2.jpg" alt="Image 2"></div>
        <div class="item"><img src="3.jpg" alt="Image 3"></div>
        <div class="item"><img src="4.jpg" alt="Image 4"></div>
        <div class="item"><img src="5.jpg" alt="Image 5"></div>
        <div class="item"><img src="6.jpg" alt="Image 6"></div>
        <div class="item"><img src="7.jpg" alt="Image 7"></div>
        <div class="item"><img src="8.jpg" alt="Image 8"></div>
        <div class="item"><img src="9.jpg" alt="Image 9"></div>
        <div class="item"><img src="1.jpg" alt="Image 1"></div>
        <div class="item"><img src="3.jpg" alt="Image 3"></div>
        <div class="item"><img src="5.jpg" alt="Image 5"></div>
    </div>
</body>
</html>
```

style.css refer **html-34**

```css
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4; /* Added background color for better contrast */
}
.container {
    display: flex;
    flex-wrap: wrap; /* Enable wrapping */
    justify-content: space-around; /* Center horizontally */
    align-items: center; /* Center vertically */
    max-width: 1000px; /* Limiting the container width */
    margin: 20px auto; /* centering the container horizontally and adding top margin */
    padding: 10px; /* Adding padding for better spacing */
    background-color: #fff; /* White background for the container */
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); /* Adding shadow for a subtle effect */
}
.item {
    padding: 10px;
    margin: 10px;
    height: 300px;
    width: 200px; /* Set a fixed width for items */
    text-align: center;
    box-sizing: border-box; /* Ensure padding and border are included in the width and height */
    display: flex;
    justify-content: center;
    align-items: center;
}
.item img {
    max-width: 100%;
    max-height: 100%;
    object-fit: cover; /* Ensure the images cover the item space appropriately */
}
```

OUTPUT:

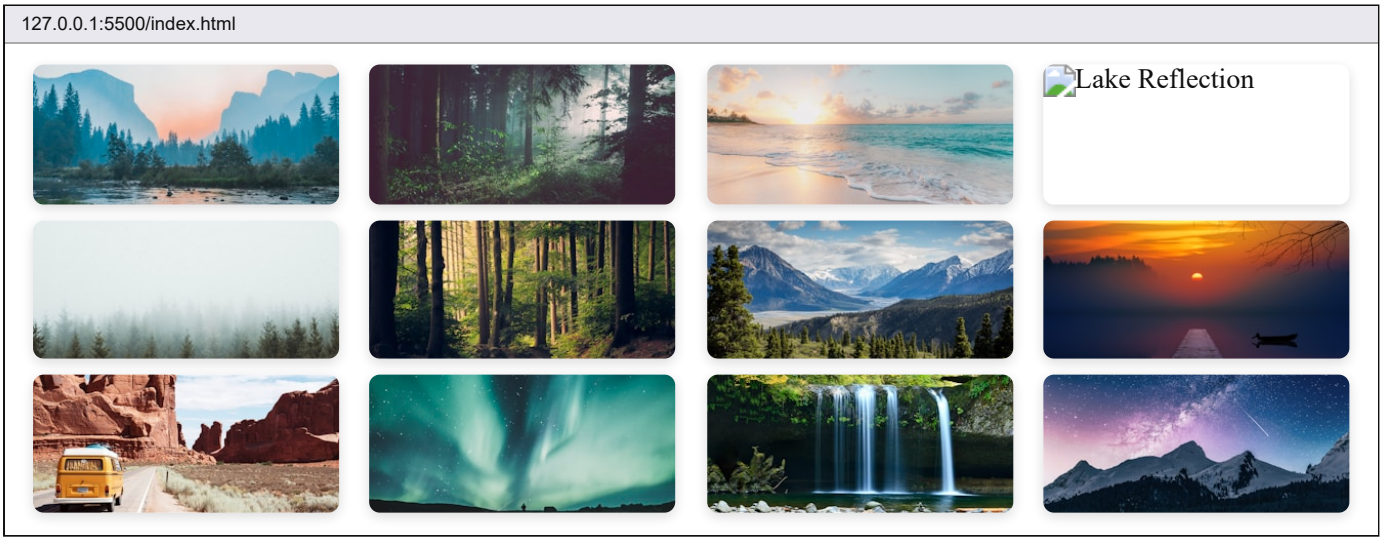


Figure 127: Output of index.html (html-34) — twelve landscape photographs laid out in a responsive wrapping flex container (flex-wrap: wrap; justify-content: space-around), four per row over three rows.

Flexbox simplifies the creation of flexible, one-dimensional layouts by allowing you to easily control the alignment, spacing, and distribution of items along a single axis, making it a powerful tool for responsive design.

CSS Units

CSS units are used to define the sizes, lengths, and positions of elements in CSS. These units can be broadly classified into two categories: absolute units and relative units.

Here's an overview of the most common CSS units:

There are 2 types:

- Absolute Units
- Relative Units

Absolute Units

Absolute units are fixed and are not relative to any other elements. They are useful when you want precise control over the layout.

- px (Pixels)(most commonly used)
    a. A pixel is a single point on the screen.
    b. Example: width: 100px; (This makes the width of the element 100 pixels.)
- pt (Points)
    a. Points are typically used in print media. One point is 1/72 of an inch.
    b. Example: font-size: 12pt; (This sets the font size to 12 points.)
- in (Inches)
    a. Inches are useful for print layouts. One inch is equal to 96 pixels.
    b. Example: width: 2in; (This sets the width of the element to 2 inches.)
- cm (Centimeters)
    a. Centimeters are also used for print layouts. One centimeter is equal to 37.8 pixels.
    b. Example: width: 5cm; (This sets the width of the element to 5 centimeters.)
- mm (Millimeters)
    a. Millimeters are another unit for print layouts. One millimeter is equal to 3.78 pixels.
    b. Example: width: 50mm; (This sets the width of the element to 50 millimeters.)
- pc (Picas)
    a. Picas are used in typography. One pica is equal to 12 points.
    b. Example: font-size: 1pc; (This sets the font size to 1 pica, or 12 points.)

Relative Units

Relative units are based on the size of other elements, making them more flexible and useful for responsive design.

- em
    a. em is relative to the font size of the element itself. 2em means 2 times the current font size.
    b. Example: font-size: 2em; (This sets the font size to twice the size of the parent element's font size.)

- rem (Root em)
    a. rem is relative to the font size of the root element (<html>).
    b. Example: font-size: 1.5rem; (This sets the font size to 1.5 times the size of the root element's font size.)
- % (Percent)
    a. Percentages are relative to the size of the parent element.
    b. Example: width: 50%; (This sets the width to 50% of the parent element's width.)
- vh (Viewport Height)

    a. vh is relative to 1% of the height of the viewport (browser window).
    b. Example: height: 50vh; (This sets the height to 50% of the viewport height.)
- vw (Viewport Width)
    a. vw is relative to 1% of the width of the viewport.
    b. Example: width: 50vw; (This sets the width to 50% of the viewport width.)
- Vmin
    a. vmin is relative to 1% of the viewport's smaller dimension (either width or height).
    b. Example: font-size: 5vmin; (This sets the font size to 5% of the smaller dimension of the viewport.)
- Vmax
    a. vmax is relative to 1% of the viewport's larger dimension (either width or height).
    b. Example: font-size: 5vmax; (This sets the font size to 5% of the larger dimension of the viewport.)
- Ch
    a. ch is relative to the width of the "0" (zero) character of the font.
    b. Example: width: 10ch; (This sets the width to 10 times the width of the "0" character.)
- ex
    a. ex is relative to the x-height of the current font (typically the height of the lowercase "x").
    b. Example: font-size: 2ex; (This sets the font size to 2 times the x-height of the font.)
- fr (Fraction)
    a. fr is used in CSS Grid to allocate a fraction of the available space.
    b. Example: grid-template-columns: 1fr 2fr; (This creates a grid with two columns, where the second column is twice as wide as the first.)

Here is a complete example demonstrating the use of both absolute and relative units:

index.html (htm-35)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CSS Units Example</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="absolute-units">
        Absolute Units Box
    </div>
    <div class="relative-units">
        Relative Units Box
    </div>
</body>
</html>
```

style.css refer **html-35**

```
body{
  font-family: Arial, sans-serif;
}
.absolute-units {
  width: 200px;          /* pixels */
  height: 5cm;           /* centimeters */
  border: 1in solid black; /* inches */
  margin: 10mm;          /* millimeters */
  font-size: 24pt;       /* points */
  padding: 1pc;          /* picas */
  background-color: lightblue;
}
.relative-units {
  width: 50%;            /* percent */
  height: 50vh;          /* viewport height */
  font-size: 2em;        /* em */
  padding: 1rem;         /* rem */
  margin: 5vmin;         /* viewport minimum */
  line-height: 2ch;      /* character height */
  background-color: lightpink;
}
```

OUTPUT:



Figure 128: Output of index.html (htm-35) — the "Absolute Units Box" (light blue with a thick black 1in border) above the much wider "Relative Units Box" (light pink; grey in the black-and-white photograph).

- The .absolute-units class uses various absolute units to define the size, border, margin, font size, and padding.
- The .relative-units class uses various relative units to define the size, font size, padding, margin, and line height.

Grid

CSS Grid is a powerful layout system that allows you to create complex and flexible web layouts. It's different from other layout systems like Flexbox because it works with both rows and columns simultaneously.

Basic Concepts of CSS Grid

- **Grid Container:** This is the element that you apply the display: grid; property to. It becomes the grid container.
- **Grid Items:** These are the direct children of the grid container. They are placed inside the grid.
- **Grid Lines:** These are the lines that divide the grid into rows and columns.
- **Grid Tracks**: These are the spaces between two adjacent grid lines, essentially the rows and columns.
- **Grid Cell:** The single unit of the grid formed by the intersection of a row and a column.
- **Grid Area**: A rectangular space made up of one or more grid cells.
- **Defining Columns and Rows:** You can define the number of columns and rows using the grid-template-columns and grid-template-rows properties. Each value specifies the size of a column or row.
- **Placing Items in the Grid:** You can place items in specific grid cells using the grid-column and grid-row properties.
- **Grid Gap:** The gap property sets the space between rows and columns.

Advanced Grid Properties

1. **grid-template-areas**

You can name areas of the grid and place items in those areas.

2. **grid-auto-rows and grid-auto-columns**

These properties define the size of rows and columns that are created implicitly.

Comprehensive example to understand Grid

index.html (htm-36)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CSS Grid Example</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Grid container -->
    <div class="container">
        <!-- Header item -->
        <div class="header">Header</div>
        <!-- Sidebar item -->
        <div class="sidebar">Sidebar</div>
        <!-- Main content item -->
        <div class="main">Main Content</div>
        <!-- Footer item -->
        <div class="footer">Footer</div>
    </div>
</body>
</html>
```

style.css refer **html-36**

```css
/* Grid container with specified columns and rows */
.container {
    display: grid; /* Define this element as a grid container */
    grid-template-columns: 1fr 3fr; /* Two columns: 1 fraction and 3 fractions of available space */
    grid-template-rows: auto 200px auto; /* Three rows: auto-height, fixed height, and auto-height */
    grid-template-areas:
        "header header" /* Header spans across two columns */
        "sidebar main" /* Sidebar and main content */
        "footer footer"; /* Footer spans across two columns */
    gap: 10px; /* 10px gap between grid items */
    padding: 10px; /* Padding around the grid container */
}
/* Header area styling */
.header {
    grid-area: header; /* Place this item in the "header" area */
    background-color: lightblue; /* Background color for the header */
    text-align: center; /* Center align text */
    padding: 20px; /* Padding inside the header */
}
/* Sidebar area styling */
.sidebar {
    grid-area: sidebar; /* Place this item in the "sidebar" area */
    background-color: lightgreen; /* Background color for the sidebar */
    padding: 20px; /* Padding inside the sidebar */
}
/* Main content area styling */
.main {
    grid-area: main; /* Place this item in the "main" area */
    background-color: lightcoral; /* Background color for the main content */
    padding: 20px; /* Padding inside the main content */
}
/* Footer area styling */
.footer {
    grid-area: footer; /* Place this item in the "footer" area */
    background-color: lightgoldenrodyellow; /* Background color for the footer */
    text-align: center; /* Center align text */
    padding: 20px; /* Padding inside the footer */
```

OUTPUT:

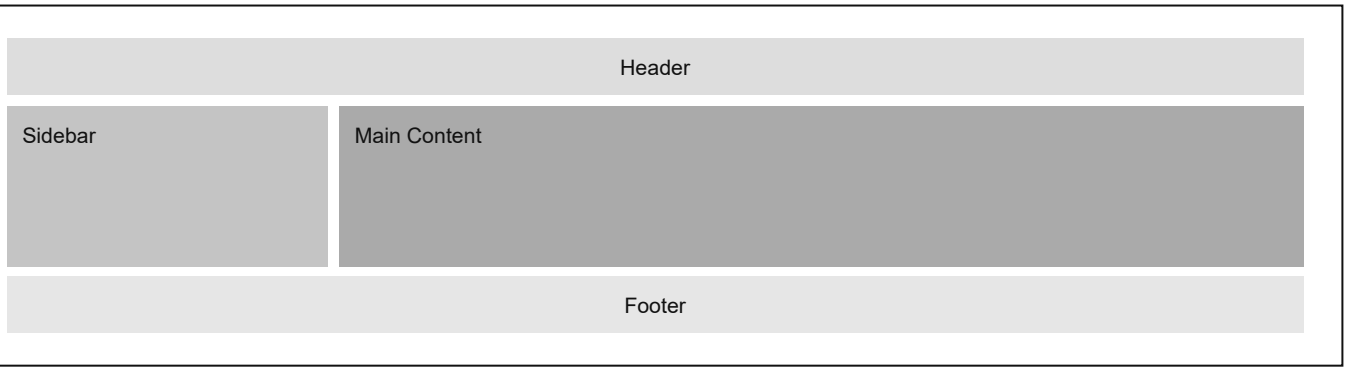


Figure 129: Output of index.html (htm-36) with the grid style sheet — the header and the footer span both columns, while the sidebar (1fr) and the main content (3fr) share the middle row. (The four boxes are light blue, light green, light coral and light goldenrod yellow in the original; grey in the black-and-white photograph.)

Responsiveness in CSS

What is Responsiveness?

Responsiveness refers to the ability of a web page to adapt its layout and content to different screen sizes and devices. A responsive design ensures that a website looks and works well on desktops, tablets, and mobile phones.

Why is Responsiveness Important?

- **User Experience**: Provides a better user experience across all devices.
- **SEO**: Google prioritizes mobile-friendly websites in search results.
- **Accessibility**: Makes websites accessible to a broader audience.

Key Concepts in Responsive Design:

1. **Media Queries:** Media queries allow you to apply different styles based on the characteristics of the user's device, such as screen width, height, resolution, etc.
2. **Flexible Grid Layouts:** Using grid or flexbox layouts that adapt to different screen sizes.
3. **Fluid Images and Media:** Ensuring images and other media scale with the size of the viewport.
4. **Viewport Meta Tag:** Setting the viewport to control the layout on mobile browsers.

Comprehensive example to understand Responsiveness:

index.html (htm-37)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive CSS Grid Example</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Grid container -->
    <div class="container">
        <!-- Header item -->
        <div class="header">Header</div>
        <!-- Sidebar item -->
        <div class="sidebar">Sidebar</div>
        <!-- Main content item -->
        <div class="main">Main Content</div>
        <!-- Footer item -->
        <div class="footer">Footer</div>
    </div>
</body>
</html>
```

style.css refer **html-37**

```css
/* Basic styling for the grid container */
.container {
    display: grid; /* Define this element as a grid container */
    grid-template-columns: 1fr 3fr; /* Two columns: 1 fraction and 3 fractions of available space */
    grid-template-rows: auto 200px auto; /* Three rows: auto-height, fixed height, and auto-height */
    grid-template-areas:
        "header header" /* Header spans across two columns */
        "sidebar main" /* Sidebar and main content */
        "footer footer"; /* Footer spans across two columns */
    gap: 10px; /* 10px gap between grid items */
    padding: 10px; /* Padding around the grid container */
}
/* Header area styling */
.header {
    grid-area: header; /* Place this item in the "header" area */
    background-color: lightblue; /* Background color for the header */
    text-align: center; /* Center align text */
    padding: 20px; /* Padding inside the header */
}
/* Sidebar area styling */
.sidebar {
    grid-area: sidebar; /* Place this item in the "sidebar" area */
    background-color: lightgreen; /* Background color for the sidebar */
    padding: 20px; /* Padding inside the sidebar */
}
/* Main content area styling */
.main {
    grid-area: main; /* Place this item in the "main" area */
    background-color: lightcoral; /* Background color for the main content */
    padding: 20px; /* Padding inside the main content */
}
/* Footer area styling */
.footer {
    grid-area: footer; /* Place this item in the "footer" area */
    background-color: lightgoldenrodyellow; /* Background color for the footer */
    text-align: center; /* Center align text */
    padding: 20px; /* Padding inside the footer */
}
/* Media query for devices with a max width of 600px */
@media (max-width: 600px) {
    .container {
        grid-template-columns: 1fr; /* Single column layout */
        grid-template-rows: auto; /* Auto height for all rows */
        grid-template-areas:
            "header" /* Header spans one column */
            "main" /* Main content spans one column */
            "sidebar" /* Sidebar spans one column */
            "footer"; /* Footer spans one column */
    }
    .sidebar {
        order: 2; /* Change order to appear after main content */
    }
}
/* Media query for devices with a max width of 900px */
@media (max-width: 900px) {
    .container {
        grid-template-columns: 1fr 2fr; /* Adjust column proportions */
        grid-template-rows: auto auto auto; /* Auto height for all rows */
        grid-template-areas:
            "header header" /* Header spans across two columns */
            "main main" /* Main content spans two columns */
            "sidebar footer"; /* Sidebar and footer on the same row */
    }
}
```

OUTPUT:

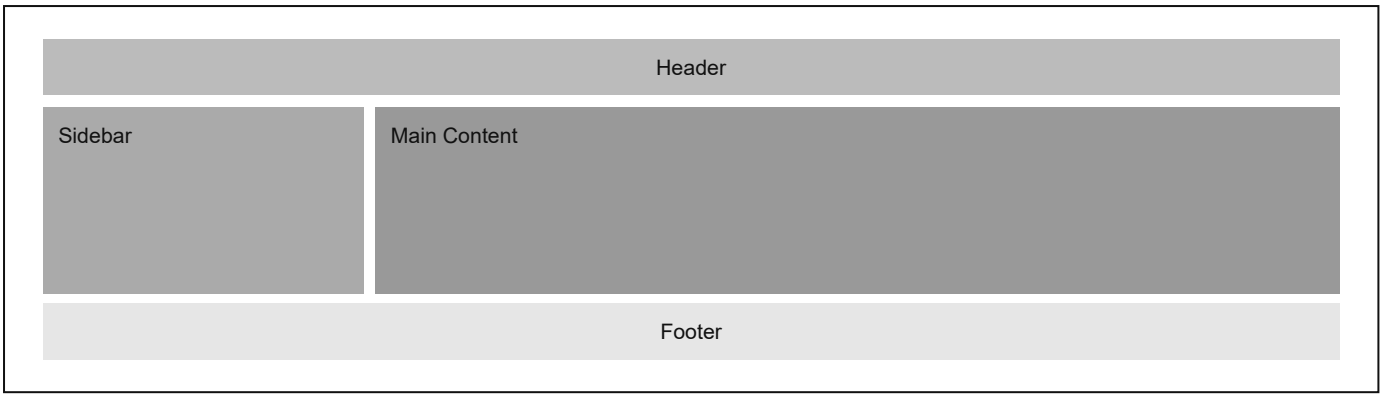


Figure 130: Output of index.html (htm-37) on a large screen — the default grid layout: the header and the footer span both columns, with the sidebar and the main content in the middle row. (Colours are grey in the black-and-white photograph.)

This setup creates a responsive design that adapts to different screen sizes:

- On small screens (max-width 600px), the layout switches to a single column.
- On medium screens (max-width 900px), the layout adjusts to two columns with a different structure.
- On large screens, the default grid layout is applied.

This example shows the key ideas and methods for creating a responsive design using CSS. It covers media queries, flexible grid layouts, and fluid images. The comments in the code explain each part clearly, making it easier for beginners to understand and use responsive design in their projects.

Summary

SUMMARY

CSS SYNTAX

<style>-----</style>

TYPES OF CSS

1. Inline CSS: If we apply styling by using the style attribute we call it as inline CSS.
2. Internal CSS: If we use a <style> tag and apply the styling, we call it as Internal CSS.
3. External CSS: If we create a separate file with the css extension and link that file to HTML then we call it as external CSS.

BASIC CSS PROPERTIES

1. color: Used to change the color of content.
2. text-align: Used to align the content in required place.
3. font-family: Used to change the font style.
4. background-color: Used to give the background color.
5. font-size: Used to change the font size.
6. Height: Used to change the height of image, video etc
7. Width: Used to change the width of image, video etc
8. font-weight: Used to make the font bold.

SELECTORS

1. element selector: Using the element name if we apply the styling to the content we call it as element selector.
   Example:
   ```
   p{
     Property: value;
   }
   ```
2. id selector: Using the id attribute value if we apply the styling to the content we call it as id selector. The suffix should be #.
   Example:
   ```
   #p{
     Property: value;
   }
   ```
3. class selector: Using the class attribute value if we apply the styling to the content we call it as id selector. The suffix should be .(dot operator).
   Example:
   ```
   .p{
     Property: value;
   }
   ```

4. group selector: If we want to apply the same styling to multiple elements we can group all of them as one and apply the styling.
   Example:
   ```
   table,th,td{
     Property: value;
   }
   ```
5. universal selector: It is used if we want to apply same styling for all the elements in a webpage.
   Example:
   ```
   *{
     Property: value;
   }
   ```
6. pseudo-element selectors: We have different types of pseudo-element selectors, for all of them the suffix should be :: (double colon).
   1. ::before: Used if we want to place any content before the contents.
   2. ::after: Used if we want to place any content after the contents.
   3. ::first-letter: Used if we want to apply the styling only for first letter of the content.
   4. ::first-line: Used if we want to apply the styling only for first line of the paragraph.
7. pseudo-class selector: We have different types of pseudo-element selectors, for all of them the suffix should be : (single colon).
   1. :link: Used if we want to change the color of unvisited link.
   2. :visited: Used if we want to change the color of visited link.
   3. :active: Used if we want to change the color of active link
   4. :hover: Used to apply the styling as soon as we keep the mouse on the content.

CSS TABLE

1. border: Used to give the border to the content.
2. border-collapse: Used to combine or merge the border as one
3. nth child(even/odd): Used to apply the styling to specific row.

BORDER

1. border: Used to give the border.
2. border-top: Used to give different border only to top border.
3. border-bottom: Used to give different border only to bottom border.
4. border-left: Used to give different border only to left border.
5. border-right: Used to give different border only to right border.

** **: This tag is used to divide the webpage into different sections.

****: This tag is used to give different style to the each content without breaking the lines.

NAVIGATION BAR

We have two different types of navigation one is horizontal and vertical navigation bar.

1. text-decoration: Used to remove the underline for link or give some other style.
2. float: Used to float the element to left
3. list-style-type: Used to give different style for list items for changing from square to circle etc.

SHADOWING

1. Text shadow: Used to give the shadows for the text by using the text-shadow css property.
2. Box shadow: Used to give the shadows for the box by using the box-shadow.

ANIMATIONS

1. @keyframes: It should be used whenever we create the animations and it is called as decorator.
2. animation-name: used to give the animation name.
3. animation-duration: Used to give the animation duration as how long it should run.
4. Animation-iteration-count: Used to give the count as how many times the animation should repeat.